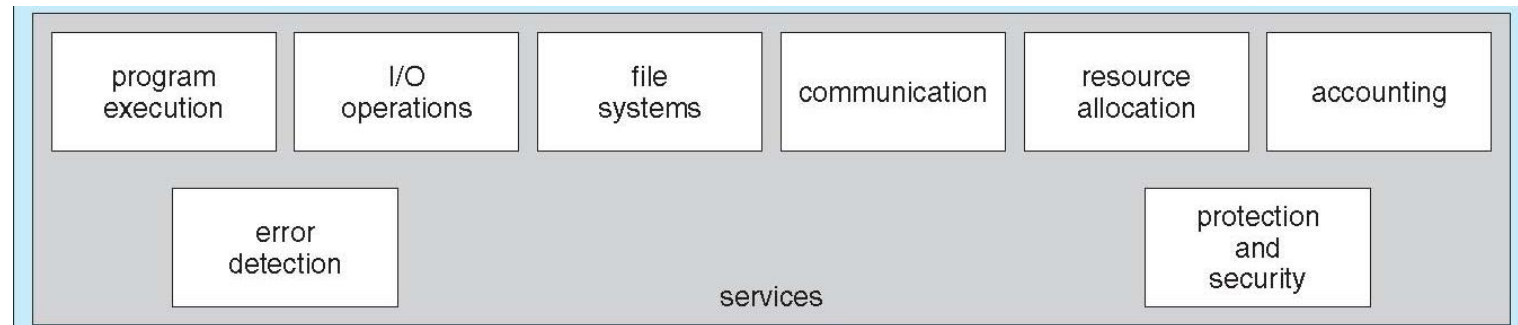


Sistem Programlama

DR. ÖĞR. ÜYESİ ABDULLAH SEVİN

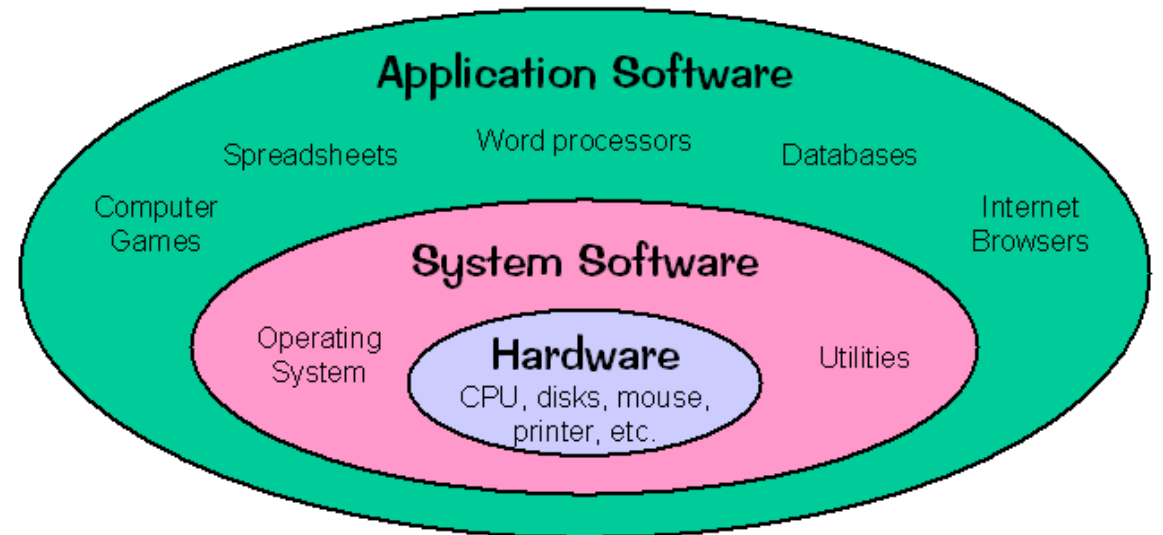
Sistem Programlama nedir?

- **Sistem programlama:** program geliştirme için sistem araçlarının kullanılmasıdır.
- Programlama bilgilerine ek olarak, *bilgisayar mimarisi ve işletim sistemi* bilgisi
- **Uygulama programları** ile direkt olarak kullanıcıya servisler sağlanırken,
Sistem programları ile *uygulama programlarına servisler sağlanır.*
- Sistem programlamada, işletim sistemi servisleri ile etkileşim kuran programlar yazılır.



Sistem Programlama

- **Bilgisayar;** donanım, sistem yazılımı ve uygulama yazılımı olmak üzere üç ana parçaya ayrılabilir.
- **Donanım ve sistem yazılımı,** uygulama programlarını çalıştırmak için kullanılır.
- **Sistem yazılımı,** donanım ve uygulama yazılımları arasında bir **soyutlama** katmanı oluşturur.



Sistem Programlama

- Sistem yazılımı, bilgisayar donanımının çalıştırılması için kullanılır.
- Sistem yazılımı, bir bilgisayar sisteminin donanım bileşenlerini **kontrol** etmek, **bütünleştirmek** ve **yönetmekten** sorumludur,
- böylece diğer yazılımlar ve sistem kullanıcıları, veri aktarımı gibi düşük seviyeli ayrıntılarla ilgilenmek zorunda kalmazlar. (Bellekten diske kopyalama veya bir ekrana yazı yazdırma vs.)

Sistem Programlama

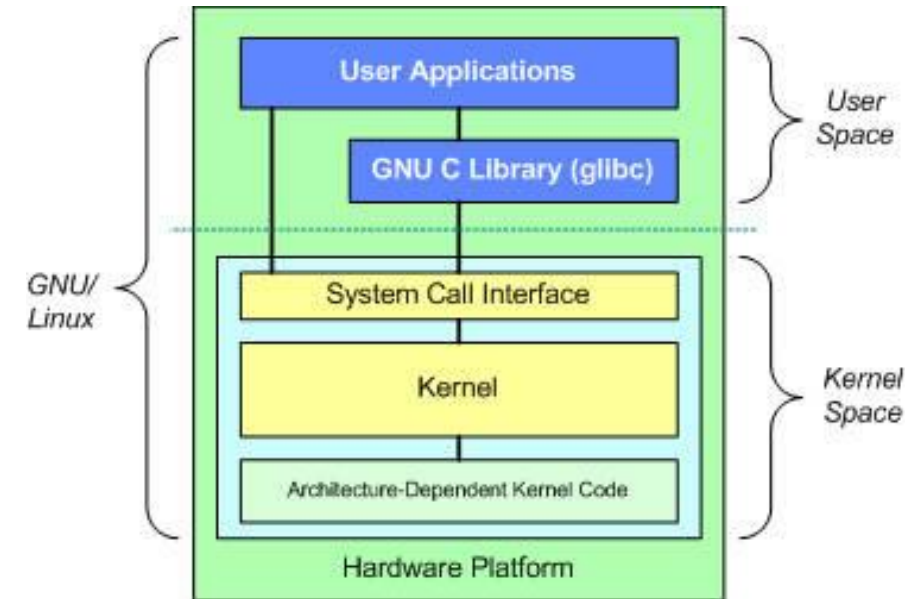
- Genel olarak, sistem yazılımı bir işletim sisteminden ve disk biçimlendiriciler, dosya yöneticileri, görüntü yöneticileri, metin düzenleyiciler, kullanıcı kimlik doğrulaması (oturum açma) ve yönetim araçları ve ağ ve cihaz kontrol yazılımı gibi bazı temel yardımcı programlardan oluşur.
- Genelde kullanıcılar, arka planda çalıştığı için sistem yazılımı ile etkileşime girmezler.

Hedefler

1. Bir işletim sisteminin temellerini kullanıcı açısından anlamak.
2. Neden C'de programlama yaptığımızı anlayacak kadar iyi C programlamak.
3. Sistem programlama hakkında bilgili olmak.
4. Temel tek işlemcili bilgisayar organizasyonunu anlamak.
5. Sistem hakkında donanımsal bilgi sahibi olmak
6. Donanımı gerektiğinde kontrol edebilmek

Sistem Programlama

- Sistem kütüphanelerinin oluşturduğu **soyutlama katmanı (abstraction)** ile bir fonksiyonu (Ör: printf) donanımın detaylarını bilmeden kullanabiliriz.
- Yanda verilen Linux mimarisinde görülen GNU C kütüphanesi gibi, bir sistem benzer kütüphaneleri içerdiği sürece uygulama o sistem üzerinde kullanılabilir.
- Sistem araçlarının kullanımı standartların oluşmasını sağlar ve böylece geliştirilen programlar diğer bilgisayarlara kolayca transfer edilir.



Neden C dili

- ❑ **Sistem programlamada C kullanılması sebepleri:**
- ❑ C programı; işletim sistemleri, aygıt sürücüleri, ağ sunucuları gibi uygulamalarda kullanılarak modern bilgisayarın temelini oluşturur.
- ❑ C programlama **en az soyutlamaya** sahiptir. Bundan dolayı donanıma daha yakındır.
- ❑ Birçok C ifadesi direkt olarak makine koduna dönüştürülebilir.
- ❑ C programlamada hafızaya işaretçiler (pointers) aracılığı ile erişilebilir ve böylece sistemin parçalarına ulaşma imkanı sağlanır.

Neden C

- ❑ Yalnızca dilin veya sistemin mevcut olduğu birçok durumda kullanılır.
 - ❑ Küçük, gömülü sistemler vs.
- ❑ "Üst düzey dilleri" desteği olmayan birçok "düşük düzey" durum var.
 - ❑ İşletim sistemleri, gerçek zamanlı sistemler, sürücüler

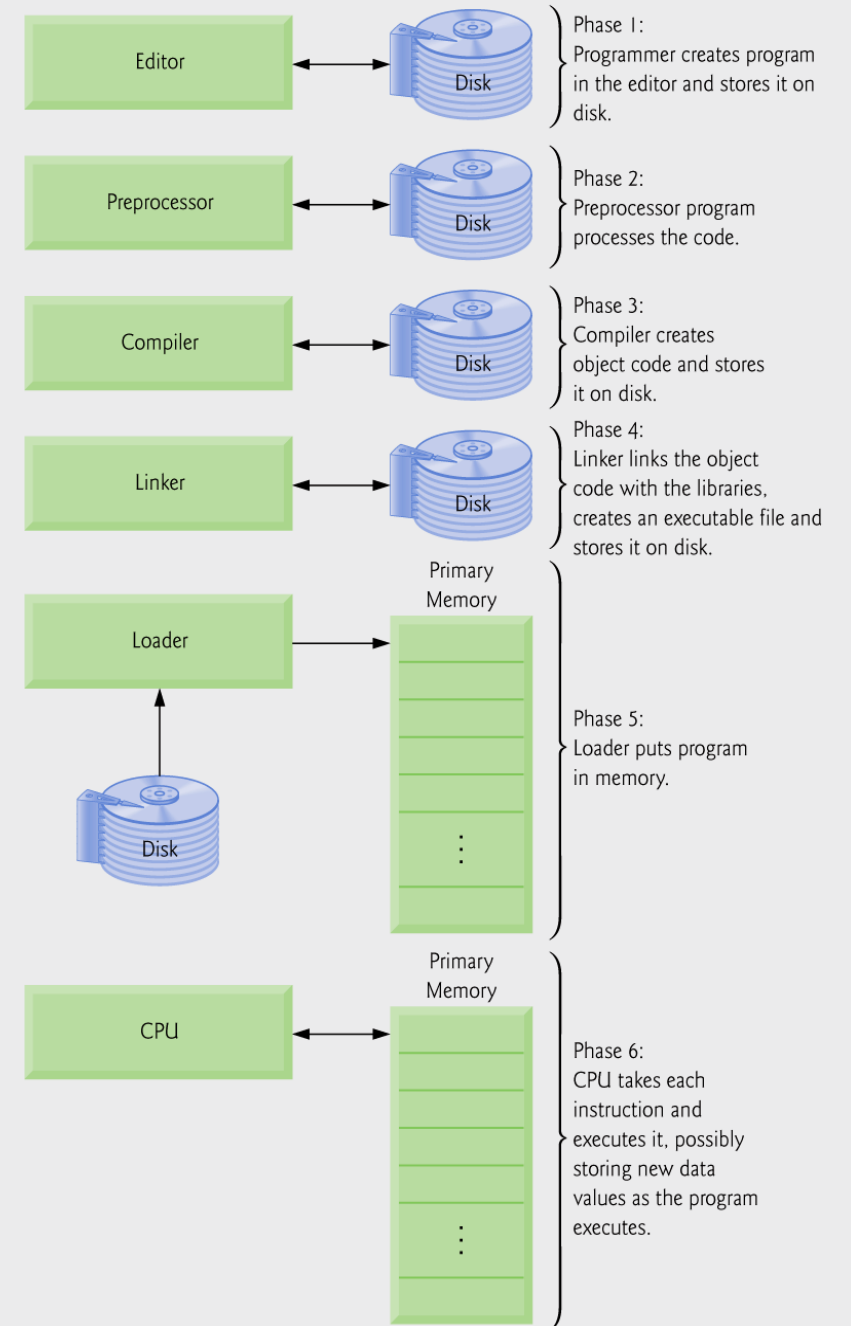
C prog. Geliştirme ortamı

Standard Adımlar

- Düzenleme
- Önışlemci
- Derleme
- Bağlama
- Yükleme
- Yürütme

Deitel & Deitel

© 2007 Pearson Ed -All rights reserved



C prog. Geliştirme ortamı

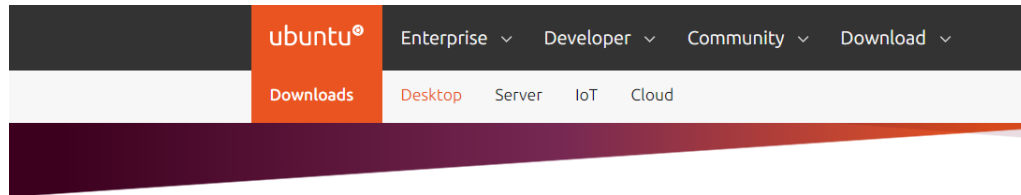
- Edit: Düzenleme
- Önışlemci: Derleme sürecinde ayrı bir adımdır. Basit bir ifadeyle, bir C Ön İşlemcisi yalnızca bir **metin değiştirme** aracıdır ve derleyiciye gerçek derlemeden önce gerekli ön işlemeyi yapmasını söyler. Tüm önışlemci komutları bir kare simgesiyle (#) başlar. (#define,..)
- Derleme
- Bağlama
- Yükleme
- Yürütme

Neden Linux?

- ❑ **Açık kaynak** olması sebebiyle, sistem programlama dersinde Linux tabanlı işletim sistemini tercih edilmektedir.
- ❑ Linux tabanlı işletim sisteminin çalışmasıyla ilgili detaylar incelenebilir. **Parçalar eklenebilir veya değiştirilebilir.**
- ❑ Ticari olmamasından dolayı kapalı bir sistem değildir.

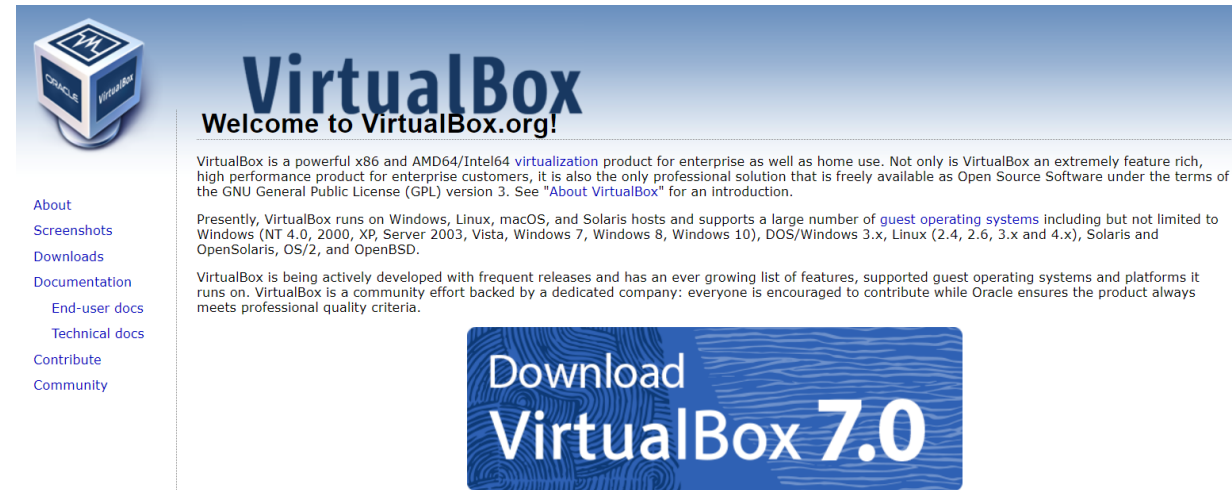
İşletim Sistemi

- ❑ Ders ile ilgili uygulamaları gerçekleştirmek için Linux tabanlı Ubuntu İşletim sistemini kullanabilirsiniz.
- ❑ Farklı İşletim Sistemi üzerine kuracak iseniz Sanal Makine kullanmalısınız. (Örn: Windows üzerinde Oracle VM VirtualBox)



Download Ubuntu Desktop

The open-source desktop operating system that powers millions of PCs and laptops around the world. Find out more about Ubuntu's features and how we support developers and organisations below.

[Ubuntu Desktop homepage](#)[Visit the Ubuntu Desktop blog ›](#)

Konular

Derse ait temel başlıklar aşağıdaki gibi sıralanabilir:

- C programlama dili hakkında temel bilgiler
- C işaretçiler, malloc, stringler, vb...
- Dosyalar ve izin dosyaları, Sinyaller
- Bağlantılar (linkler), Shell yönlendirme (Shell Redirection)
- Dosyadan okuma/yazma (File I/O), sistem çağrıları ve tampon bellek (buffer) kullanımı
- Sistem çağrıları ve giriş çıkış
- Simgesel dil (Assembly) (yerel değişkenler, fonksiyonlar, dallanma)
- Prosesler (süreçler) ve ilgili sistem çağrıları (fork, exec, dup, pipe)
- Prosesler arası iletişim, sinyaller

Değerlendirme

1. Ara Sınav	%60
1. Kısa Sınav	%10
1. Proje / Tasarım	%30
1. Final	%50

Kaynaklar

<http://web.eecs.utk.edu/~huangj/cs360/index.html>

CS360 -- Systems Programming

Jian Huang --- Fall 2022

CS360 LINKS

- [General Information](#)
- [Syllabus](#)
- Homeworks
- [Labs](#)
- [Lecture Notes](#)
- [The Lab Home Page](#) (maintained by the TA's)

INTRODUCTION AND CLASS GOALS



THE UNIVERSITY OF
TENNESSEE
KNOXVILLE

Kaynaklar

http://web.eecs.utk.edu/~jplank/plank/classes/cs360/lecture_notes.html

CS360 -- Systems Programming

James S. Plank

Lecture Notes

Although you can read the notes here, I have put them on bitbucket, and you can grab them to compile and use on your own machine. I strongly suggest that you do so, so that you have all of the programs on your own machine, do:

```
UNIX> git clone https://jimplank@bitbucket.org/jimplank/cs360-lecture-notes.git
```

The lecture notes will be in the directory **cs360-lecture-notes**. You should occasionally do a "git pull" in that directory, to make sure that your notes are up to date.

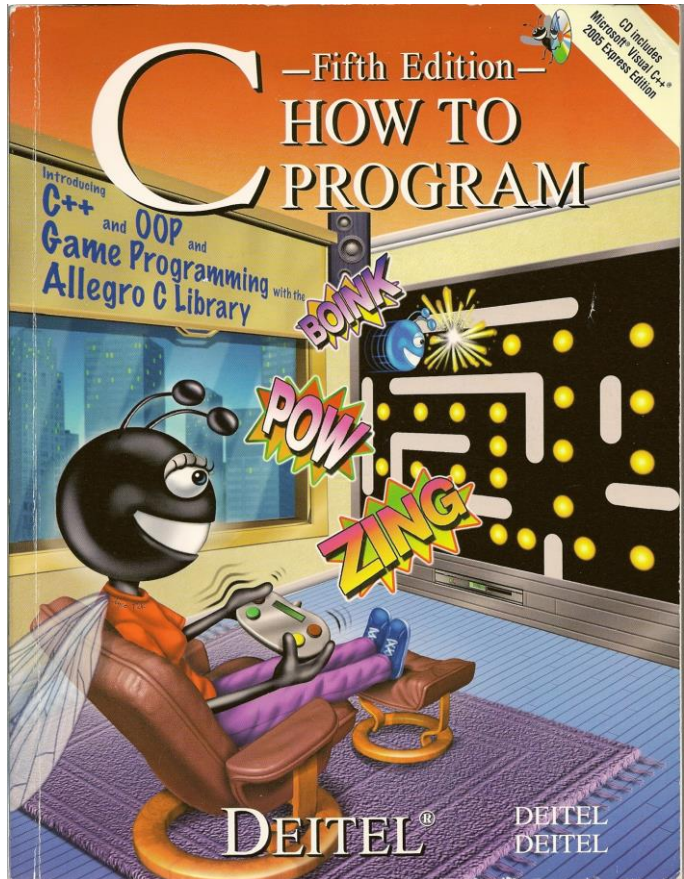
Lecture notes on bitbucket

- [Lecture 1:](#) "C Stuff 1:" Getting Started with C, Scalar Types and Aggregate Types
- [Lecture 2:](#) "C Stuff 2:" Pointers, Casting, Malloc, Segmentation Violations and Bus Errors
- [Lecture 3:](#) Pointer Arithmetic (Small Lecture)
- [Lecture 4:](#) Strings in C
- [Lecture 5a:](#) Libfdr -- The Code
- [Lecture 5b:](#) Libfdr -- Fields
- [Lecture 5c:](#) Libfdr -- Jvals
- [Lecture 5d:](#) Libfdr -- Dllist: Doubly-Linked Lists
- [Lecture 5e:](#) Libfdr -- JRB: Red-Black Trees
- [Lecture 6:](#) Some Basic Terminology
- [Lecture 7:](#) Introduction to System Calls and I/O
- [Lecture 8:](#) Cat and Buffering
- [Lecture 9:](#) Links
- [Lecture 10:](#) Sh Redirection
- [Lecture 11:](#) Stat and Opendir/Readdir/Closedir



THE UNIVERSITY OF
TENNESSEE
KNOXVILLE

Ek Kaynak



Deitel & Deitel, 5th ed.
C & C++

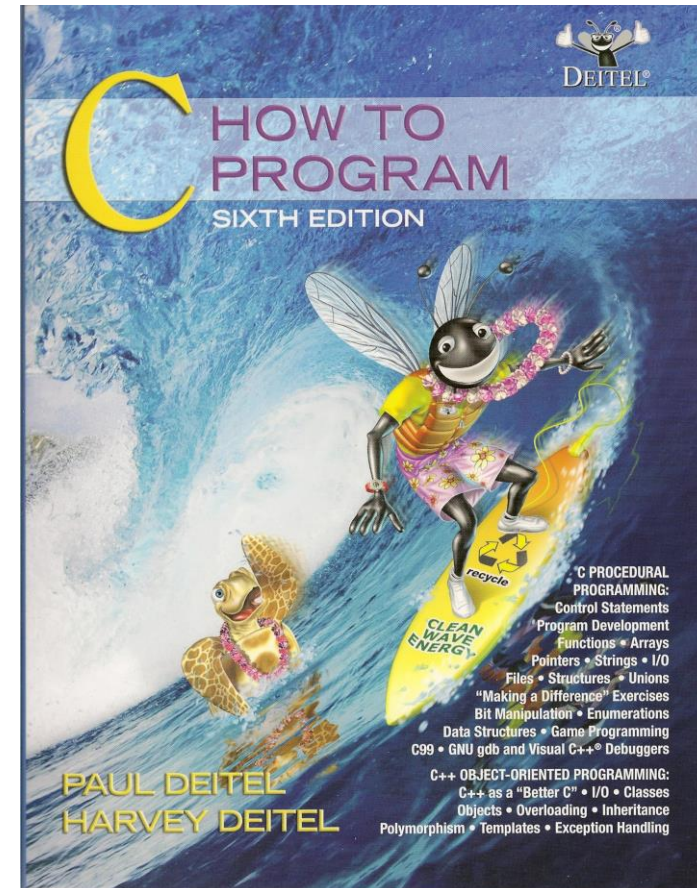
[Paul Deitel](#) (Author), [Harvey Deitel](#) (Author)

Ek Kaynak

Deitel & Deitel, 6th ed.

C & *C++*

Similar to 5th edition.

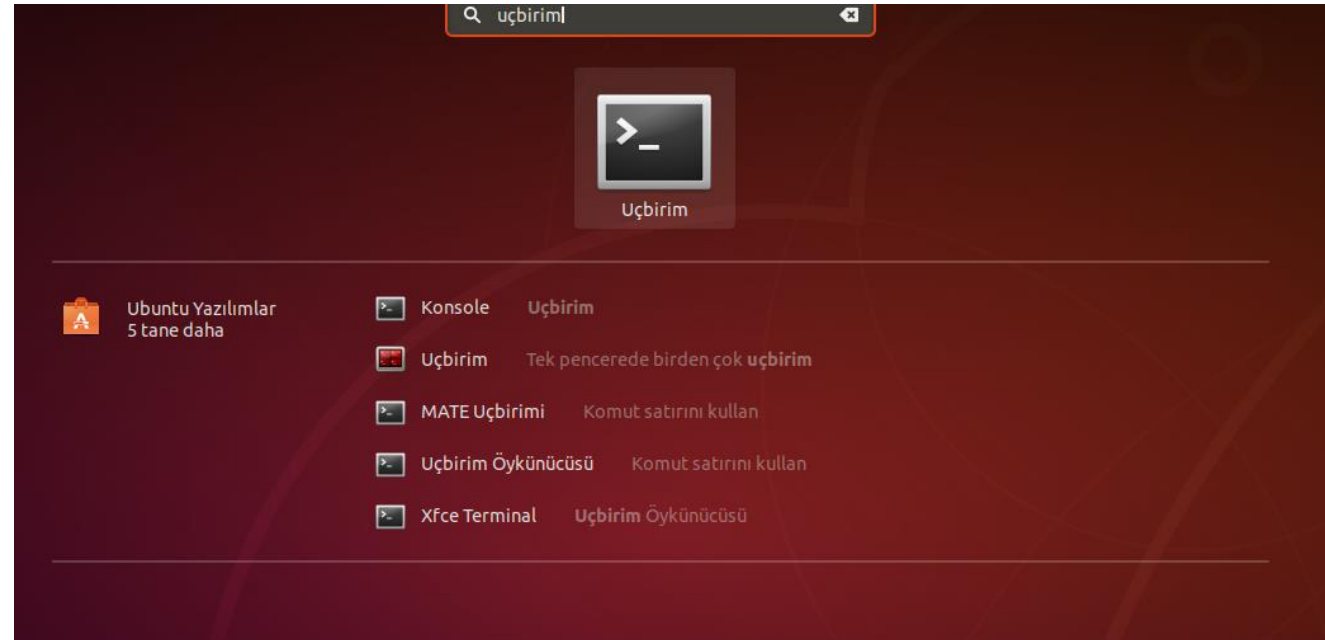


Sistem Programlama

DR. ÖĞR. ÜYESİ ABDULLAH SEVİN

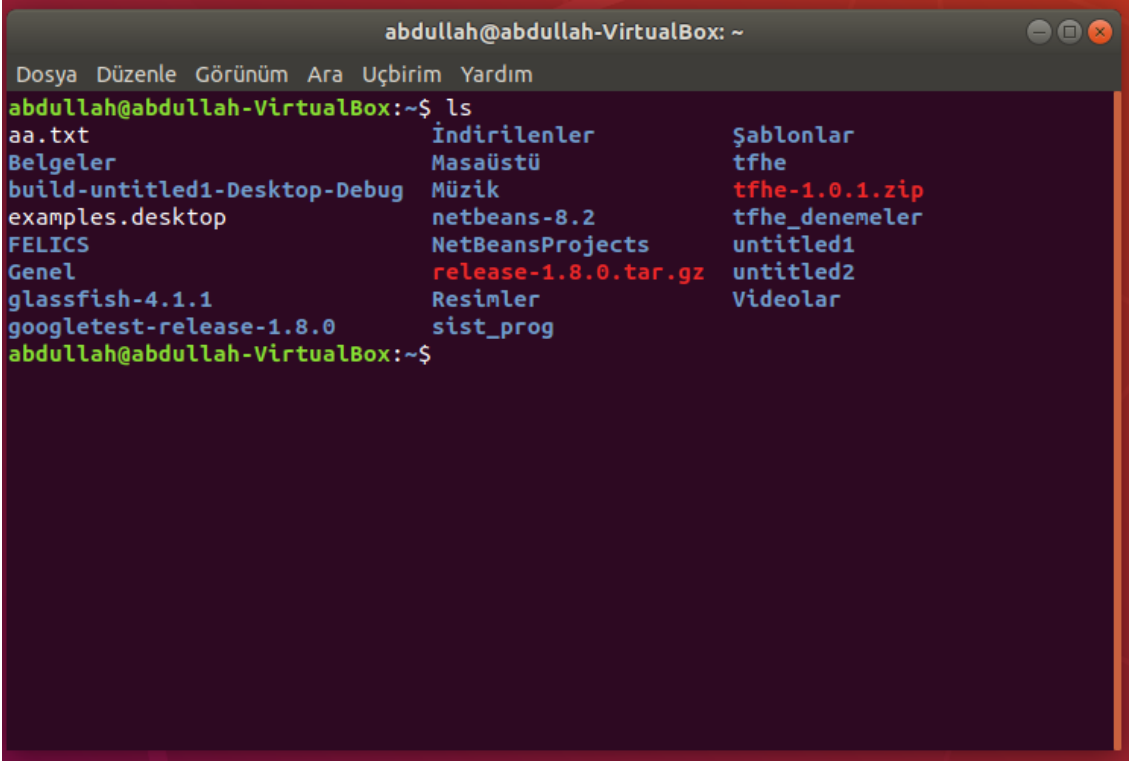
Terminal (Uçbirim)

- ❑ Shell olarak da adlandırılan Terminal ekranını kullanarak işletim sistemine komutlar verebiliriz.
- ❑ Terminal ekranını açmak için Ctrl+Alt+T veya program arama menüsünü kullanabiliriz



Terminal

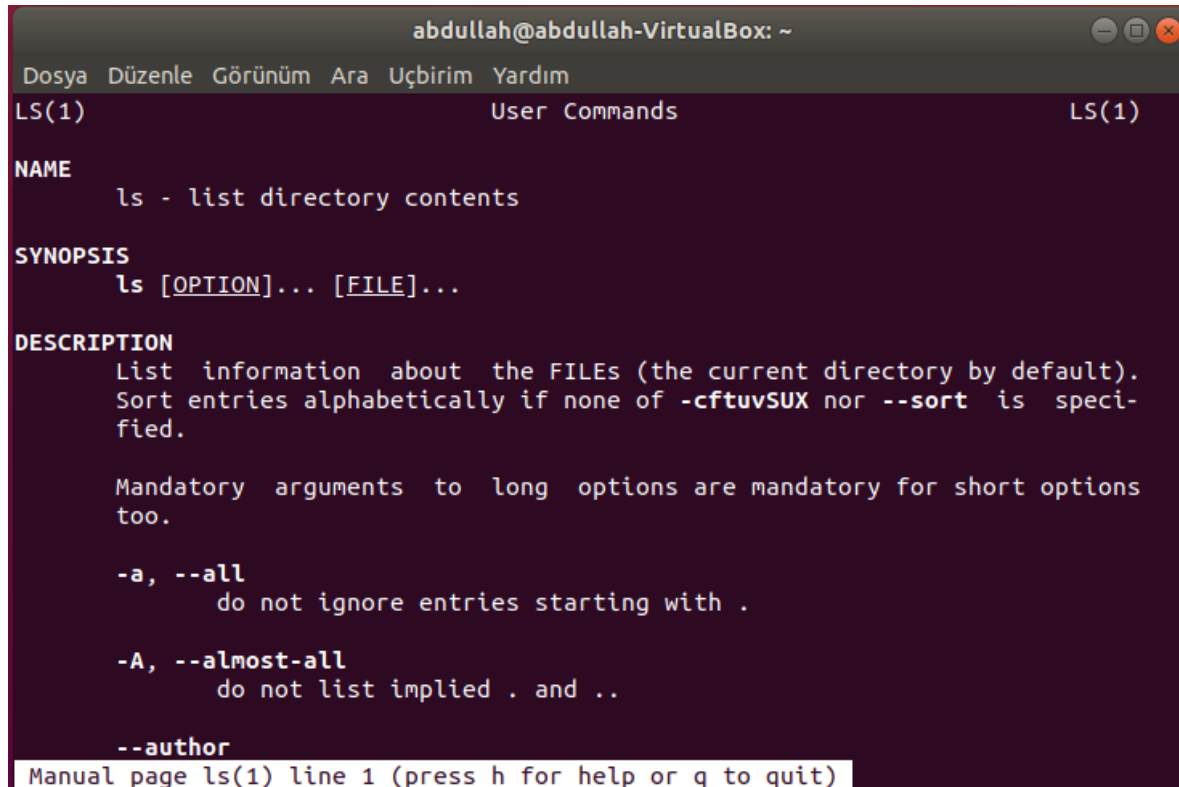
- ❑ Terminal ekranından **ls** komutunu işletirsek aktif olan klasör içindeki dosyaları listeleyebiliriz



```
abduallah@abduallah-VirtualBox: ~  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
abduallah@abduallah-VirtualBox:~$ ls  
aa.txt          İndirilenler      Şablonlar  
Belgeler        Masaüstü          tfhe  
build-untitled1-Desktop-Debug Müzik             tfhe-1.0.1.zip  
examples.desktop netbeans-8.2      tfhe_denemeler  
FELICS          NetBeansProjects  untitled1  
Genel           release-1.8.0.tar.gz untitled2  
glassfish-4.1.1 Resimler          Videolar  
googletest-release-1.8.0 sist_prog  
abduallah@abduallah-VirtualBox:~$
```


Terminal

❑ Komut hakkında bilgi sahibi olmak istersek **man ls**



```
abdullah@abdullah-VirtualBox: ~
Dosya Düzenle Görünüm Ara Uçbirim Yardım
LS(1) User Commands LS(1)

NAME
  ls - list directory contents

SYNOPSIS
  ls [OPTION]... [FILE]...

DESCRIPTION
  List information about the FILES (the current directory by default).
  Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-
  fied.

  Mandatory arguments to long options are mandatory for short options
  too.

  -a, --all
      do not ignore entries starting with .

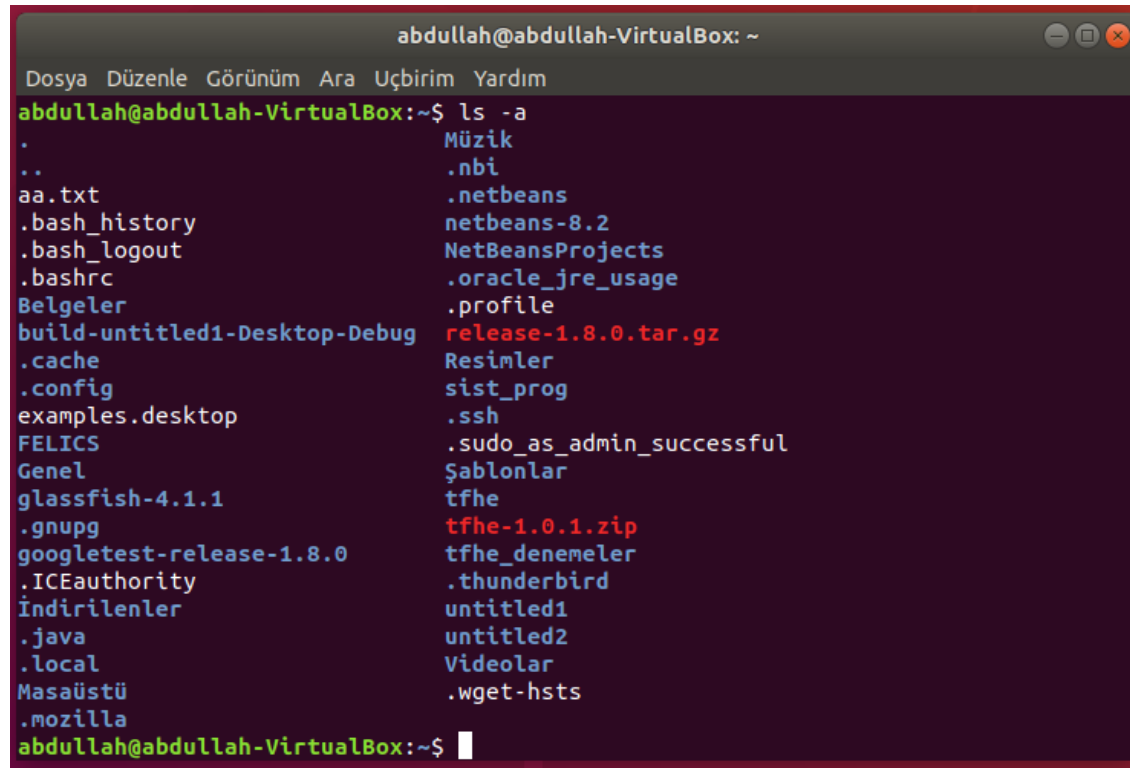
  -A, --almost-all
      do not list implied . and ..

  --author

Manual page ls(1) line 1 (press h for help or q to quit)
```

Terminal

❑ `ls -a` ile `.` ile başlayan kayıtları da görüntüleyebiliriz.



```
abduallah@abduallah-VirtualBox: ~
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~$ ls -a
.           Müzik
..          .nbi
aa.txt     .netbeans
.bash_history netbeans-8.2
.bash_logout NetBeansProjects
.bashrc    .oracle_jre_usage
Belgeler  .profile
build-untitled1-Desktop-Debug release-1.8.0.tar.gz
.cache    Resimler
.config   sist_prog
examples.desktop .ssh
FELICS    .sudo_as_admin_successful
Genel     Şablonlar
glassfish-4.1.1 tfhe
.gnupg    tfhe-1.0.1.zip
googletest-release-1.8.0 tfhe_denemeler
.ICEauthority .thunderbird
İndirilenler  untitled1
.java       untitled2
.local      Videolar
Masaüstü    .wget-hsts
.mozilla
```

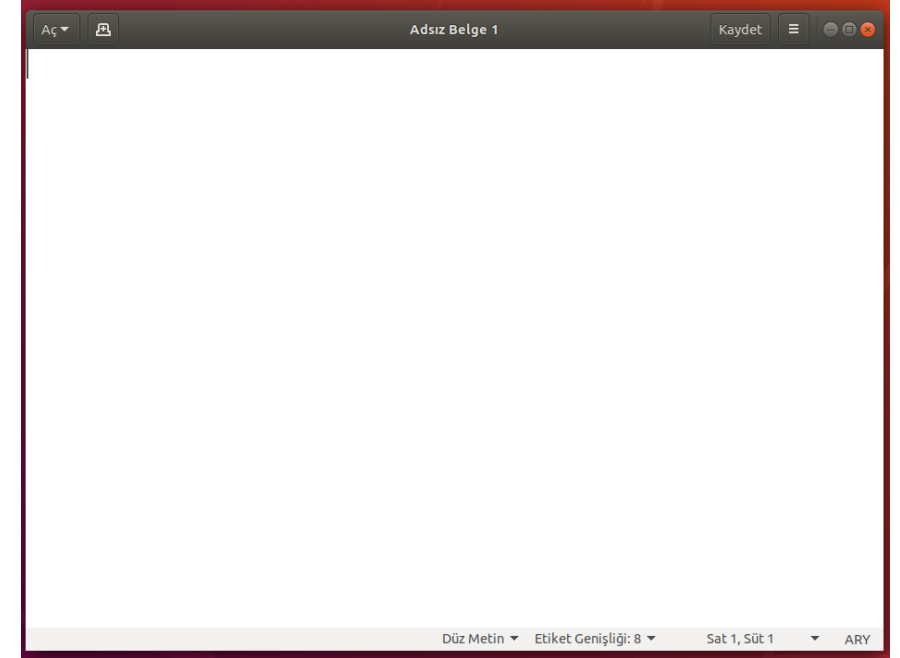
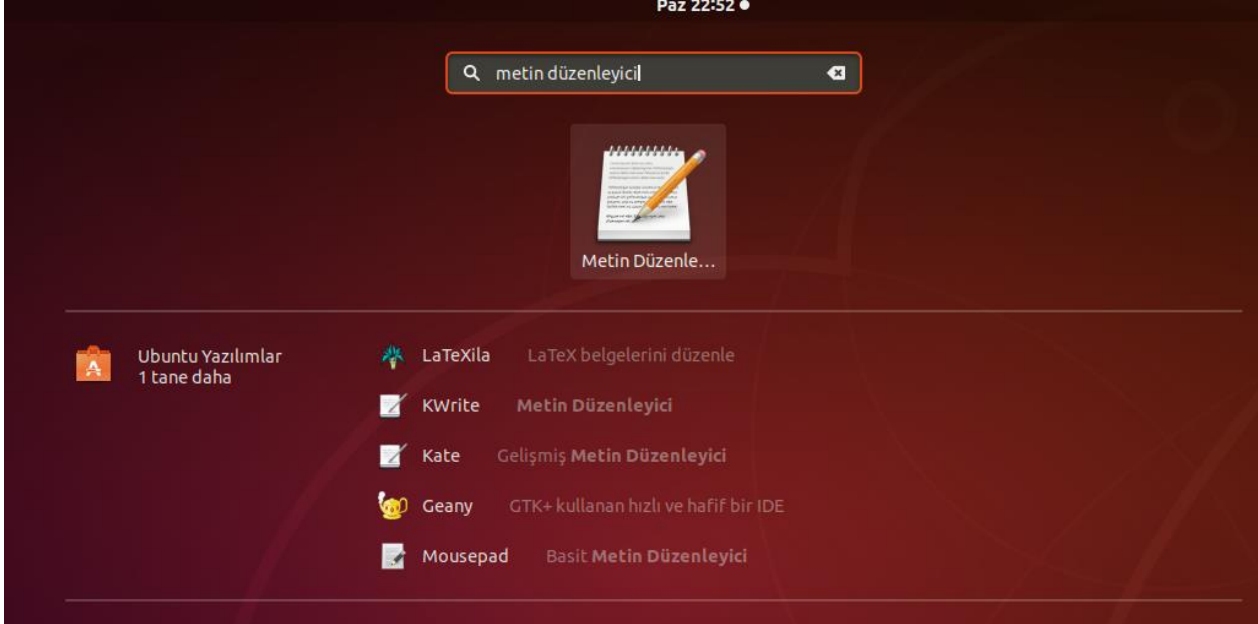
Terminal

❏ ls -l detaylı listeleme

```
abduallah@abduallah-VirtualBox: ~
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~$ ls -l
toplam 6812
-rw-rw-r-- 1 abduallah abduallah 461 Şub 12 17:37 aa.txt
drwxr-xr-x 2 abduallah abduallah 4096 Eki 24 2018 Belgeler
drwxrwxr-x 2 abduallah abduallah 4096 Eki 25 2018 build-untitled1-Desktop-Debu
ug
-rw-r--r-- 1 abduallah abduallah 8980 Eki 24 2018 examples.desktop
drwxrwxr-x 7 abduallah abduallah 4096 Şub 5 2020 FELICS
drwxr-xr-x 2 abduallah abduallah 4096 Eki 24 2018 Genel
drwxr-xr-x 8 abduallah abduallah 4096 Eki 25 2018 glassfish-4.1.1
drwxrwxr-x 5 abduallah abduallah 4096 Şub 9 17:07 googletest-release-1.8.0
drwxr-xr-x 10 abduallah abduallah 4096 Oca 17 17:50 Indirilenler
drwxr-xr-x 3 abduallah abduallah 4096 Oca 2 14:53 Masaüstü
drwxr-xr-x 2 abduallah abduallah 4096 Eki 24 2018 Müzik
drwxr-xr-x 23 abduallah abduallah 4096 Ara 6 2018 netbeans-8.2
drwxrwxr-x 4 abduallah abduallah 4096 Kas 1 2018 NetBeansProjects
-rw-rw-r-- 1 abduallah abduallah 1281617 Şub 9 17:06 release-1.8.0.tar.gz
drwxr-xr-x 2 abduallah abduallah 4096 Şub 26 22:45 Resimler
drwxrwxr-x 2 abduallah abduallah 4096 Şub 25 23:28 sist_prog
drwxr-xr-x 2 abduallah abduallah 4096 Eki 24 2018 Şablonlar
drwxrwxr-x 5 abduallah abduallah 4096 Şub 12 16:46 tfhe
-rw-r--r-- 1 abduallah abduallah 5596228 Oca 14 23:38 tfhe-1.0.1.zip
drwxrwxr-x 3 abduallah abduallah 4096 Şub 12 16:44 tfhe_denemeler
drwxrwxr-x 2 abduallah abduallah 4096 Oca 25 15:28 untitled1
```

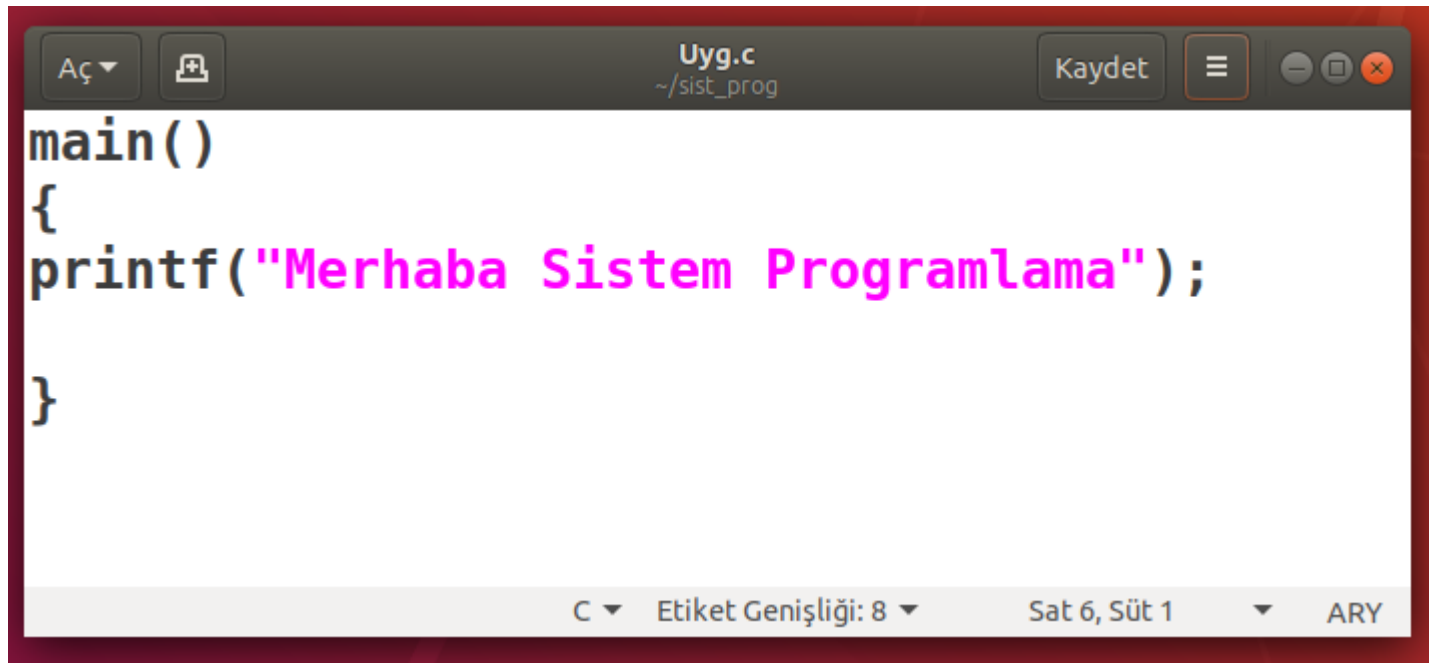
Metin Düzenleyici

❏ Kod yazımı için metin düzenleyici;



Örnek C programı

- Basit bir program ; ekrana yazı yazdırma

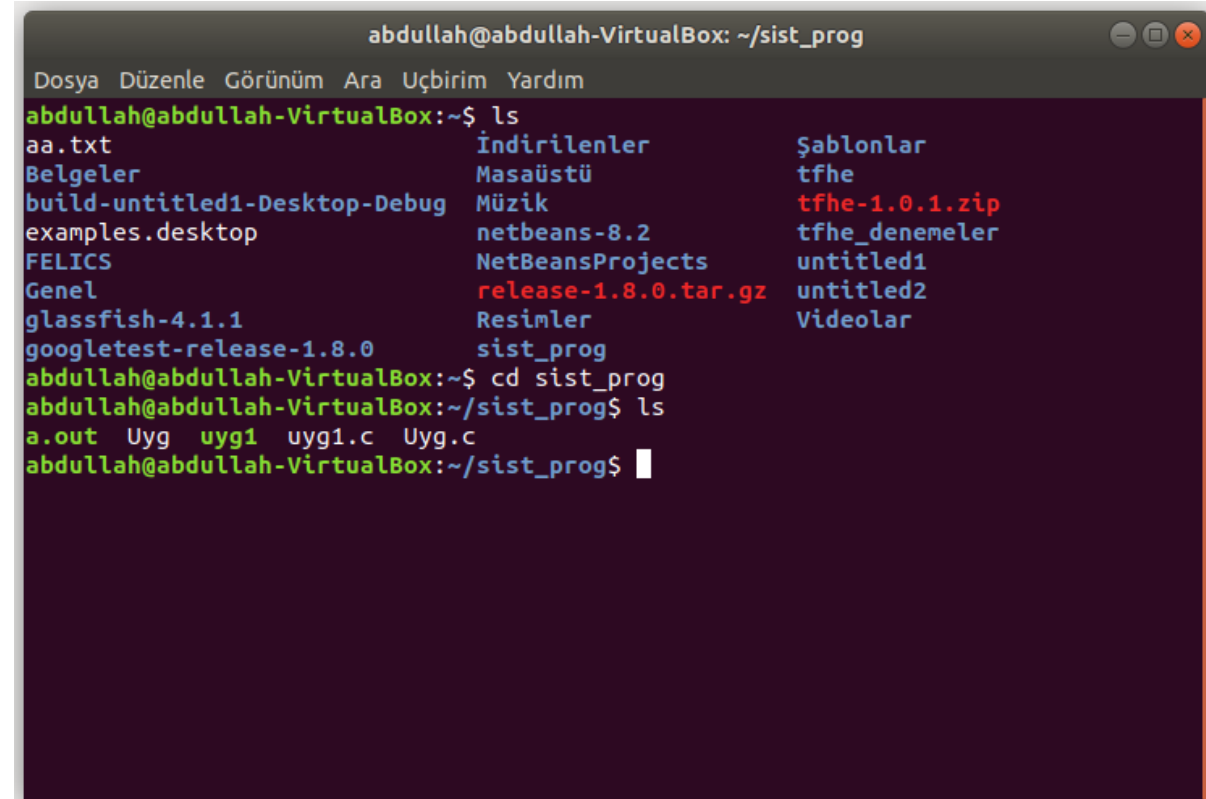


```
main()
{
printf("Merhaba Sistem Programlama");
}
```

The screenshot shows a code editor window titled 'Uyg.c' with the path '~/sist_prog'. The window has a menu bar with 'Aç' (Open) and 'Kaydet' (Save) buttons, and a toolbar with standard window controls. The code is written in C and uses a syntax highlighter where the string is magenta. The status bar at the bottom shows 'C', 'Etiket Genişliği: 8', 'Sat 6, Süt 1', and 'ARY'.

Programın derlenmesi

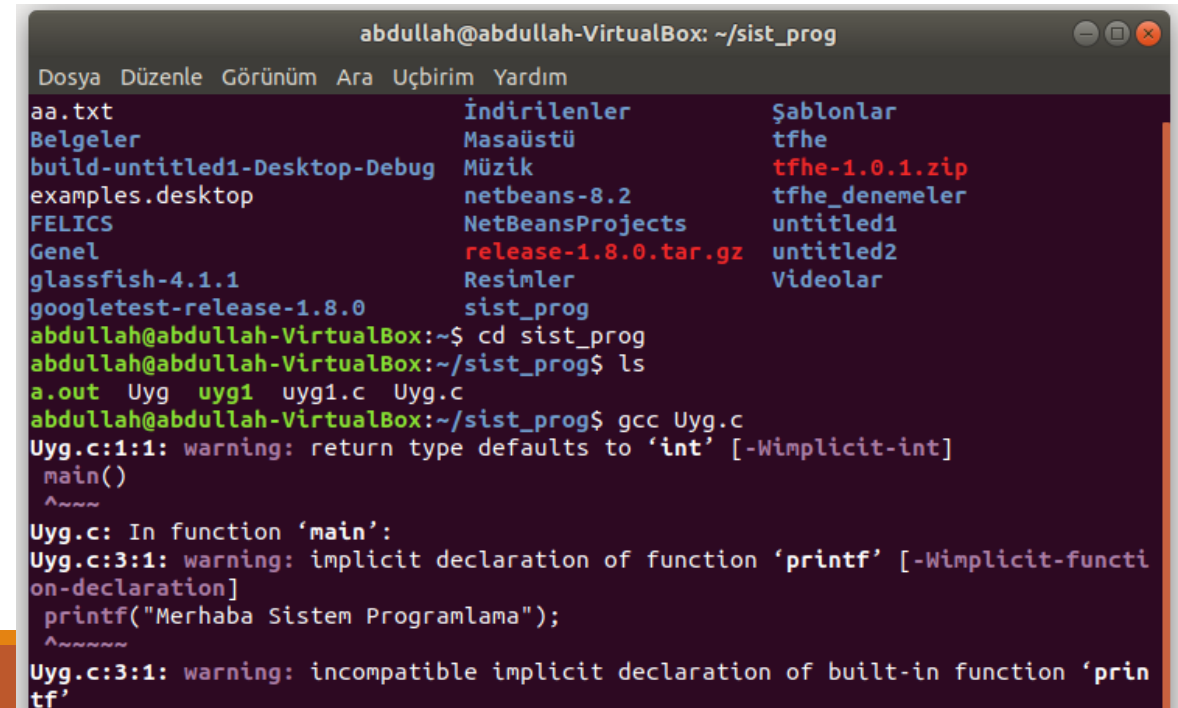
- ❑ Uyg.c dosyasını kaydettikten sonra GNU C Compiler kullanarak derleyelim.
- ❑ gcc ubuntu ile yerleşik olarak gelmekte.
- ❑ Öncelikle bulunduğumuz dizini uyg.c dosyasının bulunduğu klasör olacak şekilde değiştirelim.



```
abdullah@abdullah-VirtualBox: ~/sist_prog
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abdullah@abdullah-VirtualBox:~$ ls
aa.txt          İndirilenler      Şablonlar
Belgeler        Masaüstü          tfhe
build-untitled1-Desktop-Debug Müzik             tfhe-1.0.1.zip
examples.desktop netbeans-8.2      tfhe_denemeler
FELICS          NetBeansProjects  untitled1
Genel           release-1.8.0.tar.gz untitled2
glassfish-4.1.1 Resimler          Videolar
googletest-release-1.8.0 sist_prog
abdullah@abdullah-VirtualBox:~$ cd sist_prog
abdullah@abdullah-VirtualBox:~/sist_prog$ ls
a.out  Uyg  uyg1  uyg1.c  Uyg.c
abdullah@abdullah-VirtualBox:~/sist_prog$
```


Derleme

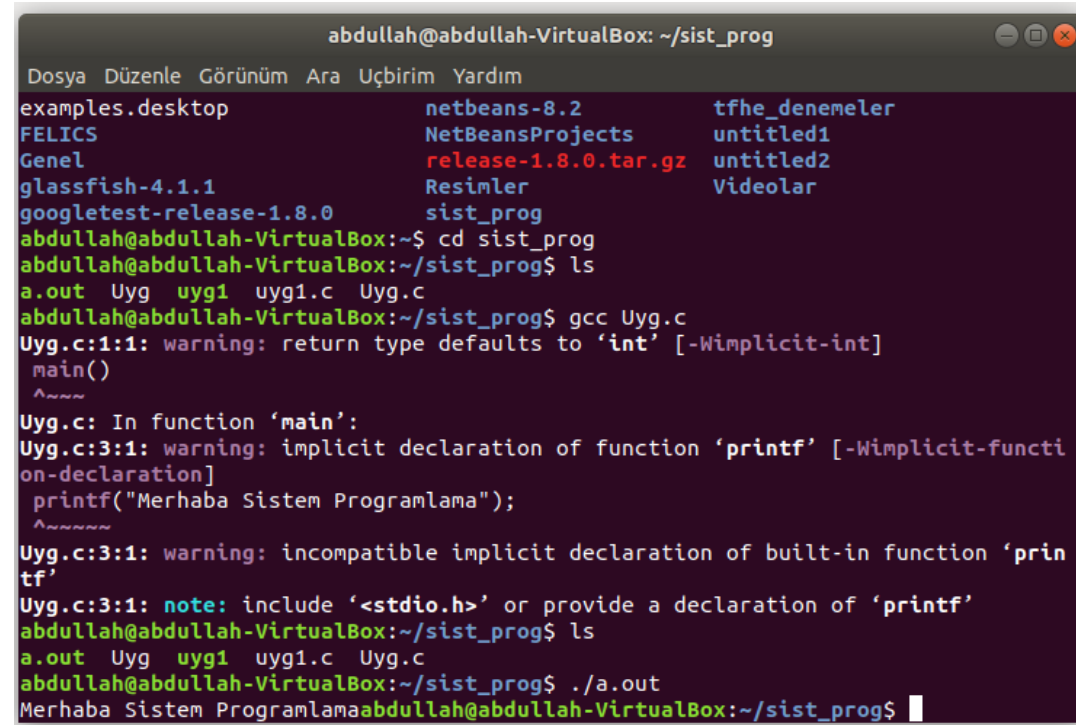
- ❑ Aşağıdaki gibi gcc yazdıktan sonra dosya ismini belirterek çalıştırırsak dosyamız derlenmiş olur.
- ❑ Derleme sonucu ortaya çıkan uyarılar eklemediğimiz kütüphanelerin imalı olarak eklenmesi ile ilgilidir.



```
abdullah@abdullah-VirtualBox: ~/sist_prog
Dosya Düzenle Görünüm Ara Uçbirim Yardım
aa.txt          İndirilenler      Şablonlar
Belgeler        Masaüstü          tfhe
build-untitled1-Desktop-Debug Müzik             tfhe-1.0.1.zip
examples.desktop netbeans-8.2      tfhe_denemeler
FELICS          NetBeansProjects  untitled1
Genel           release-1.8.0.tar.gz untitled2
glassfish-4.1.1 Resimler          Videolar
googletest-release-1.8.0 sist_prog
abdullah@abdullah-VirtualBox:~$ cd sist_prog
abdullah@abdullah-VirtualBox:~/sist_prog$ ls
a.out  Uyg  uyg1  uyg1.c  Uyg.c
abdullah@abdullah-VirtualBox:~/sist_prog$ gcc Uyg.c
Uyg.c:1:1: warning: return type defaults to 'int' [-Wimplicit-int]
main()
^~~~~
Uyg.c: In function 'main':
Uyg.c:3:1: warning: implicit declaration of function 'printf' [-Wimplicit-function-declaration]
printf("Merhaba Sistem Programlama");
^~~~~
Uyg.c:3:1: warning: incompatible implicit declaration of built-in function 'printf'
```

Derleme

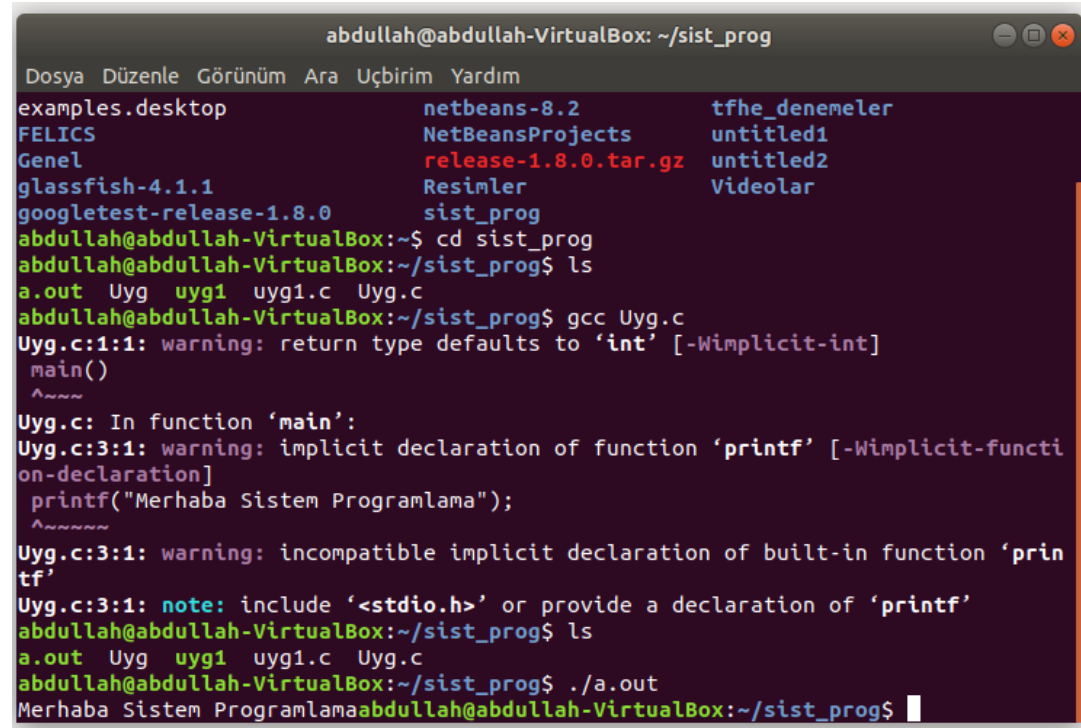
- ❑ Komut penceresinden tekrar ls yazıp klasörün içerisindeki dosyalara bakarsak, derleme sonucu **a.out** adında bir dosyanın oluşturulduğu görülür.



```
abduallah@abduallah-VirtualBox: ~/sist_prog
Dosya  Düzenle  Görünüm  Ara  Uçbirim  Yardım
examples.desktop  netbeans-8.2  tfhe_denemeler
FELICS  NetBeansProjects  untitled1
Genel  release-1.8.0.tar.gz  untitled2
glassfish-4.1.1  Resimler  Videolar
googletest-release-1.8.0  sist_prog
abduallah@abduallah-VirtualBox:~$ cd sist_prog
abduallah@abduallah-VirtualBox:~/sist_prog$ ls
a.out  Uyg  uyg1  uyg1.c  Uyg.c
abduallah@abduallah-VirtualBox:~/sist_prog$ gcc Uyg.c
Uyg.c:1:1: warning: return type defaults to 'int' [-Wimplicit-int]
main()
^
Uyg.c: In function 'main':
Uyg.c:3:1: warning: implicit declaration of function 'printf' [-Wimplicit-function-declaration]
printf("Merhaba Sistem Programlama");
^
Uyg.c:3:1: warning: incompatible implicit declaration of built-in function 'printf'
Uyg.c:3:1: note: include '<stdio.h>' or provide a declaration of 'printf'
abduallah@abduallah-VirtualBox:~/sist_prog$ ls
a.out  Uyg  uyg1  uyg1.c  Uyg.c
abduallah@abduallah-VirtualBox:~/sist_prog$ ./a.out
Merhaba Sistem Programlamaabduallah@abduallah-VirtualBox:~/sist_prog$
```

Çalıştırma

- ❑ Komut satırında **./a.out** yazarsak programın beklendiği gibi çalışıp ekrana mesaj yazdırır.



```
abduallah@abduallah-VirtualBox: ~/sist_prog
Dosya Düzenle Görünüm Ara Uçbirim Yardım
examples.desktop      netbeans-8.2          tfhe_denemeler
FELICS                NetBeansProjects      untitled1
Genel                 release-1.8.0.tar.gz  untitled2
glassfish-4.1.1       Resimler              Videolar
googletest-release-1.8.0 sist_prog
abduallah@abduallah-VirtualBox:~$ cd sist_prog
abduallah@abduallah-VirtualBox:~/sist_prog$ ls
a.out  Uyg  uyg1  uyg1.c  Uyg.c
abduallah@abduallah-VirtualBox:~/sist_prog$ gcc Uyg.c
Uyg.c:1:1: warning: return type defaults to 'int' [-Wimplicit-int]
main()
~~~~~
Uyg.c: In function 'main':
Uyg.c:3:1: warning: implicit declaration of function 'printf' [-Wimplicit-function-declaration]
printf("Merhaba Sistem Programlama");
~~~~~
Uyg.c:3:1: warning: incompatible implicit declaration of built-in function 'printf'
Uyg.c:3:1: note: include '<stdio.h>' or provide a declaration of 'printf'
abduallah@abduallah-VirtualBox:~/sist_prog$ ls
a.out  Uyg  uyg1  uyg1.c  Uyg.c
abduallah@abduallah-VirtualBox:~/sist_prog$ ./a.out
Merhaba Sistem Programlamaabduallah@abduallah-VirtualBox:~/sist_prog$
```

Uyarılar

❑ Derleme uyarılarını giderelim;



The screenshot shows a code editor window titled 'uyg1.c' with the path '~/.sist_prog'. The code is as follows:

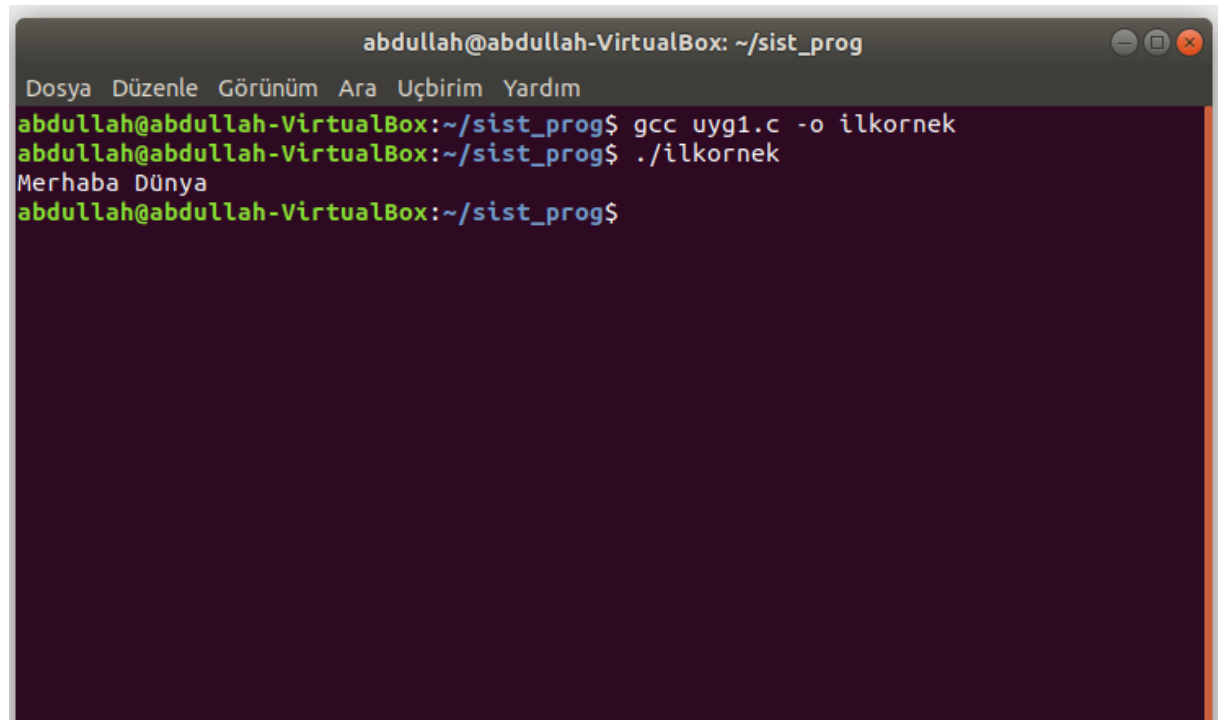
```
#include <stdio.h>

int main()
{
    printf("Merhaba Dünya\n");
    return 0;
}
```

The status bar at the bottom indicates the language is 'C', the tab width is 'Etiket Genişliği: 8', the cursor is at 'Sat 5, Süt 24', and the encoding is 'ARY'. A warning icon is visible in the top toolbar.

Derleme

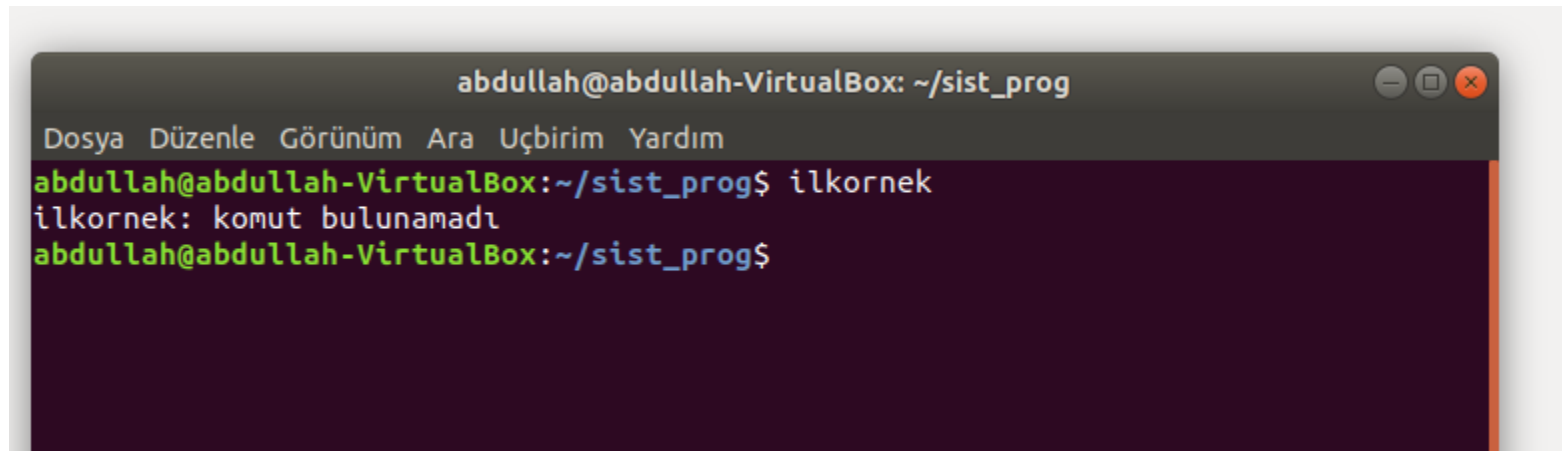
- ❑ Derlenen programın adını değiştirme;
- ❑ gcc -o derleme çıktısını bir output dosyasına yazar.



```
abduallah@abduallah-VirtualBox: ~/sist_prog
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~/sist_prog$ gcc uyg1.c -o ilkornek
abduallah@abduallah-VirtualBox:~/sist_prog$ ./ilkornek
Merhaba Dünya
abduallah@abduallah-VirtualBox:~/sist_prog$
```

Sistem yolu

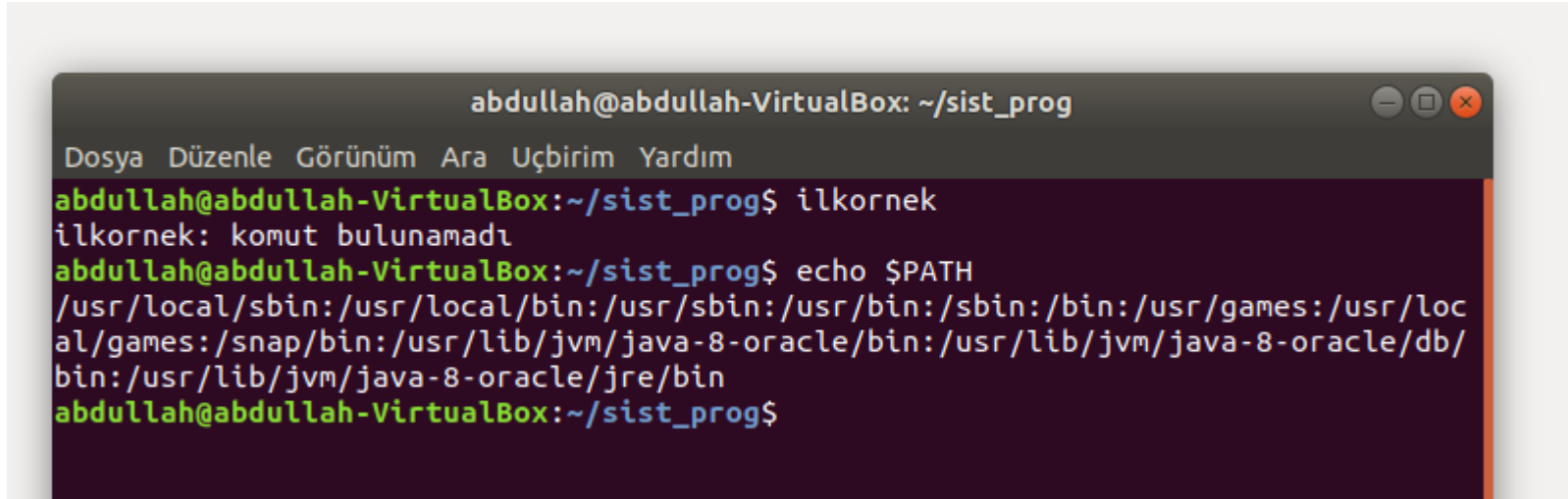
- ❑ `./` derlenmiş programın sistem yolundaki programlarla karışmasını engeller.
- ❑ `./` **kullanmadan çalıştırma için** derlenmiş kodun bulunduğu dizini sistem yoluna eklememiz gerekir. Herhangi bir sistem programı gibi bütün dizinlerden erişilebilir olması için...



```
abduallah@abduallah-VirtualBox: ~/sist_prog
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~/sist_prog$ ilkornek
ilkornek: komut bulunamadı
abduallah@abduallah-VirtualBox:~/sist_prog$
```

Sistem yolu

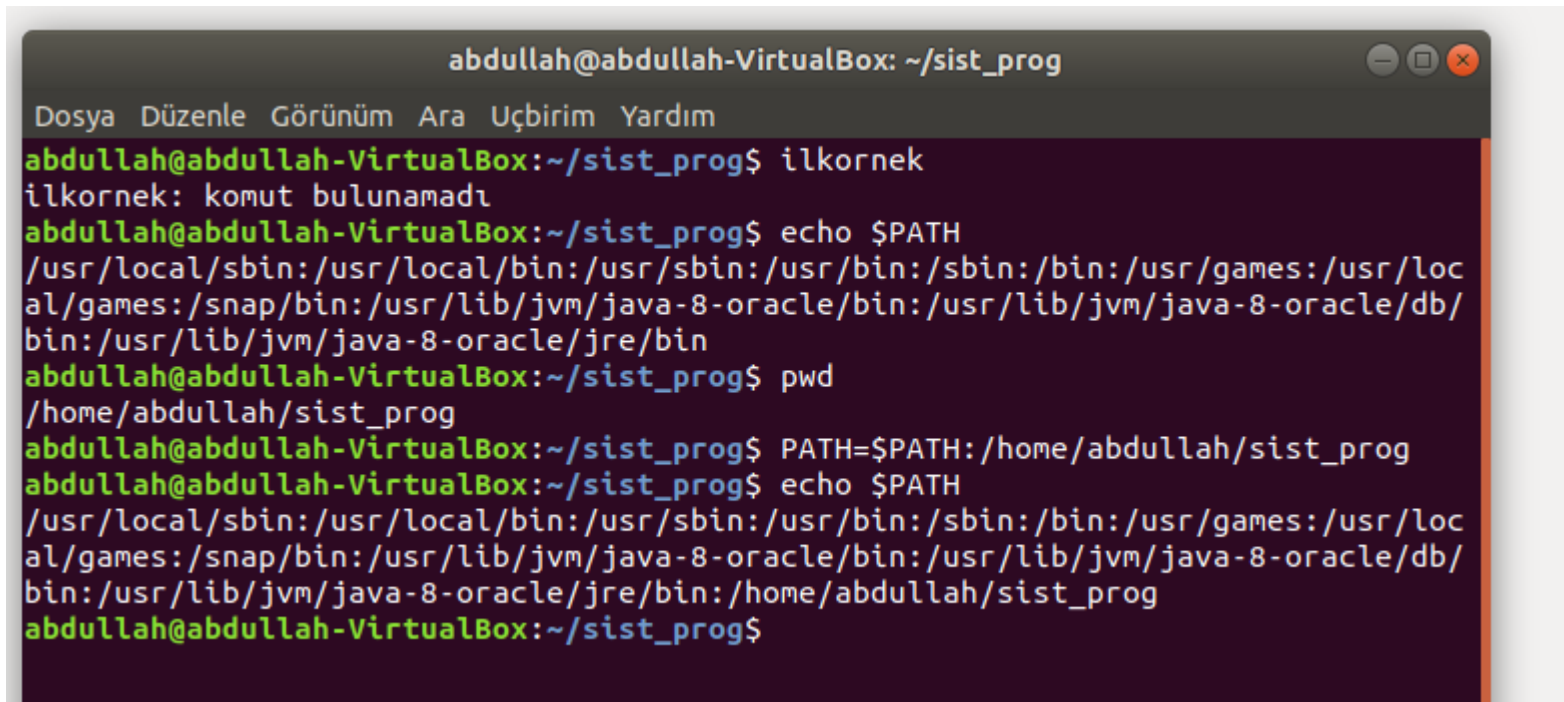
- ❑ Sistem yolundaki dizinleri görmek için **\$PATH** değişkenini **echo** ile yazdırabiliriz

A screenshot of a terminal window titled 'abduallah@abduallah-VirtualBox: ~/sist_prog'. The terminal shows a menu bar with 'Dosya', 'Düzenle', 'Görünüm', 'Ara', 'Uçbirim', and 'Yardım'. The user enters the command 'ilkornek', which results in the message 'ilkornek: komut bulunamadı'. Then, the user enters 'echo \$PATH', which outputs the system path: '/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/usr/lib/jvm/java-8-oracle/bin:/usr/lib/jvm/java-8-oracle/db/bin:/usr/lib/jvm/java-8-oracle/jre/bin'. The prompt returns to 'abduallah@abduallah-VirtualBox: ~/sist_prog\$'.

```
abduallah@abduallah-VirtualBox: ~/sist_prog
Dosya  Düzenle  Görünüm  Ara  Uçbirim  Yardım
abduallah@abduallah-VirtualBox:~/sist_prog$ ilkornek
ilkornek: komut bulunamadı
abduallah@abduallah-VirtualBox:~/sist_prog$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/usr/lib/jvm/java-8-oracle/bin:/usr/lib/jvm/java-8-oracle/db/bin:/usr/lib/jvm/java-8-oracle/jre/bin
abduallah@abduallah-VirtualBox:~/sist_prog$
```


Sistem yolu

- ❑ Bulunduğumuz dizine ait yol; **pwd** (print working directory)
- ❑ Bu yolu kopyalayarak sistem yoluna ekleyebiliriz.



```
abduallah@abduallah-VirtualBox: ~/sist_prog
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~/sist_prog$ ilkornek
ilkornek: komut bulunamadı
abduallah@abduallah-VirtualBox:~/sist_prog$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/usr/lib/jvm/java-8-oracle/bin:/usr/lib/jvm/java-8-oracle/db/bin:/usr/lib/jvm/java-8-oracle/jre/bin
abduallah@abduallah-VirtualBox:~/sist_prog$ pwd
/home/abduallah/sist_prog
abduallah@abduallah-VirtualBox:~/sist_prog$ PATH=$PATH:/home/abduallah/sist_prog
abduallah@abduallah-VirtualBox:~/sist_prog$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/usr/lib/jvm/java-8-oracle/bin:/usr/lib/jvm/java-8-oracle/db/bin:/usr/lib/jvm/java-8-oracle/jre/bin:/home/abduallah/sist_prog
abduallah@abduallah-VirtualBox:~/sist_prog$
```

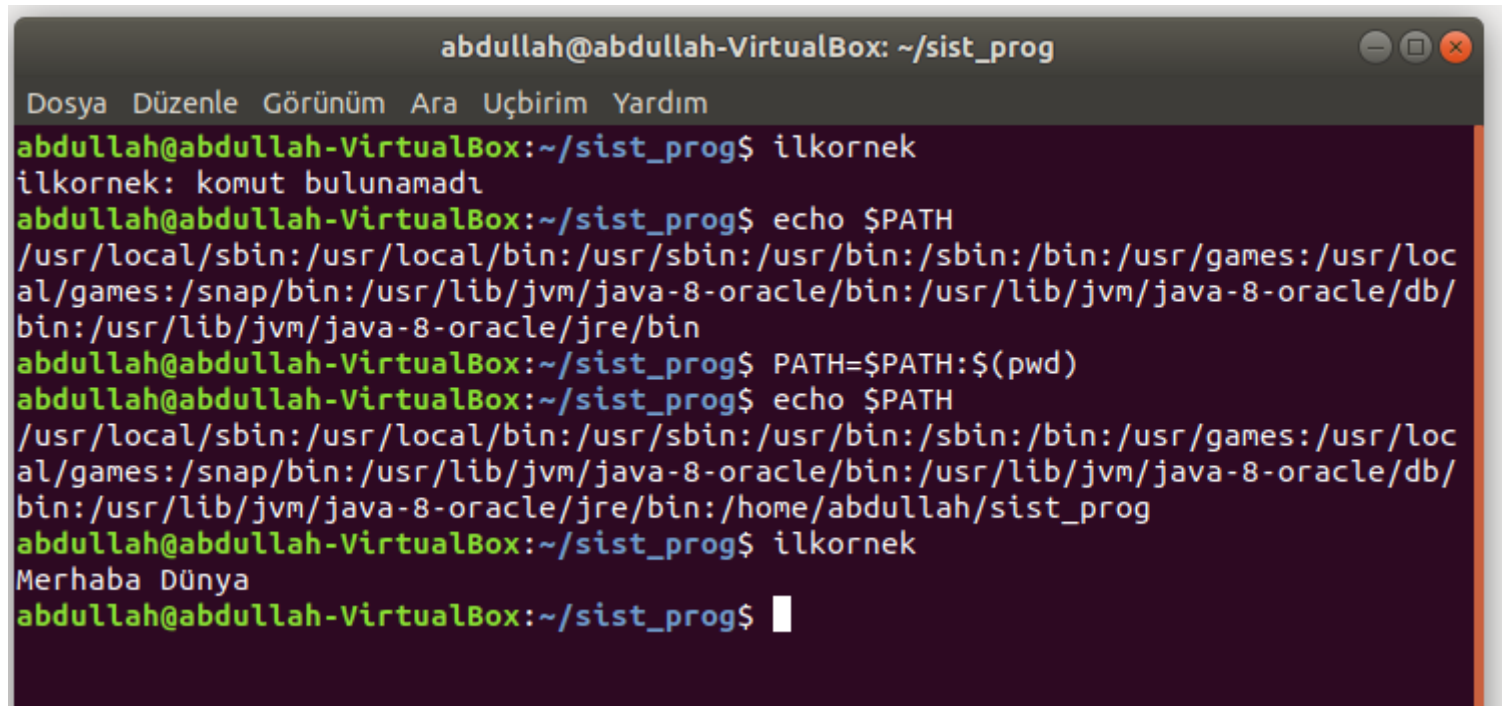
Sistem yolu

- ❑ Yolu yazmak yerine direkt olarak **pwd** komutu ile aktif dizini sistem yoluna ekleyebiliriz

```
abduallah@abduallah-VirtualBox: ~/sist_prog
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~/sist_prog$ ilkornek
ilkornek: komut bulunamadı
abduallah@abduallah-VirtualBox:~/sist_prog$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/usr/lib/jvm/java-8-oracle/bin:/usr/lib/jvm/java-8-oracle/db/bin:/usr/lib/jvm/java-8-oracle/jre/bin
abduallah@abduallah-VirtualBox:~/sist_prog$ PATH=$PATH:$(pwd)
abduallah@abduallah-VirtualBox:~/sist_prog$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/usr/lib/jvm/java-8-oracle/bin:/usr/lib/jvm/java-8-oracle/db/bin:/usr/lib/jvm/java-8-oracle/jre/bin:/home/abduallah/sist_prog
abduallah@abduallah-VirtualBox:~/sist_prog$
```

Çalıştırma

□ ./ olmadan artık çalıştırabiliriz.



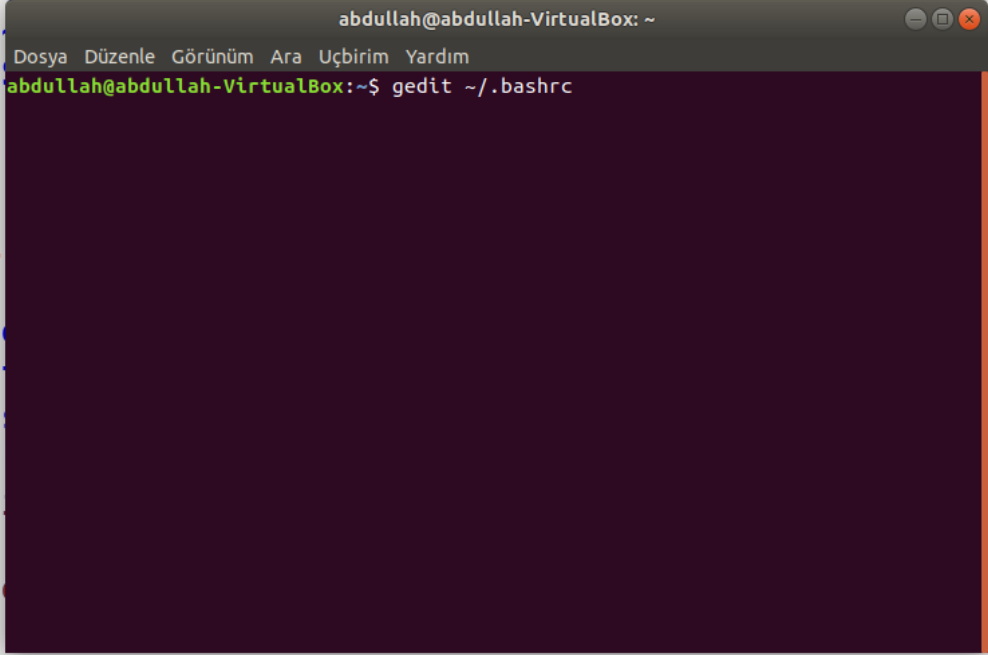
```
abduallah@abduallah-VirtualBox: ~/sist_prog
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~/sist_prog$ ilkornek
ilkornek: komut bulunamadı
abduallah@abduallah-VirtualBox:~/sist_prog$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/usr/lib/jvm/java-8-oracle/bin:/usr/lib/jvm/java-8-oracle/db/bin:/usr/lib/jvm/java-8-oracle/jre/bin
abduallah@abduallah-VirtualBox:~/sist_prog$ PATH=$PATH:$(pwd)
abduallah@abduallah-VirtualBox:~/sist_prog$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/usr/lib/jvm/java-8-oracle/bin:/usr/lib/jvm/java-8-oracle/db/bin:/usr/lib/jvm/java-8-oracle/jre/bin:/home/abduallah/sist_prog
abduallah@abduallah-VirtualBox:~/sist_prog$ ilkornek
Merhaba Dünya
abduallah@abduallah-VirtualBox:~/sist_prog$
```

bashrc

- ❑ Fakat yapılan değişiklikler terminal penceresi kapatıldığında kaybolur.
- ❑ Kalıcı olması için `~/.bashrc` dosyasını `gedit ~/.bashrc` komutu ile düzenleyebiliriz.

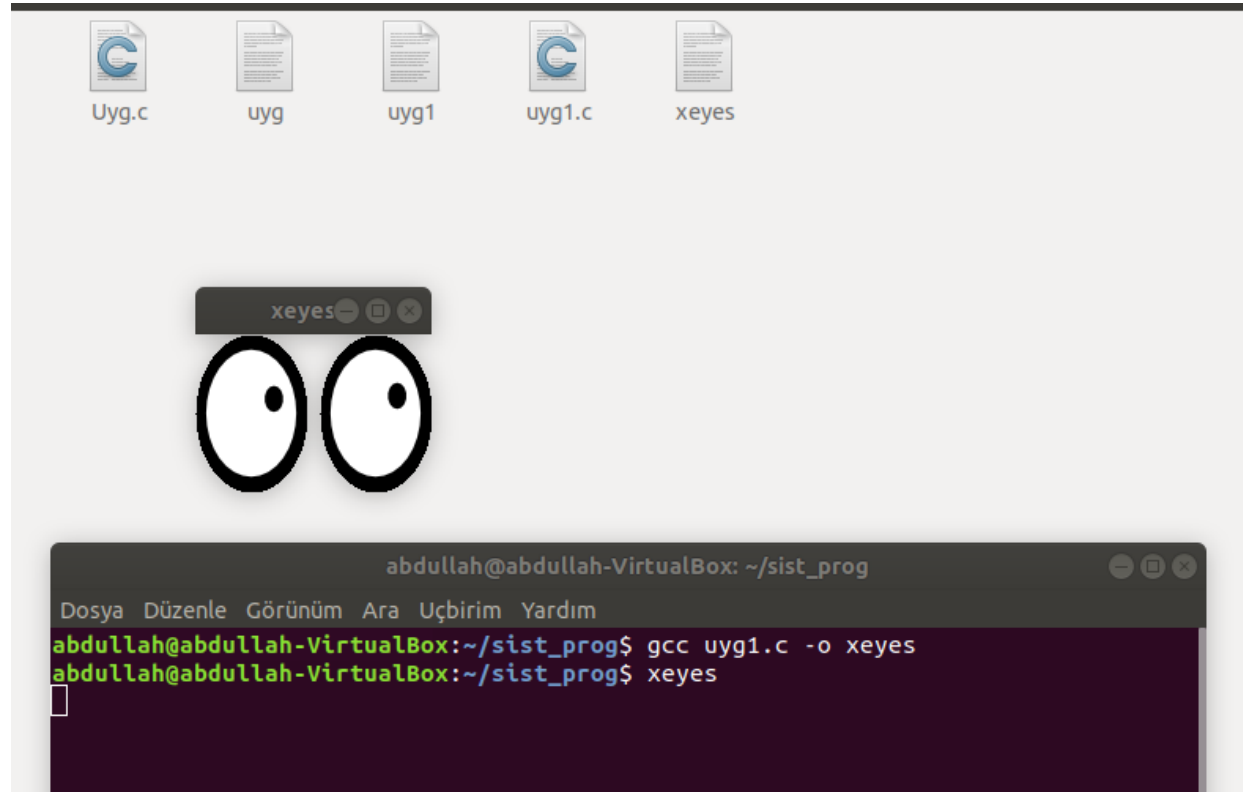
```
# Alias definitions.
# You may want to put all your additions into a separate
# file named aliases, and include that file in your bashrc
# file.
#
# Example aliases
# unalias -v
#
if [ -f ~/.aliases ]
then
source ~/.aliases
fi

# Don't put into .bashrc
#
# If you run this on a machine that has never had bashrc
# before, you will see this message:
#
# You have disabled shell completion for
# the following command(s), because it
# would interfere with completion of
# GNU command-line utilities.
#
# To disable completion for a command,
# type:
#
# sudo touch ~/.bashrc.d/_disable_completion.sh
#
# To re-enable completion for a command,
# type:
#
# sudo rm ~/.bashrc.d/_disable_completion.sh
#
#
PATH=$PATH:/home/abduallah/sist_prog
```



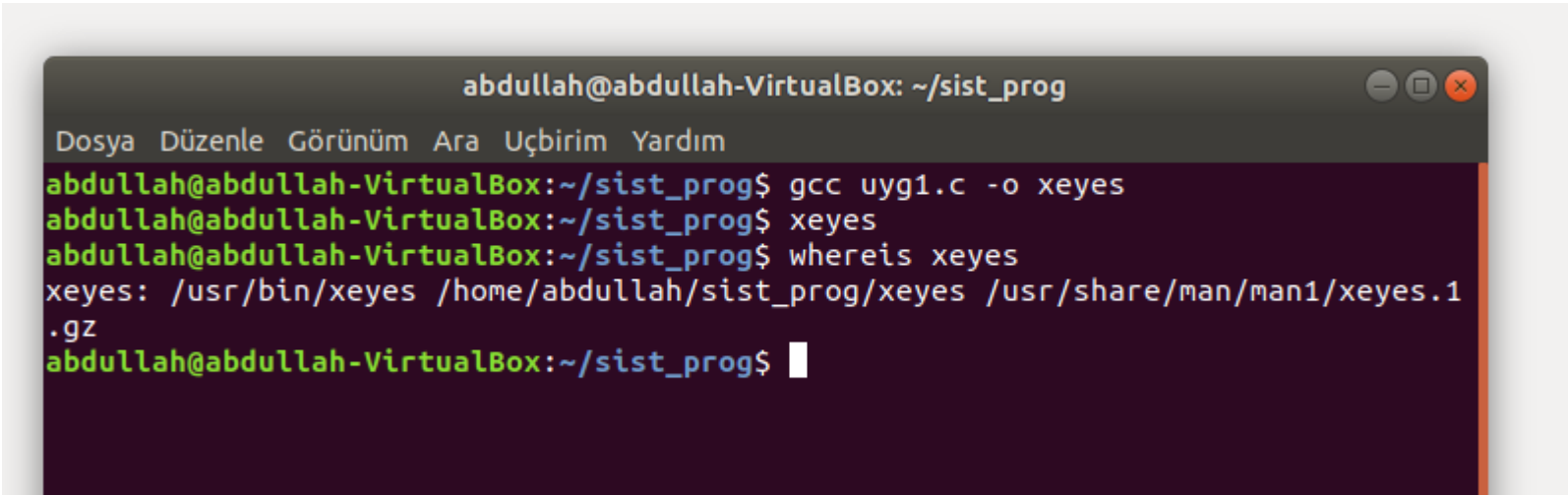
Sistem yolu

- ❑ Fakat artık isimler sistem yolundaki klasörde yer alan programlar ile aynı olmamalı



whereis

- ❑ Programın nerede olduğunu **whereis** komutu ile bulabiliriz.

A terminal window titled 'abdullah@abdullah-VirtualBox: ~/sist_prog' with standard window controls. The terminal shows a sequence of commands: 'gcc uyg1.c -o xeyes', 'xeyes', and 'whereis xeyes'. The output of the last command is 'xeyes: /usr/bin/xeyes /home/abdullah/sist_prog/xeyes /usr/share/man/man1/xeyes.1.gz'. The prompt is currently at 'abdullah@abdullah-VirtualBox:~/sist_prog\$' with a cursor.

```
abdullah@abdullah-VirtualBox: ~/sist_prog
Dosya  Düzenle  Görünüm  Ara  Uçbirim  Yardım
abdullah@abdullah-VirtualBox:~/sist_prog$ gcc uyg1.c -o xeyes
abdullah@abdullah-VirtualBox:~/sist_prog$ xeyes
abdullah@abdullah-VirtualBox:~/sist_prog$ whereis xeyes
xeyes: /usr/bin/xeyes /home/abdullah/sist_prog/xeyes /usr/share/man/man1/xeyes.1
.gz
abdullah@abdullah-VirtualBox:~/sist_prog$
```


Sistem Programlama

DR. ÖĞR. ÜYESİ ABDULLAH SEVİN

İçerik

a) .h dosyaları

b) extern ve static anahtar kelimesinin anlamı

c) derleme vs bağlama

d) bir prosesin hafızada nasıl organize edildiği (yığın, yığıt, vb.)

e) makefiles

- <http://web.eecs.utk.edu/~huangj/cs302/notes/Some%20fundamentals.htm>
- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/CStuff-1/lecture.html>
- https://www.bilgigunlugum.net/prog/cprog/c_degisken
- Cstuff-1 klasör örnekleri

Program nedir?

- ❑ Elbette işletim sistemlerinde bir program çalışırken hepimiz buna <proses> dendiğini biliyoruz.
- ❑ Programın kendisi birçok biçimde olabilir, yani makine kodu, assembly kodu, C kodu, C++ veya Fortran, Java, ...
- ❑ İşlemciler YALNIZCA **makine kodunu** yürütür. Makine kodu, işlem kodları (Opcode) ve işlenenlerden (Operands) oluşan ikili makine sözcükleriyle yazılır.
- ❑ **İşlem kodları**, örneğin bir aritmetik toplama gibi farklı talimatları temsil eder. **İşlenenler** komut tarafından çalıştırılır. Bir makine kodu şöyle görünebilir:

100010001101010010101101101011

- ❑ Çoğu akli başında insan bu formatta kod yazmak istemez. Bu yüzden assembly dili gelişti;

ADD AX, BX

Assembly

❑ Assembly dilindeki anımsatıcılar, **işlemci bağımlı** olan makine kodlarıyla birebir ilişkilidir. Fakat hala dönüşüm için «**assembler**» gerekli!

❑ Intel x86'dan Motorola 68k'ye geçerseniz, yeni bir program yazılmalıdır.

❑ İşte Motorola 68k üzerinde çalışan gerçek bir assembly programı:

❑ Bu arada, C programınızın Assembly takmaya nasıl benzediğini görmek istiyorsanız, “cc -S xxx.c” veya “gcc -S xxx.c” kullanmayı deneyin. Bu komutlar, C kodunuzun derleme sürümünü içeren .s dosyaları üretir.

```
ORG      $1000
N        EQU      5
CNT1     DC.B     0
CNT2     DC.B     0
ARRAY    DC.B     2,7,1,6,3

MAIN     ORG      $1500
         LEA      ARRAY,A2
         MOVE.B   #N,D1
         CLR      D6
         CLR      D7
         JSR      SORT
         STOP     #$2700
```

Assembly

- ❑ Yukarıdaki programda, **adresler**, bellek bankalarınızdaki ayrı baytlara karşılık gelen fiziksel adreslerdir.
- ❑ Bu adresleri sabit kodladıysanız, işlemcinizde her seferinde yalnızca bir program çalıştırmanız iyi olur!
- ❑ Assembly dilinde programlama yapmak sıkıcı olabilir! Sistem çağrılarını yapmıyorsanız, konsolunuza bir karakter koymak için 30 satır kod gerekebilir.
- ❑ Kendi kod parçanızı her yeniden kullanmak istediğinizde uğraşmanız gerekir. Bütün yapıları siz kuruyorsunuz, **Başkalarının kodunu** kullandığınızı hayal edin.
- ❑ Bu gibi problemleri aşmak için C/C++ gibi diller geliştirildi.

C dili

- ❑ Kaynak kodunuz bir **derleyici** ile çalıştırılır ve C programınızı ikili makine koduna çevirir ve genellikle 'nesne-object' olarak adlandırılan bir modül oluşturur.
- ❑ Derleyicinin ortaya çıkardığı her **object** dosyasının içinde, temelde **platforma bağlı** bir biçimde bir makine kodu modülü elde edersiniz.
- ❑ Ancak, önemli nokta; orada **kodlanmış fiziksel adres yok**. Bu işlem, modülü ana bellekte **yeniden yerleştirilebilir hale** getirir.
- ❑ Bu durumda xxx.o adlı object dosyası diske yazılır.

C dili

- ❑ daha sonra, dış referanslar (external ref.), fonksiyon çağrılarını veya değişkenler için nesne dosyanıza bakmak üzere ayrı bir **bağlayıcı-linker** programı çağrılır.
- ❑ Örneğin, “printf(“Merhaba Dünya\n”);” fonk. çağırıdınız. Derleyiciniz, sistem kitaplığında sağlandığı için bu çağırının nasıl çalıştığını bilmiyordur.
- ❑ Sistem kitaplığından bir printf makine kodu modülünü bulup xxx.o ile bir araya getirmek **bağlayıcınızın işidir.**
- ❑ Bağlama oturumunuzun sonucu, diskte yürütülebilir-executable bir dosyadır.

C dili

- ❑ komut satırına a.out yazdıktan sonra, işletim sistemindeki **yükleyici-loader** a.out'u diskten okur ve ana belleğe yazar.
- ❑ Bu sırada a.out'taki her satırın fiziksel adresi belirlenir.
- ❑ Ardından, işletim sistemi programınızın giriş noktasını (xxx.c'deki main fonksiyon) bulur ve oradan çalıştırmaya başlar.

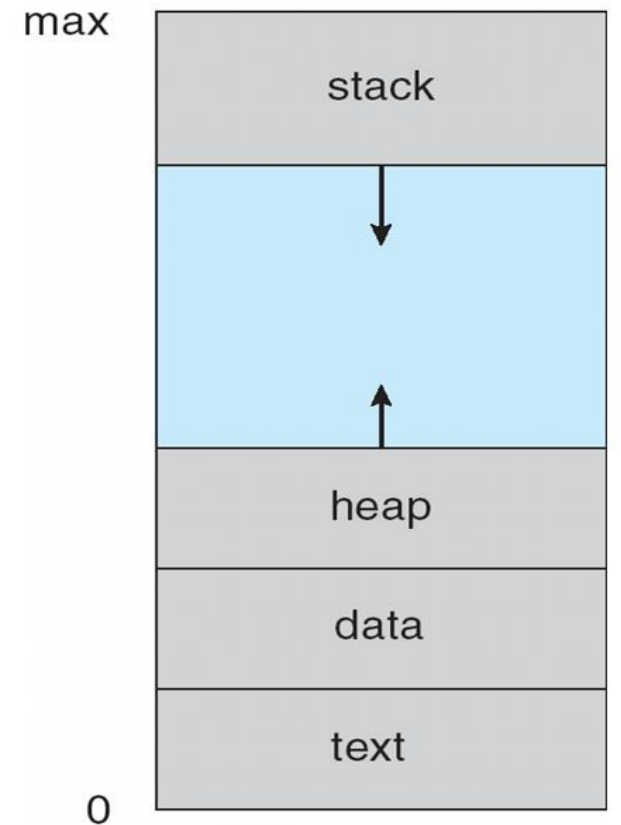
Neden .h dosyasına ihtiyacımız var?

- ❑ İşletim sistemindeki **kütüphaneleri-library** ve bazen de başkaları tarafından sağlanan kütüphaneleri kullanarak ciddi bir uygulama geliştirmek için birçok kişi gerekir.
- ❑ Modüler geliştirme bir **normdur**.
- ❑ Derleyicilerin tip kontrolleri yapması ve kütüphaneye ve diğer kişilerin fonksiyonlarına yaptığınız çağrıların doğru olduğundan emin olması gerekir.
- ❑ Çağırduğınız fonksiyon gövdesinin C kaynağına erişiminiz olsun ya da olmasın, **kodunuza bir fonksiyon gövdesi yapıştırmak kötü bir fikirdir** (neden?).
- ❑ Genellikle **veri türlerini, fonksiyon bildirimlerini ve makroları** içeren .h dosyalarının kullanılması bu sorunu çözer.

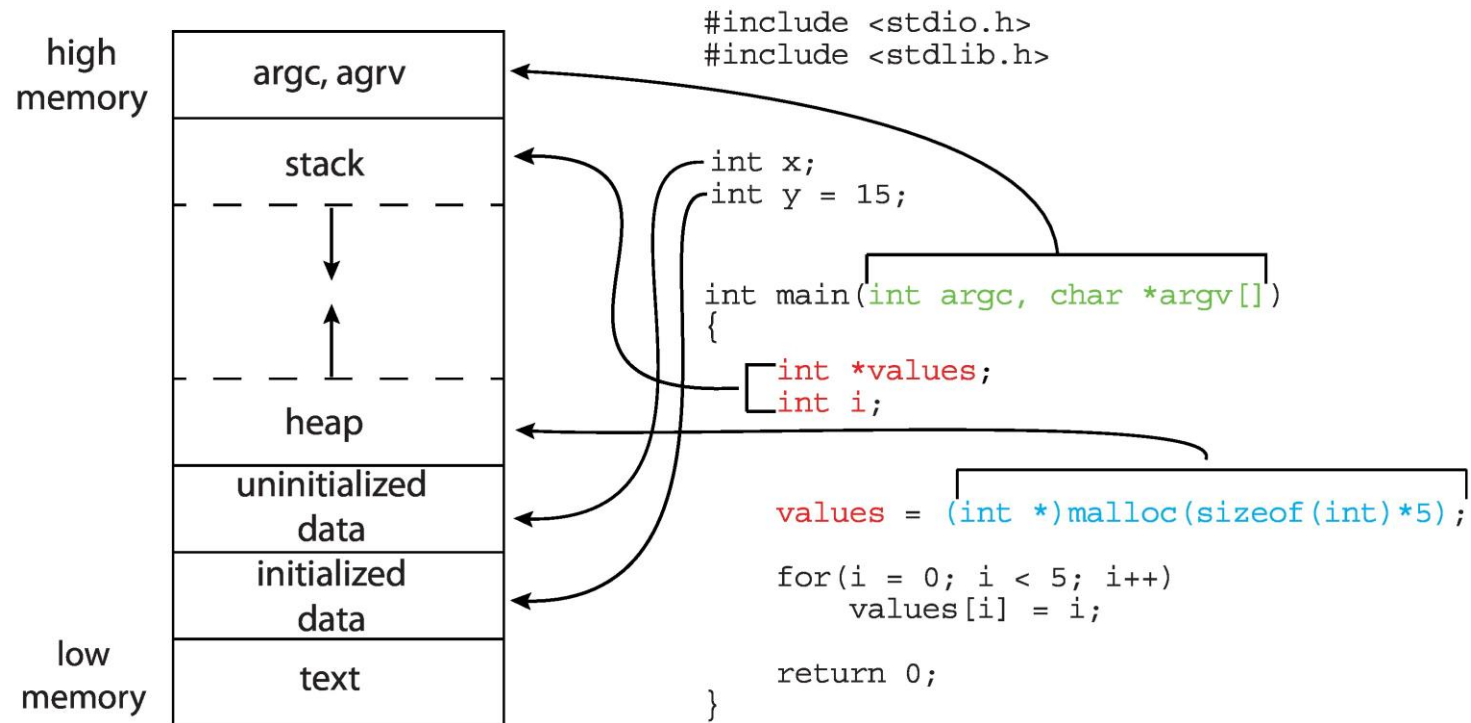
Prosesin hafızadaki yerleşimi

Bölümleri;

- **program kodu** (text section),
- **Mevcut aktiviteler, program sayacı** ve diğer kaydedicileri içeren mevcut faaliyetler.
- **Yığın** (stack) geçici veriyi içerir.
 - Fonksiyon parametreleri, geri dönüş değerleri, yerel değişkenler
- **Veri bölümü** (data section) global değişkenleri içerir.
- **Bellek kümesi-Yığın** (heap) çalışma zamanında dinamik olarak tahsis edilmiş belleği içerir.



Memory Layout of a C Program



Fonksiyonlar ve Yığın-Stack bölümü

1. OS, ana programdaki geçerli adresi yığına gönderir. (**Geri dönülecek adres**)
2. CPU kayıtları da dahil olmak üzere ana programı çalıştırırken kurtarmak için gerekli tüm verileri yığına gönderir.
3. bu fonksiyon çağrısındaki tüm bağımsız değişkenleri birer birer yığına gönderir.
4. o fonksiyonun text (kod) bölümünün baş adresini bulur ve yürütmeye başlar
5. fonksiyon içinde, yerel değişkenler için yığında daha fazla bellek alanı ayrılır
6. malloc() çağrıları varsa, yığın(heap) alanından bellek ayrılır

Fonksiyonlar ve Yığın-Stack bölümü

7. fonksiyonda hesaplamayı bitirin
8. dinamik bellek ayırmaları varsa, belki de yer ayırmanız gerekir?
9. tüm yerel değişkenleri yığından çıkarın
10. tüm fonksiyon bağımsız değişkenlerini yığından çıkarın
11. orijinal çalışma durumunu yığından çekin
12. yığından yeni alınan adresten tekrar yürütmeye başlayın

Kapsam ve Görünürlük

- ❑ Dosya kapsamı-File scope;
- ❑ Dosya kapsamına sahip tanımlayıcı adları genellikle "**genel-global**" veya "**harici-external**" olarak adlandırılır.
- ❑ Herhangi bir blok veya parametre listesinin dışında görünür ve bildiriminden sonra herhangi bir yerden erişilebilir.

- ❑ Fonksiyon Kapsamı-Function scope;
- ❑ Bir fonksiyon içerisinde tanımlıdır ve Etiket adları, bir fonksiyon içinde benzersiz olmalıdır.

Kapsam ve Görünürlük

- ❑ Blok kapsamı-Block scope;
- ❑ Yalnızca bildirimi veya tanımı noktasından bildirimi veya tanımı içeren bloğun sonuna kadar görünür.
- ❑ Kapsamı, o blokla ve o blokta iç içe geçmiş herhangi bir blokla sınırlıdır ve ilişkili bloğu kapatan kaşlı ayraçta biter. Bu tür tanımlayıcılara bazen "**yerel değişkenler**" denir.
- ❑ Fonksiyon Prototipi-Function-prototype scope;
- ❑ Bir fonksiyon prototipindeki parametre bildirimleri listesinde görünür.

Extern ve static

□ **static** ile bildirilen bir değişken. Statik değişkeni açıkça sabit bir ifadeyle başlatabilirsiniz. Başlatıcıyı atlarsanız, değişken varsayılan olarak 0 olarak başlatılır.

```
static int k = 16;  
static int k;
```

Extern ve static

- ❑ **static** belirleyicisini lokal değişkenlerle kullanabiliriz.
- ❑ Normal olarak, içinde lokal değişken tanımlanan bir fonksiyonu her çağırdığımızda, lokal değişken değeri yenilenir.
- ❑ Ancak, bu lokal değişkeni **static** olarak tanımlarsak, fonksiyonu her çağırmamızda, lokal değişken bir önceki fonksiyon çağrısındaki en son değerini korur.
- ❑ Sonuç olarak, **static** lokal bir değişkene sadece fonksiyonun ilk çağrılışında bir defaya mahsus olmak üzere değer verebiliriz.

Extern ve static

```
#include <stdio.h>

void fonk(void);
void fonk_sta(void);

int main(void)
{
    fonk();
    fonk_sta();
    printf("\n");
    fonk();
    fonk_sta();
    return 0;
}
```

```
void fonk(void) {
    int id = 1;
    printf("fonk() id değişken değeri: %d\n", id);
    id = id + 21;
    printf("fonk() id değişken değeri: %d\n", id);
}

void fonk_sta(void) {
    static int id = 1;
    // Sadece fonksiyonun ilk çağrısında çalışır.
    printf("fonk_sta() id değişken değeri: %d\n", id);
    id = id + 21;
    printf("fonk_sta() id değişken değeri: %d\n", id);
}
```

Extern ve static

```
fonk() id değişken değeri: 1  
fonk() id değişken değeri: 22  
fonk_sta() id değişken değeri: 1  
fonk_sta() id değişken değeri: 22  
fonk() id değişken değeri: 1  
fonk() id değişken değeri: 22  
fonk_sta() id değişken değeri: 22  
fonk_sta() id değişken değeri: 43
```

Extern ve static

- ❑ **static** belirleyicisini global değişkenlerle de kullanabiliriz.
- ❑ **static** global bir değişken tanımladığımızda, bu değişkeni sadece içinde tanımlandığı dosyada bulunan fonksiyonlar kullanabilir.
- ❑ Bunun yanında, aynı programa ait farklı dosyalarda bulunan ve aynı isme sahip biri normal diğeri de **static** olan iki global değişken tanımlayabiliriz.

Extern ve static

```
// deneme1.c
#include <stdio.h>
void fonk1(void);
void fonk2(void);
static int gid = 21; // Static global int değişken bildirimi
int main(void) {
    fonk1();
    fonk2();
    return 0; }
void fonk1(void)
{ printf("deneme1.c gid değişken değeri: %d\n", gid); }
```

```
// deneme2.c
#include <stdio.h>
int gid = 35; // global int değişken bildirimi
void fonk2(void)
{printf("deneme2.c gid değişken değeri: %d", gid);}
```

deneme1.c gid değişken değeri: 21
deneme2.c gid değişken değeri: 35

Extern

- ❑ Bir değişkeninin başında **extern** ifadesini kullanmak, o değişkenin projede yer alan kod dosyalarının birinde tanımlandığını ve o değişkene **extern** ifadesini kullandığımız dosya içinden erişim sağlamak istediğimizi gösterir.

```
// deneme1.c
#include <stdio.h>
void fonk(void);
int gid = 287; // global int değişken tanımlaması
int main(void) {
    printf("deneme1.c gid değişken değeri: %d\n", gid);
    fonk(); // deneme2.c dosyasındaki fonk() fonksiyonuna çağrı
    return 0;
}

// deneme2.c
extern int gid; // global int değişken bildirimi
void fonk(void)
{ printf("deneme2.c gid değişken değeri: %d", gid); }
```

```
deneme1.c gid değişken değeri: 287
deneme2.c gid değişken değeri: 287
```

Lifetime

- ❑ “Yaşam süresi”, bir değişkenin veya fonksiyonun var olduğu bir programın yürütülmesi sırasında periyottur.
- ❑ Tanımlayıcının depolanma süresi, statik süre (global yaşam süresi) veya otomatik süre (yerel yaşam süresi) olarak yaşam süresini belirler.
- ❑ Statik depolama sınıfı belirticisi olmadan bildirilen bir tanımlayıcı, bir fonksiyon veya blok içinde bildirilmişse, otomatik depolama süresine sahiptir.
- ❑ **Global yaşam süresi:** Tüm fonksiyonların global ömrü vardır. Bu nedenle, program yürütme sırasında her zaman var olurlar.
- ❑ **Yerel yaşam süresi:** Yerel bir değişkenin bir başlatıcısı varsa, değişken her oluşturulduğunda başlatılır (statik olarak bildirilmedikçe).

Make

Header files

- ❑ C++'da olduğu gibi, standart başlık dosyalarını `#include` ile ekleriz.
- ❑ Küçüktür/büyüktür işaretlerine dosya adını ve `.h` uzantısını ekleriz.

```
#include <iostream>  
using namespace std;
```

- ❑ C' de ise;

```
#include <stdio.h>  
#include <stdlib.h>
```


C'de veri tipleri

- ❑ C'de değişkenlerin sahip olabileceği üç tür tür vardır –
- ❑ skalerler, toplamalar/eklemeli ve işaretçiler (scalars, aggregates, and pointers)
- ❑ Skaler tipler;
 - ❑ char -- 1 byte
 - ❑ short -- 2 bytes
 - ❑ int -- 4 bytes
 - ❑ long -- 4 or 8 bytes, depending on the system and compiler
 - ❑ float -- 4 bytes
 - ❑ double -- 8 bytes
 - ❑ (pointer -- 4 or 8 bytes, depending on the system and compiler)

C'de veri tipleri

- ❑ C'de bir türün boyutunu doğrulamak veya kullanmak istiyorsanız, sizeof() makrosunu kullanırsınız.
- ❑ Örneğin, sizeof(long), sisteminizde bir long'un ne kadar büyük olduğuna bağlı olarak 4 veya 8 değerini döndürür.

```
#include <stdio.h>
#include <stdlib.h>
int i;
int main(int argc, char **argv) {
int j; /* Copy argc to j to i and print i */
j = argc;
i = j;
printf("Argc: %d\n", i); /* Print the size of a long. */
j = sizeof(long);
printf("Sizeof(long): %d\n", j); /* Print the size of a pointer. */
j = sizeof(int *);
printf("Sizeof(int *): %d\n", j); return 0; }
```

```
UNIX> bin/p1
Argc: 1
Sizeof(long): 8
Sizeof(int *): 8
UNIX> bin/p1 using many arguments
Argc: 4
Sizeof(long): 8
Sizeof(int *): 8
UNIX>
```

C'de veri tipleri

- ❑ Bazı makineler, işaretçileri ve uzunlukları dört bayt olmaya zorlayan 32 bit modunda derlemenize izin verir;
- ❑ **sudo apt-get install multi-lib** 32-bit derleme için gerekli yoksa hata mesajı:

```
abduallah@abduallah-VirtualBox:~/sist_prog/cs360-lecture-notes/CStuff-1$ gcc -m32  
-o bin/p1-32 src/p1.c  
In file included from src/p1.c:5:0:  
/usr/include/stdio.h:27:10: fatal error: bits/libc-header-start.h: Böyle bir dos  
ya ya da dizin yok  
#include <bits/libc-header-start.h>
```

UNIX> gcc -m32 -o bin/p1-32 src/p1.c

UNIX> bin/p1-32

Argc: 1

Sizeof(long): 4

Sizeof(int *): 4

UNIX>

C'de veri tipleri

- ❑ Aggregate (Küme-Eklemeli tipler);
- ❑ **Diziler ve struct'lar**, C'deki küme türleridir.
- ❑ Skaler'lerden daha karmaşıktırlar. Bir diziye statik olarak global veya yerel bir değişken olarak bildirebilirsiniz.

```
#include <stdio.h>
#include <stdlib.h>
char s1[15];
int main(int argc, char **argv) {
char s2[4];
...
```

C'de veri tipleri

- ❑ Bir dizi statik olarak tanımlanmışsa, onu başka bir diziye atayamazsınız.
- ❑ ``s2 = "Jim" ifadesi C'de geçersiz çünkü s2 statik olarak bildirildi. Bu programı derlemeye çalışırsanız, gcc size bir hata verecektir:

```
#include <stdio.h>
#include <stdlib.h>

char s1[15];

int main(int argc, char **argv)
{
    char s2[4];

    s2 = "Jim";      // Derleme hatası.
    return 0;
}
```

```
UNIX> gcc -o bin/p2 src/p2.c
src/p2.c:16:6: error: array type 'char [4]' is not assignable
s2 = "Jim"; // This line will not compile.
~~ ^
1 error generated.
UNIX>
```

Pointer

- ❑ İşaretçi-Pointer aslında bir sayıdır, bellekteki bir adresi temsil eden bir tam sayıdır.
- ❑ Bir yandan, birçok programlama dilinde işaretçi kavramı yoktur.
- ❑ Ancak C'nin en büyük avantajlarından biri, yani esneklik, işaretçi kullanımından gelir.
- ❑ Öte yandan dizi, çoğu programlama dilinin sahip olduğu bir kavramdır.
- ❑ C'de de aynı şeye sahibiz.

```
int myarray [30];
```

- ❑ **myarray** değişkeni gerçekten (int *) türünde bir işaretçidir. C'de dizi öğelerine işaretçiler kullanılarak erişilir:

```
myarray[5] aslında *(myarray + 5) olarak çevrilebilir
```

Pointer

- ❑ İşaretçiyi de işaret edebiliriz;

```
int myarray[30];  
main()  
{  
    int * myptr;  
    int ** mydblptr;  
    myptr = myarray;  
    mydblptr = &myarray;  
}
```

- ❑ Bss segmentinde de tahsis edilmiş 30 tamsayı vardır. Bss'deki bu 120 bayt, 30 tamsayınızı gerçekten koyduğunuz yerdir ve myarray, bu 120 baytın baş adresine eşittir.

Pointer

- ❑ İşaretçiyi de işaret edebiliriz;

```
int myarray[30];
main()
{
    int * myptr;
    int ** mydblptr;
    myptr = myarray;
    mydblptr = &myarray;
}
```

- ❑ Bss segmentinde de tahsis edilmiş 30 tamsayı vardır. Bss'deki bu 120 bayt, 30 tamsayınızı gerçekten koyduğunuz yerdir ve myarray, bu 120 baytın en baştaki adresine eşittir.

Örnek

```
#include <stdio.h>
#include <stdlib.h>

/* This sets all lower-case letters in a to upper case. */

void change_case(char a[20])
{
    int i;

    for (i = 0; a[i] != '\0'; i++) {
        if (a[i] >= 'a' && a[i] <= 'z') a[i] += ('A' - 'a');
    }
}

/* This initializes a 19-character string of lower-case letters, and then calls change_case(). */

int main()
{
    int i;
    char s[20];

    /* Set s to "abcdefghijklmnopqrs". */

    for (i = 0; i < 19; i++) s[i] = 'a' + i;
    s[19] = '\0';

    /* Print, call change_case() and print again. */

    printf("First, S is %s.\n", s);
    change_case(s);
    printf("Now, S is   %s.\n", s);

    return 0;
}
```

- ☐ Dizileri parametre olarak ilettiğinizde, diziler değil işaretçiler iletilir.
- ☐ A dizisi 20 karakterlik bir dizi olarak bildirseniz de, yordama iletilen işaretçinin yalnızca işaretçi olduğunu fark edeceksiniz.
- ☐ Bu nedenle change_case() bir kopya üzerinde değil dizi üzerinde çalışır:

```
UNIX> bin/p2a First, S is abcdefghijklmnopqrs.
                Now, S is ABCDEFGHIJKLMNOPQRS.
UNIX>
```

Soru

- Harici bir c dosyasından okunan bitiş karakter değerine kadar harfleri yazan bir kod yazın. Harfleri bastırmak için bir fonksiyon yazınız. Fonksiyon parametre almıyor void. Fonksiyon char değerleri 1 arttırarak ekrana basıyor. (Extern ve static değişkenler kullanarak)

Sistem Programlama

DR. ÖĞR. ÜYESİ ABDULLAH SEVİN

İçerik

a) makefiles

b) Pointers

c) Structs

d) Tip dönüşümü

- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/CStuff-1/lecture.html>
- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/CStuff-2/lecture.html>
- Cstuff-1 klasör örnekleri
- Cstuff-2 klasör örnekleri

Structure (Yapı)

- ❑ Programlama dillerinde, "structure" terimi, farklı veri türlerinin bir araya getirildiği bir bileşik veri türüdür.
- ❑ Bu, programcılara, bir veya daha fazla veri türünden oluşan bir bileşenleri olan yeni bir veri türü oluşturma imkanı verir.
- ❑ Bellekte **stack** bölgesinde saklanır.
- ❑ Örneğin, bir öğrencinin adı, numarası ve notları gibi birçok veriye ihtiyacımız varsa, structure yapısı bu verileri bir arada tutmamızı sağlar.

Structure (Yapı)

- ❑ Verileri toplamamanın diğer bir yolu bir yapı kullanmaktır.
- ❑ Bir yapı, bazı dikkate değer eksiklikler dışında biraz C++ sınıfına benzer:
 - ❑ «public/protected/private» yok.
 - ❑ «constructors/destructors» yok.
 - ❑ Varsayılan "kopyalama" yöntemi yok.
 - ❑ Metotlar yok.

Structure (Yapı)

```
/* A very simple program to show a struct
   that aggregates an integer and a double. */

#include <stdio.h>
#include <stdlib.h>

struct intdouble {
    int i;
    double d;
};

int main()
{
    struct intdouble id1;

    id1.i = 5;
    id1.d = 3.14;

    printf("%d %.2lf\n", id1.i, id1.d);
    return 0;
}
```

```
/* This program is identical to src/id1.c,
   except it uses a typedef so that you can
   assign a type to the struct. */

#include <stdio.h>
#include <stdlib.h>

typedef struct intdouble {
    int i;
    double d;
} ID;

int main()
{
    ID id1;

    id1.i = 5;
    id1.d = 3.14;

    printf("%d %.2lf\n", id1.i, id1.d);
    return 0;
}
```

typedef

Structure (Yapı)

```
/* This is a C++ program, which shows how you can copy one st  
#include <cstdio>  
#include <iostream>  
using namespace std;  
  
struct intdouble {  
    int i;  
    double d;  
};  
  
int main()  
{  
    intdouble id1, id2;  
  
    id1.i = 5;           /* Set id1 to 5 and 3.14 as before. */  
    id1.d = 3.14;  
  
    id2 = id1;           /* This makes a copy of id and then add  
    id2.i += 5;  
    id2.d += 5;  
  
    printf("1: %d %.21f\n", id1.i, id1.d); /* Print them out.  
    printf("2: %d %.21f\n", id2.i, id2.d);  
  
    return 0;  
}
```

❑ Yapılar **C++'da farklı** semantikte çalışır bu yüzden karışıklığa neden olabilir.

❑ C ve C++'da bir yapı diğerine kopyalanabilir

```
UNIX> bin/id3 1: 5 3.14  
2: 10 8.14  
UNIX>
```


Structure (Yapı)

```
/* Copying src/id3.cpp to src/id5.c, and fixing the use of intdouble
   so that it compiles. It works as in C++, copying the struct, but you
   should be wary of it. */

#include <stdio.h>
#include <stdlib.h>

struct intdouble {
    int i;
    double d;
};

int main()
{
    struct intdouble id1, id2;

    id1.i = 5;
    id1.d = 3.14;

    id2 = id1; /* THIS IS THE OFFENDING LINE */
    id2.i += 5;
    id2.d += 5;

    printf("1: %d %.21f\n", id1.i, id1.d);
    printf("2: %d %.21f\n", id2.i, id2.d);
    return 0;
}
```

❑ ([src/id4.c](#)) hatalı

❑ "intdouble"ın önüne "struct" yazmalıyız,

Structure (Yapı)

```
/* While C doesn't let you copy arrays, it lets you copy a struct that holds
   an array. I don't think this really makes sense, but there is it. In this
   code, we copy 4000 bytes in a single statement. */

#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int a[1000];
} SID;

int main()
{
    SID s1, s2;
    int i;

    for (i = 0; i < 1000; i++) s1.a[i] = i;      /* Set s1. */

    s2 = s1;      /* This statement copies 4000 bytes. */

    for (i = 0; i < 1000; i++) printf("%4d %4d\n", s1.a[i], s2.a[i]); /* Print s1 and s2. */

    return 0;
}
```

- ❑ Yapılardaki atama deyimi ile belirtilmemiş sayıda baytı kopyalayabileceğiniz C'nin tek parçasıdır.
- ❑ Bu, dilin bir zayıflığı olarak ifade edilebilir
- ❑ ``s2 = s1" satırı 4000 bayt kopyalar.

Structure (Yapı)

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int a[1000];
} SID;

void a(SID s)    /* Although this procedure changes element 999 of s, */
{               /* s is a copy of the calling parameter, so it is    */
    s.a[999] = -1; /* deleted at the end of the procedure.          */
}               /* In other words, the procedure does nothing.      */

int main()
{
    SID s1;
    int i;

    for (i = 0; i < 1000; i++) s1.a[i] = i;    /* Set the elements of s1. */

    a(s1);    /* This does nothing, because it modifies a copy of s1 */

    printf("Element 999: %d\n", s1.a[999]);

    return 0;
}
```

- ❑ Bir fonksiyona parametre olarak gönderebiliriz,
- ❑ Fakat burada kopyası oluşturulur ve yapıdaki tanımlanan dizide yapılan değişiklik prosedürden çıktıktan sonra kaybolur **yani bir değişiklik olmaz.**

Structure (Yapı)

- ❑ C++ yapıları hakkında son bir not.
- ❑ Temel olarak [sadeleştirilmiş](#) sınıflardır - içlerine yöntemler koyabilir ve ardından yapı değişkenlerini kullanarak fonksiyonları uygulayabilirsiniz.

```
/* Unlike C structs, you can put methods in C++ structs. */
#include <cstdio>
#include <iostream>
using namespace std;

struct intdouble {
    int i;
    double d;
    void Print();
};

void intdouble::Print()
{
    printf("    %d %.21f\n", i, d);
}

int main()
{
    intdouble id1, id2;

    id1.i = 5;
    id1.d = 3.14;

    id2 = id1;
    id2.i += 5;
    id2.d += 5;

    id1.Print();
    id2.Print();
    return 0;
}
```

İşaretçi (Pointer)

- ❑ İşaretçiler, çoğu insanın C'de hata yaptığı yerlerdir.
- ❑ Bir işaretçi, basitçe hafızanın bir **indeksidir**.
- ❑ Bellek, iki yoldan biriyle tahsis edilebilir -- **değişkenleri** bildirerek veya **malloc()**'u çağırarak.
- ❑ Bellek tahsis edildiğinde, ona bir işaretçi ayarlayabilirsiniz.
- ❑ Ne zaman x bayt bellek ayırsak, bellek dizisinden x **bitişik öğe ayırmış** oluyoruz. (Unutmayalım sanal bellekte)
- ❑ Bu baytlara bir işaretçi ayarlarsak, o işaretçi bellekte ayrılan ilk baytın indisi olacaktır.

Pointer

```
/* Print out pointers of local variables */

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;
    char j[14];
    int *ip;
    char *jp;

    ip = &i;
    jp = j;

    printf("ip = 0x%x.  jp = 0x%x\n", ip, jp);
    return 0;
}
```

- ❑ Bu program bir tamsayı (**i**), 14 karakterlik bir dizi (**j**) ve iki işaretçi (**ip ve jp**) ayırır.
- ❑ Daha sonra işaretçileri, **i ve j** için ayrılan belleğe işaret edecek şekilde ayarlanır.
- ❑ Son olarak, bu işaretçilerin değerlerini yazdırır - bunlar bellek dizisinin indeksleridir.
- ❑ 'long unsigned int' uyarısı alıyoruz derlerken , buna geleceğiz.
- ❑ **i değişkeni için 0x7fff2efcdd9c, 0x7fff2efcdd9d 0x7fff2efcdd9e, ve 0x7fff2efcdd9f tahsis edilmiş (int -- 4 bytes)**
- ❑ **0x7fff2efcdda0 - 0x7fff2efcddad arası j dizisi (char -- 1 byte)**

Pointer

```
/* Print out pointers of local variables */

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;
    char j[14];
    int *ip;
    char *jp;

    ip = &i;
    jp = j;

    printf("ip = 0x%lx.  jp = 0x%lx\n", ip, jp);
    return 0;
}
```

- ❑ ``jp = j" ve ``ip = &i" dikkat !
- ❑ Bunun nedeni, **dizinin bir işaretçiye eşdeğer olmasıdır.**
- ❑ Tek fark, bir dizi değişkenine değer atayamazsınız.
- ❑ Böylece ``jp = j" diyebilirsiniz, ancak ``j = jp" diyemezsiniz.

Pointer

```
#include <stdio.h>
#include <stdlib.h>

void f1(int * ptr, int len)
{
    int i;
    ptr = (int*) malloc (sizeof(int)*len);
    for (i = 0; i < len; i ++)
        ptr[i] = i;
}

int main()
{
    int i, * array;
    f1(array, 5);
    for (i = 0; i < 5; i ++)
        printf("got value %d\n", array[i]);
    return 0;
}
```


Pointer

- ❑ Yukarıdaki kodda, f1 fonksiyonu bir işaretçi ve bir uzunluk parametresi alır.
- ❑ İşaretçi önce bellekten bir dizi oluşturmak için **malloc** fonksiyonu kullanılarak yeniden ayarlanır.
- ❑ Bu dizi, 0'dan **len** değişkenine kadar sayıları içerir. Ancak, bu kodda önemli bir sorun vardır.
- ❑ Ana fonksiyonda, işaretçi değişkeni **array**, f1 fonksiyonuna geçirilir, ancak f1 içinde işaretçi **kendisi yeniden atandığından**, ana fonksiyonda orijinal işaretçi değişkeninin değeri değişmez ve dinamik olarak tahsis edilen bellek serbest bırakılmadan kalır.

Pointer

```
#include <stdio.h>

#include <stdlib.h>

void f1(int ** ptr, int len)
{   int i;

    *ptr = (int*) malloc(sizeof(int)*len);

    if (*ptr == NULL) {

        printf("Bellek tahsisi basarisiz oldu.\n");

        return;   }

    for (i = 0; i < len; i++)

        (*ptr)[i] = i;

}
```

```
int main()

{

    int i, *array;

    f1(&array, 5);

    if (array == NULL) {

        printf("Bellek tahsisi basarisiz oldu.\n");    return -1;

    }

    for (i = 0; i < 5; i++)

        printf("got value %d\n", array[i]);    free(array);

    return 0;

}
```

Pointer- Ek Örnek

```
#include <stdio.h>

void swap(int* x, int* y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int a = 5, b = 10;
    printf("a: %d, b: %d\n", a, b);
    swap(&a, &b);
    printf("a: %d, b: %d\n", a, b);
    return 0;
}
```

- ❑ Burada, **swap** fonksiyonu iki tamsayı işaretçisi alır.
- ❑ Bu işaretçiler, işaret ettikleri tamsayıların değerlerini değiştirmek için kullanılır.
- ❑ İşaretçilerin **kendileri değil, işaret ettikleri değerler** değiştirilir.
- ❑ Fonksiyonda **temp** adlı geçici bir değişken kullanılarak, değerler birbirine atanır ve değerler değiştirilir.

Pointer-Ek örnek

```
#include <stdio.h>
typedef long unsigned int LUI;

void swap(int* x, int* y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
abdullah@abdullah-VirtualBox: ~/sist_prog/hafta3
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abdullah@abdullah-VirtualBox:~/sist_prog/hafta3$ ./p1
a: 5, b: 10
*a: 0x7ffcb7650e40, *b: 0x7ffcb7650e44
a: 10, b: 5
*a: 0x7ffcb7650e40, *b: 0x7ffcb7650e44
abdullah@abdullah-VirtualBox:~/sist_prog/hafta3$
```

```
int main() {

    int a = 5, b = 10;
    int *ap, *bp;

    printf("a: %d, b: %d\n", a, b);
    ap=&a;      bp=&b;

    printf("*a: 0x%lx, *b: 0x%lx\n", (LUI) ap, (LUI) bp);

    swap(&a, &b);

    printf("a: %d, b: %d\n", *ap, *bp);

    printf("*a: 0x%lx, *b: 0x%lx\n", (LUI) ap, (LUI) bp);

    return 0;
}
```

!! Uyarılar düzeltildi

Tip dönüşümü (Type Casting)

```
/* This program assigns a char to an int, and the int to a float.
   Although it looks benign, there are some things going on
   beneath the hood (changing number formats). */

#include <stdio.h>
#include <stdlib.h>

int main()
{
    char c;
    int i;
    float f;

    c = 'a';
    i = c;
    f = i;

    printf("c = %d (%c).   i = %d (%c).   f = %f\n", c, c, i, i, f);
    return 0;
}
```

❑ Bazen x baytta depolanan bir değişkeni alıp y baytta depolanan bir değişkene atamak istersiniz.

❑ Buna ``tip dönüşümü" denir.

- **char** -- 1 byte
- **short** -- 2 bytes
- **int** -- 4 bytes
- **long** -- 4 or 8 bytes, depending on the system and compiler
- **float** -- 4 bytes
- **double** -- 8 bytes
- (pointer -- 4 or 8 bytes, depending on the system and compiler)

Tip dönüşümü

- ❑ Yukarıdakiler gibi bazı tip dönüşümleri çok doğaldır.
- ❑ C derleyicisi bunları şikayet etmeden sizin yerinize yapacaktır.
- ❑ Diğerlerinin çoğu için, özellikle bir tip dönüşümü yaptığınızı söylemediğiniz sürece, C derleyicisi bir uyarı verecektir.
- ❑ Bu, derleyiciye "Evet, ne yaptığımı biliyorum" demenin bir yoludur.

Tip dönüşümü

- ❑ Olan şu ki, derleyici printf() format dizgisini ayrıştırıyor ve "%lx" in **long unsigned int** istediğini, ancak bir (int *) alıyor.

[src/p5.c](#)

```
/* Adding typecast statements to make the
   warnings from src/p3.c go away. */

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;
    char j[14];
    int *ip;
    char *jp;

    ip = &i;
    jp = j;

    printf("ip = 0x%lx.  jp = 0x%lx\n",
        /* Here they are. */
        (long unsigned int) ip,
        (long unsigned int) jp);
    return 0;
}
```

[src/p5a.c](#)

```
/* Using a typedef to make the typecast
   statements a little less cumbersome. */

#include <stdio.h>
#include <stdlib.h>

typedef long unsigned int LU;      /* Now, I can use "(LU)" rather
                                   than "(long unsigned int)". */

int main()
{
    int i;
    char j[14];
    int *ip;
    char *jp;

    ip = &i;
    jp = j;

    printf("ip = 0x%lx.  jp = 0x%lx\n", (LU) ip, (LU) jp);
    return 0;
}
```

"Evet, bu bir (int *), ancak (long unsigned bir int) gibi davran, lütfen. Ne yaptığımı biliyorum."

Tip dönüşümü

- ❑ Bazı makinelerde (Pi gibi, 32 bit-4 byte), hem işaretçiler hem de int 4 bayttır.
- ❑ Bu, birçok insanın işaretçileri ve integer'ı birbirinin yerine geçebilir olarak görmesine yol açtı.

Tip dönüşümü

```
/* A program where we inadvisedly typecast a pointer to an int and back again.
   On machines with 8-byte pointers, this is a buggy activity, because you lose
   data when you typecast from an integer to a pointer. */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef long unsigned int LUI;

int main()
{
    char s[4];
    int i;
    char *s2;

    /* Copy the string "Jim" to s, then turn the pointer into an integer i.
       Print out the pointer's value, and i's value. */

    strcpy(s, "Jim");
    i = (int) s;
    printf("Before incrementing i.\n");
    printf("i = %d (0x%x)\n", i, i);
    printf("s = %ld (0x%lx)\n", (LUI) s, (LUI) s);

    /* Now increment i, and turn it back into a pointer.
       Print out the pointers, and then attempt to print out what they point to. */

    i++;
    s2 = (char *) i;
    printf("\n");
    printf("After incrementing i.\n");
    printf("s = 0x%lx. s2 = 0x%lx, i = 0x%x\n", (LUI) s, (LUI) s2, i);
    printf("s[0] = %c, s[1] = %c, *s2 = %c\n", s[0], s[1], *s2);
    return 0;
}
```

src/p8.c:

- ❑ i'yi s'ye eşitlediğinizde, 4 bayt bilgi kaybedersiniz, çünkü int dört bayttır ve işaretçiler sekiz bayttır.
- ❑ s2'yi tekrar i'ye ayarladığınızda, i'nin eksik olduğu dört baytı genellikle sıfırlarla, ancak bazen -1'lerle doldurur.
- ❑ Her iki durumda da, geçersiz bir adres olacak ve bir segmentasyon ihlali alacaksınız:
- ❑ 32-bit makinede bir sıkıntı olmaz

Tip dönüşümü

```
/* This is the same as src/p8.c, but we've changed i to a long. */  
  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
typedef long unsigned int LUI;  
  
int main()  
{  
    char s[4];  
    long i;  
    char *s2;  
  
    strcpy(s, "Jim");  
    i = (long) s;  
    printf("Before incrementing i.\n");  
    printf("i = %ld (0x%lx)\n", i, i);  
    printf("s = %ld (0x%lx)\n", (LUI) s, (LUI) s);  
  
    i++;  
    s2 = (char *) i;  
    printf("\n");  
    printf("After incrementing i.\n");  
    printf("s = 0x%lx. s2 = 0x%lx, i = 0x%lx\n", (LUI) s, (LUI) s2, i);  
    printf("s[0] = %c, s[1] = %c, *s2 = %c\n", s[0], s[1], *s2);  
    return 0;  
}
```

- ❑ Bunun yerine bir int yerine i için long kullanırsak, her şey yolunda gider,
- ❑ çünkü long ve işaretçilerin aynı boyutta olması garanti edilir, ister 4 ister 8 bayt olsun.

Malloc ve Free

- ❑ C'de new veya delete yoktur.
- ❑ İşlevsellikleri **malloc()** ve **free()** kütüphane çağrıları tarafından sağlanır.

```
#include <stdlib.h>
```

```
void *malloc(size_t size);  
void free(void *ptr);
```

- ❑ **New gibi** malloc() işletim sisteminden **istenilen bayt miktarını bellekten ayırır.**
- ❑ Ayırdığı veri türü hakkında bilgi vermenizi gerektiren new'in aksine, **malloc()** **yalnızca bayt sayısını** sorar ve başarılı olursa, en az o kadar bayt için işletim sistemi tarafından **bir işaretçi döndürür.**
- ❑ Bir işaretçi olduğu anlamına gelen bir void * döndürür, ancak **malloc()** **neye işaret ettiğini bilmez.**

Malloc ve Free

- ❑ malloc fonksiyonu, **programcılara çalışma zamanında (runtime) bellek ayırma olanağı** sağlar.
- ❑ Programlar çalışırken, çeşitli nedenlerden dolayı programın **ihtiyaç duyduğu bellek boyutu değişebilir**.
- ❑ Bu durumda, malloc gibi fonksiyonlar, ihtiyaç duyulan bellek boyutunu **çalışma zamanında ayrılabilir ve kullanılabilir** hale getirir.
- ❑ Ayrıca, büyük bellek blokları gibi sabit boyutlu verileri depolamak için dağıtılmamış bellek alanları oluşturmak da mümkündür.
- ❑ Ancak, bu işlem, bazı durumlarda **bellek yönetimi için gereksiz karmaşıklığa neden olabilir**. Bu nedenle, malloc fonksiyonu bu tür durumlarda daha uygun bir çözüm sunar.

Malloc ve Free

- ❑ `malloc()`'tan kaç bayta ihtiyacınız olduğunu bulmak için **`sizeof(type)`** öğesini çağırırsınız.
- ❑ Örneğin, bir tamsayı tahsis etmek için **`malloc(sizeof(int))`** işlevini çağırırsınız.
- ❑ Genellikle bir veri türü dizisi tahsis etmek isteriz. Bunu yapmak için, `sizeof(type)` öğesini öğe sayısıyla çarparsınız.
- ❑ **İşaretçiniz** bu öğelerin ilkini gösterecektir.
- ❑ Bir sonraki öğe, işaretçiden sonra `sizeof(type)` bayt olacaktır.

Malloc ve Free

```
/* The point of this program is to show how one may pass a region of bytes
   (an array) from procedure to procedure using a pointer. */

#include <stdio.h>
#include <stdlib.h>

/* This allocates n integers, error checks and returns a pointer to them. */

int *give_me_some_ints(int n)
{
    int *p;
    int i;

    p = (int *) malloc(sizeof(int) * n);
    if (p == NULL) { fprintf(stderr, "malloc(%d) failed.\n", n); exit(1); }
    return p;
}

/* This takes a pointer to n integers and assigns them to random numbers. */

void fill_in_the_ints(int *a, int n)
{
    int i;

    for (i = 0; i < n; i++) a[i] = lrand48();
}

/* This reads the command line, allocates, assigns and prints n integers. */
```

```
/* This reads the command line, allocates, assigns and prints n integers. */

int main(int argc, char **argv)
{
    int *array;
    int size;
    int i;

    if (argc != 2) { fprintf(stderr, "usage: pm size\n"); exit(1); }
    size = atoi(argv[1]);

    array = give_me_some_ints(size);
    fill_in_the_ints(array, size);

    for (i = 0; i < size; i++) printf("%4d %10d\n", i, array[i]);
    return 0;
}
```

Malloc ve Free

- ❑ `give_me_some_ints()` prosedürü, `n` tam sayıdan oluşan bir dizi tahsis eder ve diziye bir işaretçi döndürür.
- ❑ `fill_in_the_ints()`, diziye bir işaretçi artı boyutunu alır ve onu doldurur.
- ❑ İşaretçileri geçtiğimiz için, dizinin hiçbir kopyası yapılmaz.
- ❑ Başka bir deyişle, `fill_in_the_ints()`, `malloc()` çağrısı tarafından oluşturulan diziye doldurur.
- ❑ Son olarak diziye yazdırıyoruz.

Malloc ve Free

❑ C ve C++ arasındaki farklar;

1. malloc()'a kaç bayt istediğinizi söylemelisiniz.
2. malloc() belleğin nasıl kullanıldığını bilmez -- sadece baytları ayırır.
3. Dizin boyutunu takip etmelisiniz. Bu, bir vektöre kıyasla elverişsizdir.
4. C'de referans değişkenleri yoktur. Parametreler her zaman kopyalanır. Burada kopyalanan işaretçidir, işaret ettiği veri değil.
5. Bu nedenle, yukarıdaki programda dizinin yalnızca bir kopyası vardır.

Malloc ve Free

❑ free(), yeniden kullanılabilmesi için belleği serbest bırakır. C++'da silmeye (delete) benzer.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Bellek alanı ayırmak için malloc kullanımı
    int *ptr = (int*) malloc(5 * sizeof(int));

    // Bellek ayrılamazsa kontrol edin
    if (ptr == NULL) {
        printf("Bellek ayrılamadı.\n");
        exit(0);
    }
```

```
// Bellek alanını kullanmak için örnek veri yazın
for (int i = 0; i < 5; i++) {
    *(ptr + i) = i;
}
// Bellek alanında veri yazdırmak
for (int i = 0; i < 5; i++) {
    printf("%d ", *(ptr + i));
}
// Bellek alanını serbest bırakmak için free kullanımı
free(ptr);
ptr = NULL; // İyi bir uygulama için ptr'nin null'a atanması

return 0;
}
```

Sistem Programlama

DR. ÖĞR. ÜYESİ ABDULLAH SEVİN

İçerik

a) Segmentasyon İhlalleri ve Veri Yolu Hataları

b) Little-Endian

c) Struct'ların hafızada yerleşimi

d) Pointer aritmetiği

e) String işlemler

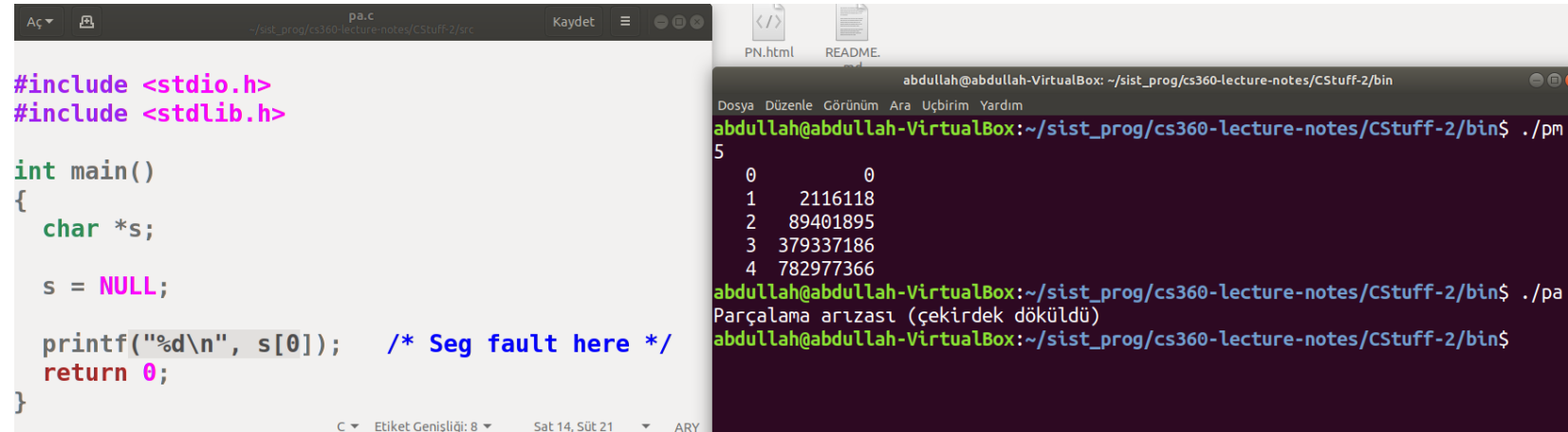
- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/CStuff-2/lecture.html>
- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/Pointer-Arithmetic/index.html>
- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/Strings-In-C/index.html>

Segmentasyon ihlalleri

- ❑ Bellek, dev bir bayt dizisidir.
- ❑ Ancak, bu dizinin belirli bölümlerine erişilemez.
- ❑ Örneğin, genelde 0 - 0x1000 öğelerine erişilemez.
- ❑ Erişilemeyen bir öğeye erişmeye çalıştığınızda, bir segmentasyon ihlali oluşturursunuz.
- ❑ (Eskiden, belleğin içeriğini çekirdek dökümü (core dump) adı verilen bir dosyada da depolardık. Bu günlerde bellek o kadar büyük ki, yapabilmemize rağmen çekirdek dökümleri (core dump) oluşturmuyoruz.)
- ❑ Segmentasyon ihlali; bir pointerı başlatmayı unuttuğumuzda ve ilk adresini yazdırmak istersek.
- ❑ (core dump or a crash dump is a memory snapshot of a running process)

Segmentasyon ihlalleri

❏ Örnek:



The image shows a code editor on the left and a terminal window on the right. The code editor displays a C program named `pa.c` with the following content:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char *s;

    s = NULL;

    printf("%d\n", s[0]);    /* Seg fault here */
    return 0;
}
```

The terminal window shows the execution of the program. It starts with the command `./pm` and displays the output:

```
5
0      0
1    2116118
2    89401895
3    379337186
4    782977366
```

Following the memory dump, the terminal shows the command `./pa` and the error message:

```
Parçalama arızası (çekirdek döküldü)
```

The terminal window title is `abduallah@abduallah-VirtualBox: ~/sist_prog/cs360-lecture-notes/CStuff-2/bin`.

Veri Yolu Hataları

- ❑ Birçok makinede, bir skaler türe her eriştiğinizde, bellekteki değeri sıralı olmalıdır.
- ❑ Bunun anlamı, veri türü 4 bayt ise, bellekteki konumu 4'ün katı olan bir bellek dizininde başlamalıdır.
- ❑ Örneğin, `i` bir `(int *)` ise, o zaman `i` 4'ün katı değilse , bu hatalara sebebiyet verecektir. Bu hata, bir veri yolu hatasıyla kendini gösterir.

Little Endian

- ❑ C ile, bellek ile ilgili "tehlikeli" şeyler yapabilirsiniz.
- ❑ Başka bir deyişle, bir bayt bölgeniz olduğunu varsayalım. Bu bölgeye, istediğiniz herhangi bir türü tutabiliriz- tamsayılar, karakterler, double, struct vs.
- ❑ Tipik olarak, bu esneklikten yararlanmak istemezsiniz, çünkü bu sizi çok fazla belaya sokabilir.
- ❑ Ancak, sistem programları yazarken, bu **esneklik** genellikle önemlidir.
- ❑ Ayrıca, programlar çalışırken bellekte ve makinenizde neler olup bittiğini anlamanız önemlidir!

Little Endian

```
#include <stdio.h>

typedef unsigned long UL;

int main()
{
    unsigned int array[4]; /* An array of four integers. */
    unsigned int *ip;      /* An integer pointer that we're going to set to one byte beyond array */
    unsigned char *cp;     /* An unsigned char pointer for exploring the individual bytes in array */
    unsigned short *sp;    /* An unsigned short to show two-byte access. */
    int i;

    /* Set array to equal four integers, which we specify in hexadecimal. */

    array[0] = 0x12345678;
    array[1] = 0x9abcdef0;
    array[2] = 0x13579bdf;
    array[3] = 0x2468ace0;

    /* For each value of array, print it out in hexadecimal. Also print out its location in memory. */

    for (i = 0; i < 4; i++) {
        printf("Array[%d]'s location in memory is 0x%lx. Its value is 0x%x\n",
            i, (UL) (array+i), array[i]);
    }
```


Little Endian

```
/* Now, print out the sixteen bytes as bytes, printing each byte's location first. */

printf("\n");
printf("Viewing the values of array as bytes:\n");
printf("\n");

cp = (unsigned char *) array;

for (i = 0; i < 16; i++) {
    printf("Byte %2d. Pointer: 0x%lx - Value: 0x%02x\n", i, (UL) (cp+i), cp[i]);
}
```

- ❑ Çıktıya, özellikle bayt değerlerine dikkat edin. İlk başta karışık gelebilir!
- ❑ İlk baytın 0x12 olmasını beklemiyor muydunuz?

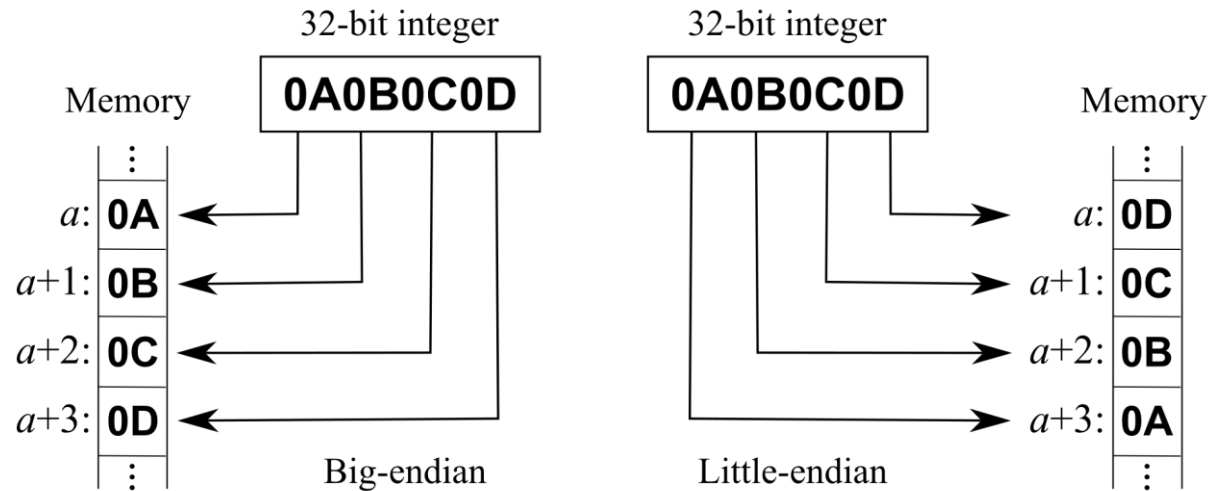
```
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/CStuff-2/bin$ ./endian
Array[0]'s location in memory is 0x7ffda6bd4960. Its value is 0x12345678
Array[1]'s location in memory is 0x7ffda6bd4964. Its value is 0x9abcdef0
Array[2]'s location in memory is 0x7ffda6bd4968. Its value is 0x13579bdf
Array[3]'s location in memory is 0x7ffda6bd496c. Its value is 0x2468ace0
```

Viewing the values of array as bytes:

Byte	0.	Pointer:	0x7ffda6bd4960	- Value:	0x78
Byte	1.	Pointer:	0x7ffda6bd4961	- Value:	0x56
Byte	2.	Pointer:	0x7ffda6bd4962	- Value:	0x34
Byte	3.	Pointer:	0x7ffda6bd4963	- Value:	0x12
Byte	4.	Pointer:	0x7ffda6bd4964	- Value:	0xf0
Byte	5.	Pointer:	0x7ffda6bd4965	- Value:	0xde
Byte	6.	Pointer:	0x7ffda6bd4966	- Value:	0xbc
Byte	7.	Pointer:	0x7ffda6bd4967	- Value:	0x9a
Byte	8.	Pointer:	0x7ffda6bd4968	- Value:	0xdf
Byte	9.	Pointer:	0x7ffda6bd4969	- Value:	0x9b
Byte	10.	Pointer:	0x7ffda6bd496a	- Value:	0x57
Byte	11.	Pointer:	0x7ffda6bd496b	- Value:	0x13
Byte	12.	Pointer:	0x7ffda6bd496c	- Value:	0xe0
Byte	13.	Pointer:	0x7ffda6bd496d	- Value:	0xac
Byte	14.	Pointer:	0x7ffda6bd496e	- Value:	0x68
Byte	15.	Pointer:	0x7ffda6bd496f	- Value:	0x24

Little-Endian vs Big-Endian

- ❑ Little-Endian: Verinin düşük değerlikli bitleri bellekte ilk sıralanır.
- ❑ Big-Endian: Verinin yüksek değerlikli bitleri bellekte ilk sıralanır.
- ❑ Aynı Arapça veya Türkçedeki sağdan-soldan yazım kuralı gibi işlemcilerde özgü bellek yazım biçimi



Little-Endian

<u>Memory</u>		array = 0xbff344a8				(array+2) = 0xbff344b0				(array+1) = 0xbff344ac				(array+3) = 0xbff344b4					
		↓				↓				↓				↓					
Addresses	0	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x		
	b	b	b	b	b	b	b	b	b	b	b	b	b	b	b	b	b		
	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f		
	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3		
Values as bytes (hex)	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4		
	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4		
	a	a	a	a	a	a	a	a	b	b	b	b	b	b	b	b	b		
	7	8	9	a	b	c	d	e	f	0	1	2	3	4	5	6	7	8	
		?	78	56	34	12	f0	de	bc	9a	df	9b	57	13	e0	ac	68	24	?
		array[0] as an integer is: 0x12345678				array[1] as an integer is: 0x9abcdef0				array[2] as an integer is: 0x13579bdf				array[3] as an integer is: 0x2468ace0					

Little-Endian

- ❑ Şimdi cp++ yaptık ve ip, (ip+1), (ip+2) ve (ip+3) tarafından gösterilen dört tam sayıyı yazdırıyoruz.

```
/* Finally, set the pointer ip to be one byte greater than array,  
   and then print out locations and integers. */  
  
printf("\n");  
printf("Setting the pointer ip to be one byte greater than array:\n");  
printf("\n");  
  
cp++;  
ip = (unsigned int *) cp;  
for (i = 0; i < 4; i++) {  
    printf("(ip+%d) is 0x%lx. *(ip+%d) is 0x%x\n", i, (UL) (ip+i), i, *(ip+i));  
}
```

Setting the pointer ip to be one byte greater than array:

```
(ip+0) is 0x7ffda6bd4961. *(ip+0) is 0xf0123456  
(ip+1) is 0x7ffda6bd4965. *(ip+1) is 0xdf9abcde  
(ip+2) is 0x7ffda6bd4969. *(ip+2) is 0xe013579b  
(ip+3) is 0x7ffda6bd496d. *(ip+3) is 0x602468ac
```

- ❑ Bazı makinelerde, ip dördün katı olmadığı için bu kodda bir veri yolu hatası olacaktır. Çıktıyı kafa karıştırıcı bulabilirsiniz ama merak etmeyin, üzerinden geçeceğiz

Little-Endian

Ayrıca, 0x612468ac'deki "61" orijinal diziden olmayan bir bayttır. Başka bir değeri olabilir - ne olması gerektiğini gerçekten bilemeyiz. (Benim kodda 60 çıktı)

<u>Memory</u>		ip = 0xbff344a9				(ip+2) = 0xbff344b1				(ip+1) = 0xbff344ad				(ip+3) = 0xbff344b5			
		↓				↓				↓				↓			
Addresses	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x
	b	b	b	b	b	b	b	b	b	b	b	b	b	b	b	b	b
	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f	f
Values as bytes (hex)	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
	a	a	a	a	a	a	a	a	b	b	b	b	b	b	b	b	b
		7	8	9	a	b	c	d	e	f	0	1	2	3	4	5	6
		?	78	56	34	12	f0	de	bc	9a	df	9b	57	13	e0	ac	68
																	24
																	61
		*ip as an integer is: 0xf0123456				*(ip+1) as an integer is: 0xdf9abcde				*(ip+2) as an integer is: 0xe013579b				*(ip+3) as an integer is: 0x612468ac			

Little-Endian

- ❑ Tamsayılar gibi, short'da little-endian ile temsil edilir, bu nedenle ilk short verinin en küçük baytı 0x56 değeriyle 0xbff344a9 adresindedir ve ilk short değerin en büyük baytı 0x78 değeriyle 0xbff344a8 adresindedir:

```
/* Now, set sp to equal array.  Sp is a pointer to shorts.  We print out sp[0] and sp[1]. */  
  
printf("\n");  
printf("Finally printing the first four bytes of array as two shorts.\n");  
printf("\n");  
  
sp = (unsigned short *) array;  
printf("Location: 0x%lx - Value as a short: 0x%04x\n", (UL) sp, sp[0]);  
printf("Location: 0x%lx - Value as a short: 0x%04x\n", (UL) (sp+1), sp[1]);  
printf("\n");  
  
return 0;  
}
```

Finally printing the first four bytes of array as two shorts.

```
Location: 0x7ffc9b3f6d60 - Value as a short: 0x5678  
Location: 0x7ffc9b3f6d62 - Value as a short: 0x1234
```

Yapıların (Struct) yerleşimi

- ❑ Bazı makineler işaretçilerin bellekte belli aralıklarla hizalanmasını gerektirir.
- ❑ Bu, **integer** işaretçiler için dördün katları, **double** işaretçilerin sekizin katları ve **short** işaretçilerinin ikinin katları olması gerektiği anlamına gelir.
- ❑ Bu gereksinimi karşılamak için derleyiciler ve çalışma zamanı kütüphaneleri (runtime libraries) iki özellekle tasarlanmıştır:
 1. Malloc() her zaman **8'in katları olan işaretçiler döndürür**. Malloc()'un ayırdığı belleği kullanacak veri türünü bilmediğini unutmayın. Bu nedenle, sadece güvende olmak için her zaman 8'in katlarını döndürür.
 2. Derleyici, değişkenleri bellekte sıralı olacak şekilde yapıları(struct) düzenler ve yapının kendisinin temel işaretçisi sekizin katıysa bunlar hizalanır. Bu, derleyicinin bir yapıya biraz **dolgu** koyabileceği ve onu olması gerektiğini düşündüğünüzden daha büyük yapabileceği anlamına gelir.

Yapıların (Struct) yerleşimi

```
typedef struct {  
    char b;  
    int i;  
} Char_Int;
```

```
typedef struct {  
    char b1;  
    char b2;  
    char b3;  
    char b4;  
    int i1;  
} CCCC_Int;
```

```
typedef struct {  
    char b1;  
    int i1;  
    char b2;  
    int i2;  
} C_I_C_I;
```

```
typedef struct {  
    int i;  
    char b;  
} Int_Char;
```

```
Char_Int *ci;
```

```
ci = (Char_Int *) malloc(sizeof(Char_Int)*2);  
printf("The size of a Char_Int is %ld\n", sizeof(Char_Int));  
printf("I have allocated an array, ci, of two Char_Int's at location 0x%lx\n", (UL) ci);  
printf("&(ci[0].b) = 0x%lx\n", (UL) &(ci[0].b));  
printf("&(ci[0].i) = 0x%lx\n", (UL) &(ci[0].i));  
printf("&(ci[1].b) = 0x%lx\n", (UL) &(ci[1].b));  
printf("&(ci[1].i) = 0x%lx\n", (UL) &(ci[1].i));  
printf("\n");
```


Yapıların (Struct) yerleşimi

- ❑ Yukarıdaki örnekte yer alan Yapı(struct) yalnızca beş bayt kullansa da (b için bir ve i için dört bayt), yapının boyutu 8 bayttır.
- ❑ İşaretçilere baktığımızda b'yi bulduğunuz yerde struct'ın ilk byte'ı olduğunu ve b'den sonra i'nin 4 byte olduğunu görüyoruz.
- ❑ b ve i arasındaki üç bayt kullanılmaz. Neden böyle?
- ❑ Bunun nedeni, i işaretçisinin dördün katı olması ve b'nin adresinin i'nin adresinden önce gelmesi gerektiğidir (bu bir derleyici standardıdır).
- ❑ Bunu yapmanın tek yolu, b'den sonraki üç baytın kullanılmamasıdır.

Yapıların (Struct) yerleşimi

- ❑ CCCC_Int yapısının çıktısına bakalım.
- ❑ Gördüğünüz gibi, bu yapı, belleği en iyi şekilde kullanır -- değişkenler 8 bayt yer kaplar ve veri yapısının boyutu 8'dir. i işaretçisi hizalanmıştır - Mükemmel!

```
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/CStuff-2/bin$ ./pd
The size of a Char_Int is 8
I have allocated an array, ci, of two Char_Int's at location 0x55f989718260
&(ci[0].b) = 0x55f989718260
&(ci[0].i) = 0x55f989718264
&(ci[1].b) = 0x55f989718268
&(ci[1].i) = 0x55f98971826c

The size of a CCCC_Int is 8
I have allocated an array, cccci, of two CCCC_Int's at location 0x55f989718690
&(cccci[0].b1) = 0x55f989718690
&(cccci[0].b2) = 0x55f989718691
&(cccci[0].b3) = 0x55f989718692
&(cccci[0].b4) = 0x55f989718693
&(cccci[0].i1) = 0x55f989718694
&(cccci[1].b1) = 0x55f989718698
&(cccci[1].b2) = 0x55f989718699
&(cccci[1].b3) = 0x55f98971869a
&(cccci[1].b4) = 0x55f98971869b
&(cccci[1].i1) = 0x55f98971869c
```

Yapıların (Struct) yerleşimi

- ❑ C_I_C_I yapısı daha az optimal olarak düzenlenmiştir; i1 ve i2 işaretçilerinin yerleşiminden emin olmak için, b1'den sonraki üç baytı ve b2'den sonraki üç baytı boşa harcamalıyız.
- ❑ b1'den hemen sonra b2'yi tanımlasaydık, veri yapısının boyutu 16 yerine 12 olurdu.

```
The size of a C_I_C_I is 16
I have allocated an array, cici, of two C_I_C_I's at location 0x55f9897186b0
&(cici[0].b1) = 0x55f9897186b0
&(cici[0].i1) = 0x55f9897186b4
&(cici[0].b2) = 0x55f9897186b8
&(cici[0].i2) = 0x55f9897186bc
&(cici[1].b1) = 0x55f9897186c0
&(cici[1].i1) = 0x55f9897186c4
&(cici[1].b2) = 0x55f9897186c8
&(cici[1].i2) = 0x55f9897186cc
```

```
The size of a Int_Char is 8
I have allocated an array, ic, of two Int_Char's at location 0x55f9897186e0
&(ic[0].i) = 0x55f9897186e0
&(ic[0].b) = 0x55f9897186e4
&(ic[1].i) = 0x55f9897186e8
&(ic[1].b) = 0x55f9897186ec
```

Yapıların (Struct) yerleşimi

- ❑ Şimdi, `Int_Char` için son çıktı satırlarına bakın. Sizi şaşırtabilir:
- ❑ Çoğumuz `sizeof(Int_Char)` öğesinin beş olması gerektiğini düşünür çünkü `i` işaretçisi yapının başlangıcıdır ve hizalanması gerekir ve `b` işaretçisinin yerleşim konusunda endişelenmesi gerekmez.
- ❑ Bu yapılardan yalnızca birini tahsis edersek bu doğru olur. Ancak, eğer iki yapıdan oluşan bir dizi tahsis edersek, o zaman ikincisinin de hizalanması gerekir -- eğer yapının boyutu 5 olsaydı, o zaman `ic[1].i` hizalanmazdı. Bunu düzeltmek için yapının boyutu 8'dir ve `b`'den sonraki üç bayt boşa gider.

```
The size of a Int_Char is 8
I have allocated an array, ic, of two Int_Char's at location 0x55f9897186e0
&(ic[0].i) = 0x55f9897186e0
&(ic[0].b) = 0x55f9897186e4
&(ic[1].i) = 0x55f9897186e8
&(ic[1].b) = 0x55f9897186ec
```

Yaygın Bir Hata Türü

```
/* This program shows how you lose information as you
   convert data from larger types to smaller types. */

#include <stdio.h>
#include <stdlib.h>

int main()
{
    char c;
    int i;
    int j;

    i = 10000;
    c = i;          /* We are losing information here, because 10000 cannot be stored in a byte. */
    j = c;

    printf("I: %d,   J: %d,       C: %d\n", i, j, c);
    printf("I: 0x%04x, J: 0x%04x,   C: 0x%04x\n", i, j, c);
    return 0;
}
```

Yaygın Bir Hata Türü

- ❑ c bir karakter olduğu için 10000 değerini tutamaz.
- ❑ Bunun yerine i'nin en düşük baytını, yani 16'yı (0x10) tutar.
- ❑ Sonra j'yi c'ye ayarladığınızda, j'nin 16 olduğunu göreceksiniz.

Pointer Aritmetiği

- ❑ Değişken bildiriminde [size] koyarak bir diziye statik olarak deklare edebiliriz.
- ❑ Örneğin, aşağıdaki değişken bildirimi, on tam sayıdan oluşan bir **iarray** dizisi oluşturacaktır:

```
int iarray[10];
```

- ❑ iarray öğelerine köşeli parantez içinde erişebilirsiniz. Özellikle, iarray'in boyutu hiçbir yerde saklanmaz -- onu kendiniz takip etmeniz gerekir.
- ❑ Gerçekte iarray, dizinin ilk elemanına bir işaretçidir. Başka bir deyişle, dizi için ayrılan 40 bayt vardır (çünkü tamsayıların her biri dört bayttır) ve iarray bunlardan ilkinin işaret eder.
- ❑ İstersek iarray'e ikinci bir işaretçi ayarlayabiliriz ve işaretçiyi artırıp başvurusunu kaldırarak iarray'in öğelerini yazdırabiliriz.

Pointer Aritmetiği

□ For döngüsünde 5 nicelik yazdıracağız. İnceleyelim;

```
/* This program sets a pointer to an array, and then dereferences each of the
   elements of the array using the pointer and pointer arithmetic. It prints
   the pointers in hexadecimal while it does so. */

#include <stdio.h>
#include <stdlib.h>

typedef unsigned long (UL);

int main()
{
    int iarray[10];
    int *ip;
    int i;

    /* Set the 10 elements of iarray to be 100 to 109, and print the array's address. */

    for (i = 0; i < 10; i++) iarray[i] = 100+i;
    printf("iarray = 0x%lx\n", (UL) iarray);
```


Pointer Aritmetiği

```
/* Set ip equal to array, and then print the 10 elements using both iarray and ip.  
The following quantities will be printed for each element:
```

- The index i (goes from 0 to 9)
- The value of iarray[i] (goes from 100 to 109)
- The pointer ip (will start at iarray and increment by four each time).
- What *ip points to (this will be 100 to 109 again)
- Pointer arithmetic: (ip-array) -- this will be the value of i.

```
*/
```

```
ip = iarray;
```

```
for (i = 0; i < 10; i++) {  
    printf("i=%d.  ",      i              );  
    printf("iarray[i]=%d.  ", iarray[i]    );  
    printf("ip = 0x%lx.  ", (UL) ip        );  
    printf("*ip=%d.  ",    *ip            );  
    printf("(ip-iarray)=%ld.\n", (UL) (ip-iarray));  
    ip++;  
}
```

```
return 0;
```

```
}
```

Pointer Aritmetiği

UNIX> **bin/ptr**

iarray = 0x7fff5fbfdc40

i=0. iarray[i]=100. ip = 0x7fff5fbfdc40. *ip=100. (ip-iarray)=0

i=1. iarray[i]=101. ip = 0x7fff5fbfdc44. *ip=101. (ip-iarray)=1

i=2. iarray[i]=102. ip = 0x7fff5fbfdc48. *ip=102. (ip-iarray)=2

i=3. iarray[i]=103. ip = 0x7fff5fbfdc4c. *ip=103. (ip-iarray)=3

i=4. iarray[i]=104. ip = 0x7fff5fbfdc50. *ip=104. (ip-iarray)=4

i=5. iarray[i]=105. ip = 0x7fff5fbfdc54. *ip=105. (ip-iarray)=5

i=6. iarray[i]=106. ip = 0x7fff5fbfdc58. *ip=106. (ip-iarray)=6

i=7. iarray[i]=107. ip = 0x7fff5fbfdc5c. *ip=107. (ip-iarray)=7

i=8. iarray[i]=108. ip = 0x7fff5fbfdc60. *ip=108. (ip-iarray)=8

i=9. iarray[i]=109. ip = 0x7fff5fbfdc64. *ip=109. (ip-iarray)=9

UNIX>

Pointer Aritmetiği

- ❑ Hex ile ifade edilen adresler makineden makineye değişir.
- ❑ Ancak aralarındaki ilişki her zaman aynı olacaktır.
- ❑ Bu program çalışmaya başladığında, işletim sistemi onu 0x7fff5fbfdc40 ile başlayan 40 baytın iarray'in saklandığı yer olmasını sağlayacak şekilde kurmuştur. Bu nedenle iarray, 0x7fff5fbfdc40'a eşittir.
- ❑ iarray[0], 0x7fff5fbfdc40'ta başlayan dört baytsa, o zaman iarray[1], 0x7fff5fbfdc44'te başlayan dört bayt olmalıdır.
- ❑ Bu nedenle ip, for döngüsünün ikinci yinelemesinde 0x7fff5fbfdc44'e eşittir.
- ❑ ip'e bir eklemek aslında işaretçinin değerine dört ekler. Buna "**işaretçi aritmetiği**" denir - bir işaretçiye x eklediğinizde, gerçekten ona **sx** ekler, burada s, işaretçinin işaret ettiği verinin boyutudur.

Pointer Aritmetiği

- ❑ For döngüsünde yazdırılan son sütun da biraz kafa karıştırıcı.
- ❑ `ip`, `0x7fff5fbfdc44`'e eşittir, bu nedenle `(ip-iarray)`'nin dört eşit olacağını düşünürsünüz. Değil, çünkü derleyici işaretçi aritmetiği yapıyor -- derleyicinin bakış açısından, "`ip-iarray`" dediğinizde, `ip` ve `iarray` arasındaki öğelerin sayısını soruyorsunuz.
- ❑ Bu, öğenin boyutuna bölünen işaretçiler arasındaki fark olacaktır. Bu durumda, $(0x7fff5fbfdc44 - 0x7fff5fbfdc40) / 4$, bire eşittir.

Pointer Aritmetiği

```
/* This is the same as src/iptr.c, except iarray is an array of "Two" structs instead
   of ints.  The "Two" struct is simply a struct of two doubles. */

#include <stdio.h>
#include <stdlib.h>

typedef struct {
    double d1;
    double d2;
} Two;

#include <stdio.h>
#include <stdlib.h>

typedef unsigned long (UL);

int main()
{
    Two iarray[10];
    Two *ip;
    int i;

    for (i = 0; i < 10; i++) iarray[i].d1 = 100+i;    /* Set the d1 field to be 100 + i */
    for (i = 0; i < 10; i++) iarray[i].d2 = 200+i;    /* Set the d2 field to be 200 + i */
}
```

Pointer Aritmetiği

```
printf("iarray = 0x%lx\n", (UL) iarray);

ip = iarray;

for (i = 0; i < 10; i++) {
    printf("i=%d. ", i);
    printf("iarray[i]={%.21f,%.21f}. ", iarray[i].d1, iarray[i].d2);
    printf("ip = 0x%lx. ", (UL) ip);
    printf("*ip={%.21f,%.21f}. ", ip->d1, ip->d2);
    printf("(ip-iarray)=%ld.\n", (UL) (ip-iarray));
    ip++;
}

return 0;
}
```

Pointer Aritmetiği

UNIX> **bin/sptr**

iarray = 0x7ffeed0e10f0

i=0. iarray[i]={100.00,200.00}. ip = 0x7ffeed0e10f0. *ip={100.00,200.00}. (ip-iarray)=0.
i=1. iarray[i]={101.00,201.00}. ip = 0x7ffeed0e1100. *ip={101.00,201.00}. (ip-iarray)=1.
i=2. iarray[i]={102.00,202.00}. ip = 0x7ffeed0e1110. *ip={102.00,202.00}. (ip-iarray)=2.
i=3. iarray[i]={103.00,203.00}. ip = 0x7ffeed0e1120. *ip={103.00,203.00}. (ip-iarray)=3.
i=4. iarray[i]={104.00,204.00}. ip = 0x7ffeed0e1130. *ip={104.00,204.00}. (ip-iarray)=4.
i=5. iarray[i]={105.00,205.00}. ip = 0x7ffeed0e1140. *ip={105.00,205.00}. (ip-iarray)=5.
i=6. iarray[i]={106.00,206.00}. ip = 0x7ffeed0e1150. *ip={106.00,206.00}. (ip-iarray)=6.
i=7. iarray[i]={107.00,207.00}. ip = 0x7ffeed0e1160. *ip={107.00,207.00}. (ip-iarray)=7.
i=8. iarray[i]={108.00,208.00}. ip = 0x7ffeed0e1170. *ip={108.00,208.00}. (ip-iarray)=8.
i=9. iarray[i]={109.00,209.00}. ip = 0x7ffeed0e1180. *ip={109.00,209.00}. (ip-iarray)=9.

- ❑ Gördüğünüz gibi, işaretçiler iptr'nin çalışmasından farklıdır. Ancak aralarındaki ilişki aynıdır ve ip'yi her artırdığınızda değeri 16 artar çünkü yapının(struct) boyutu 16 bayttır.

String-strcpy()

`char *strcpy(char *s1, const char *s2);`

- ❑ Strcpy(), s2'nin boş/null ile sonlandırılmış bir dize olduğunu ve s1'in, sondaki boş karakter de dahil olmak üzere s2'yi tutmak için yeterli karaktere sahip bir (char *) olduğunu varsayar.
- ❑ Strcpy() daha sonra s2'yi s1'e kopyalar.
- ❑ Ayrıca s1 döndürür.

String Örnek

```
/* Initialize three strings using strcpy() and print them. */

#include <stdio.h>
#include <string.h>

int main()
{
    char give[5];
    char him[5];
    char six[5];

    strcpy(give, "Give");
    strcpy(him, "Him");
    strcpy(six, "Six!");

    printf("%s %s %s\n", give, him, six);
    return 0;
}
```

String Örnek

```
/* What happens when you call strcpy and didn't allocate enough memory? */

#include <stdio.h>
#include <string.h>

typedef unsigned long UL;

int main()
{
    char give[5];
    char him[5];
    char six[5];

    /* Print the addresses of the three arrays. */

    printf("give: 0x%lx  him: 0x%lx  six: 0x%lx\n", (UL) give, (UL) him, (UL) six);

    /* This is the same as before -- nice strcpy() statements, and then print. */

    strcpy(give, "Give");
    strcpy(him, "Him");
    strcpy(six, "Six!");
    printf("%s %s %s\n", give, him, six);

    /* Now, this strcpy() is copying a string that is too big. */

    strcpy(him, "T.J. Houshmandzadeh");
    printf("%s %s %s\n", give, him, six);

    return 0;
}
```

String Örnek

- ❑ Açıkça yukarıdaki örnekle ilgili bir sorun var -- "T.J. Houshmandzadeh" dizisi beş karakterden çok daha büyük.
- ❑ bazı derleyiciler bunu derleyecek, ancak bazıları diğerleri bununla ilgili sıkıntı çıkartabilir.

```
UNIX> gcc -o bin/strcpy2 src/strcpy2.c
```

```
src/strcpy2.c: In function 'main':
```

```
src/strcpy2.c:21: warning: call to __builtin___strcpy_chk will always overflow destination buffer
```

```
UNIX>
```

- ❑ Bu akıllı bir derleyici. Ancak, derleyiciler her şeyi gören ve her şeyi bilen değildir. strcpy() etrafına kendi ifadelerimizi yazarak onu kandırabiliriz -- şimdi sorunu çözemez.

String

```
/* This is the same as strcpy2.c, but I write a procedure to call strcpy(), so that
   even a smart compiler won't figure out that I have a problem. */

#include <stdio.h>
#include <string.h>

typedef unsigned long UL;

void my_strcpy(char *s1, char *s2)
{
    strcpy(s1, s2);
}

int main()
{
    char give[5];
    char him[5];
    char six[5];

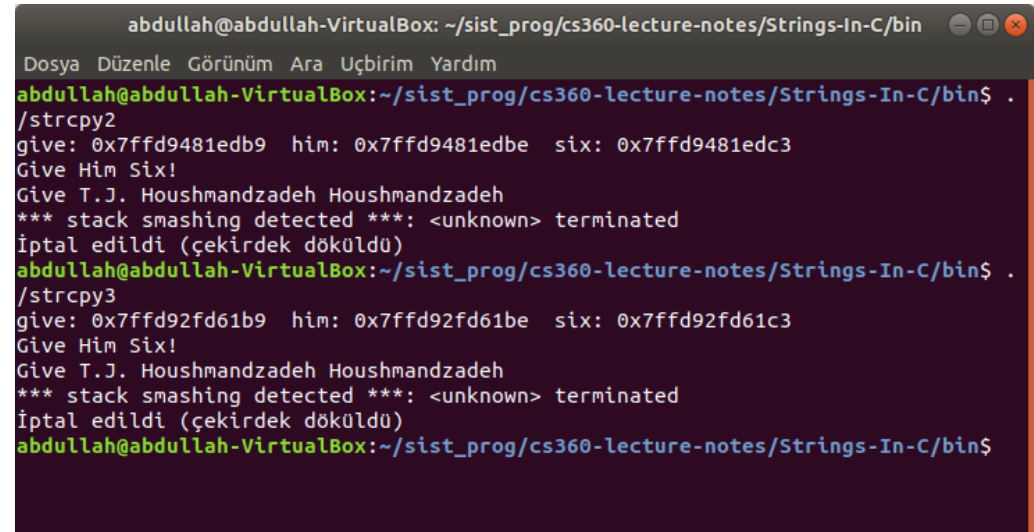
    printf("give: 0x%lx him: 0x%lx six: 0x%lx\n", (UL) give, (UL) him, (UL) six);

    strcpy(give, "Give");
    strcpy(him, "Him");
    strcpy(six, "Six!");

    printf("%s %s %s\n", give, him, six);

    my_strcpy(him, "T.J. Houshmandzadeh");

    printf("%s %s %s\n", give, him, six);
    return 0;
}
```



```
abdullah@abdullah-VirtualBox: ~/sist_prog/cs360-lecture-notes/Strings-In-C/bin
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/Strings-In-C/bin$ .
./strcpy2
give: 0x7ffd9481edb9 him: 0x7ffd9481edbe six: 0x7ffd9481edc3
Give Him Six!
Give T.J. Houshmandzadeh Houshmandzadeh
*** stack smashing detected ***: <unknown> terminated
İptal edildi (çekirdek döküldü)
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/Strings-In-C/bin$ .
./strcpy3
give: 0x7ffd92fd61b9 him: 0x7ffd92fd61be six: 0x7ffd92fd61c3
Give Him Six!
Give T.J. Houshmandzadeh Houshmandzadeh
*** stack smashing detected ***: <unknown> terminated
İptal edildi (çekirdek döküldü)
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/Strings-In-C/bin$
```

String-strcat()

`char *strcat(char *s1, const char *s2);`

- ❑ Strcat(), s1 ve s2'nin her ikisinin de boş/null ile sonlandırılmış dizeler olduğunu varsayar.
- ❑ Strcat() daha sonra s2'yi s1'in sonuna birleştirir.
- ❑ Strcat(), s1'de bu ekstra karakterleri tutmak için yeterli alan olduğunu varsayar. Aksi takdirde, ayırmadığınız hafızayı ezmeye başlarsınız.

```
UNIX> bin/strcat
Give
Give Him
Give Him Six!
UNIX>
```

```
/* Using strcpy() and strcat() to create the string "Give Him Six!" incrementally. */
#include <stdio.h>
#include <string.h>

int main()
{
    char givehimsix[15];

    strcpy(givehimsix, "Give");
    printf("%s\n", givehimsix);
    strcat(givehimsix, " Him");
    printf("%s\n", givehimsix);
    strcat(givehimsix, " Six!");
    printf("%s\n", givehimsix);
    return 0;
}
```

Soru

1-) komut satırından 5 tane kelimeyi (en fazla 10 karakter) bir char dizisine tek tek ekleyelim (strcat). Sonra bu dizinin adreslerini yazdıralım ve char pointer ile diziye işaret edip yazılış sırasını bozmadan ekrana tekrar yazdıralım.

2-) komut satırından 5 tane integer değer alın . Bunları bir char dizisine(tek boyut) ekleyelim (strcat). Sonra bu dizinin adreslerini yazdıralım ve char pointer ile diziye işaret edip yazılış sırasını bozmadan ekrana tekrar yazdıralım.

Sistem Programlama

DR. ÖĞR. ÜYESİ ABDULLAH SEVİN

İçerik

a) String işlemler

- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/Strings-In-C/index.html>
- Strings-In-C Klasör Örnekleri

String-strcat()

`char *strcat(char *s1, const char *s2);`

- ❑ Strcat(), s1 ve s2'nin her ikisinin de boş/null ile sonlandırılmış dizeler olduğunu varsayar.
- ❑ Strcat() daha sonra s2'yi s1'in sonuna birleştirir.
- ❑ Strcat(), s1'de bu ekstra karakterleri tutmak için yeterli alan olduğunu varsayar. Aksi takdirde, ayırmadığınız hafızayı ezmeye başlarsınız.

```
UNIX> bin/strcat
Give
Give Him
Give Him Six!
UNIX>
```

```
/* Using strcpy() and strcat() to create the string "Give Him Six!" incrementally. */
#include <stdio.h>
#include <string.h>

int main()
{
    char givehimsix[15];

    strcpy(givehimsix, "Give");
    printf("%s\n", givehimsix);
    strcat(givehimsix, " Him");
    printf("%s\n", givehimsix);
    strcat(givehimsix, " Six!");
    printf("%s\n", givehimsix);
    return 0;
}
```

String-strcat()

❑ src/strcat2.c'ye bakalım. Çıktının neden böyle olduğunu açıklayabilir misiniz?

```
char give[16];  
char him[5];  
char six[5];
```

```
abduallah@abduallah-VirtualBox: ~/sist_prog/cs360-lecture-notes/Strings-In-C/src  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
abduallah@abduallah-VirtualBox:~/sist_prog/cs360-lecture-notes/Strings-In-C/src$ ./strcat22  
give: 0x7ffeef830500 him: 0x7ffeef8304f6 six: 0x7ffeef8304fb  
Give Him Six!  
mandzadeh T.J. Houshmandzadeh Houshmandzadeh  
give: 0x7ffeef830500 him: 0x7ffeef8304f6 six: 0x7ffeef8304fb  
mandzadeh Help! T.J. Houshmandzadeh Help! Houshmandzadeh Help!
```

```
char give[16];  
char him[16];  
char six[16];
```

```
abduallah@abduallah-VirtualBox: ~/sist_prog/cs360-lecture-notes/Strings-In-C/src  
Dosya Düzenle Görünüm Ara Uçbirim Yardım  
abduallah@abduallah-VirtualBox:~/sist_prog/cs360-lecture-notes/Strings-In-C/src$ ./strcat22  
give: 0x7ffd6ff3ed90 him: 0x7ffd6ff3eda0 six: 0x7ffd6ff3edb0  
Give Him Six!  
Give T.J. Houshmandzadeh deh  
give: 0x7ffd6ff3ed90 him: 0x7ffd6ff3eda0 six: 0x7ffd6ff3edb0  
Give Help! T.J. Houshmandzadeh deh  
abduallah@abduallah-VirtualBox:~/sist_prog/cs360-lecture-notes/Strings-In-C/src$
```

❑ *adres yerleşimlerine dikkat !!

```
UNIX> bin/makej 50
|||||
UNIX> bin/strcat3 50
|||||
UNIX>
```

String-strcat()

- ❑ C tarzı string işlemleri, C++ tarzı string işlemlere göre biraz daha zordur.
- ❑ Örneğin, belirli sayıda **j** içeren bir dizi oluşturmak istediğinizi varsayalım.
- ❑ C’de diziye tahsis etmek için önce malloc()’u **çağırmanız gerektiğinden** bu biraz daha zordur.

```

/* Trying to use strcat() like C++ string concatenation. */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv)
{
    char *s;
    int i;
    int n;

    if (argc != 2) { fprintf(stderr, "usage: strcat3 number\n"); exit(1); }

    n = atoi(argv[1]);
    s = (char *) malloc(sizeof(char)*(n+1));
    strcpy(s, "");

    for (i = 0; i < n; i++) strcat(s, "j"); /* Here's the strcat() call, wh

    printf("%s\n", s);
    return 0;
}

```

```
/* Create a string with a given number of j's by using string concatenation. */

#include <iostream>
#include <cstdio>
#include <cstdlib>
using namespace std;

int main(int argc, char **argv)
{
    int i, n;
    string s;

    if (argc != 2) { fprintf(stderr, "usage: makej number\n"); exit(1); }
    n = atoi(argv[1]);

    for (i = 0; i < n; i++) s += "j";    // Here is the string concatenation.
    cout << s << endl;
    return 0;
}
```

String-strcat()

- ❑ Gerçekten büyük bir sayı üzerinde deneyelim.
- ❑ Burada, standart çıktıyı /dev/null'a yönlendirelim,

```
UNIX> time sh -c "bin/makej 1000 > /dev/null"
0.002u 0.004s 0:00.01 0.0% 0+0k 0+0io 0pf+0w # Blink of an eye.
UNIX> time sh -c "bin/makej 10000 > /dev/null"
0.002u 0.004s 0:00.00 0.0% 0+0k 0+0io 0pf+0w # Blink of an eye.
UNIX> time sh -c "bin/makej 100000 > /dev/null"
0.004u 0.004s 0:00.01 0.0% 0+0k 0+0io 0pf+0w # Blink of an eye.
UNIX> time sh -c "bin/strcat3 1000 > /dev/null"
0.002u 0.004s 0:00.00 0.0% 0+0k 0+0io 0pf+0w # Blink of an eye.
UNIX> time sh -c "bin/strcat3 10000 > /dev/null"
0.039u 0.004s 0:00.04 75.0% 0+0k 0+0io 0pf+0w # A little slower
UNIX> time sh -c "bin/strcat3 100000 > /dev/null"
3.468u 0.005s 0:03.47 99.7% 0+0k 0+0io 0pf+0w # Nearly 100 times slower!
UNIX>
```

- ❑ >
- ❑ Redirection-Yönlendirmeler, ileride detaylı olarak bahsedilecek

- ❑ /dev/null
- ❑ Her bir Linux sisteminde bulunan özel bir dosyadır. Ancak, diğer birçok sanal dosyanın aksine, okumak yerine yazmak için kullanılır. /dev/null'a yazdığınız her şey atılır (çöp kutusu gibi). Bir UNIX sisteminde boş aygıt olarak bilinir.

String-strcat()

- ❑ Sorunu görüyor musunuz? C++ dizesi, dizinin uzunluğunu korur, bu nedenle birleştirme hızlıdır.
- ❑ Aksine, strcat() her çağrıda dizgenin sonunu bulmak zorundadır, bu da programı $O(n^2)$ yapar.
- ❑ String'in sonunun nerede olduğunu bildiğimiz için düzeltebiliriz. **(s+i)**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char **argv)
{
    char *s;
    int i;
    int n;

    if (argc != 2) { fprintf(stderr, "usage: strcat4 number\n"); exit(1); }

    n = atoi(argv[1]);
    s = (char *) malloc(sizeof(char)*(n+1));
    strcpy(s, "");

    for (i = 0; i < n; i++) strcat(s+i, "j"); /* The only changed line */

    printf("%s\n", s);
    return 0;
}
```

```
UNIX> bin/strcat4 50 jooooooooooooooooooooooooooooooooooooooooooooo
UNIX> time sh -c "bin/strcat4 100000 > /dev/null"
0.003u 0.004s 0:00.01 0.0% 0+0k 0+0io 0pf+0w
UNIX>
```

- **command > file**: Sends standard output to **<file>**
- **command < input**: Feeds a command input from **<input>**

```
$ command1 | command2 | command3
```

*Bir komutu diğerine "iletmek" için

String-strlen()

- Strlen(), s'nin boş sonlandırılmış bir dize olduğunu varsayar. Boş karakterden önceki karakter sayısını döndürür. Yani uzunluğunu!

```
#include <stdio.h>
#include <string.h>

int main()
{
    char give[5];
    char him[5];
    char six[5];

    strcpy(give, "Give");
    strcpy(him, "Him");
    strcpy(six, "Six!");

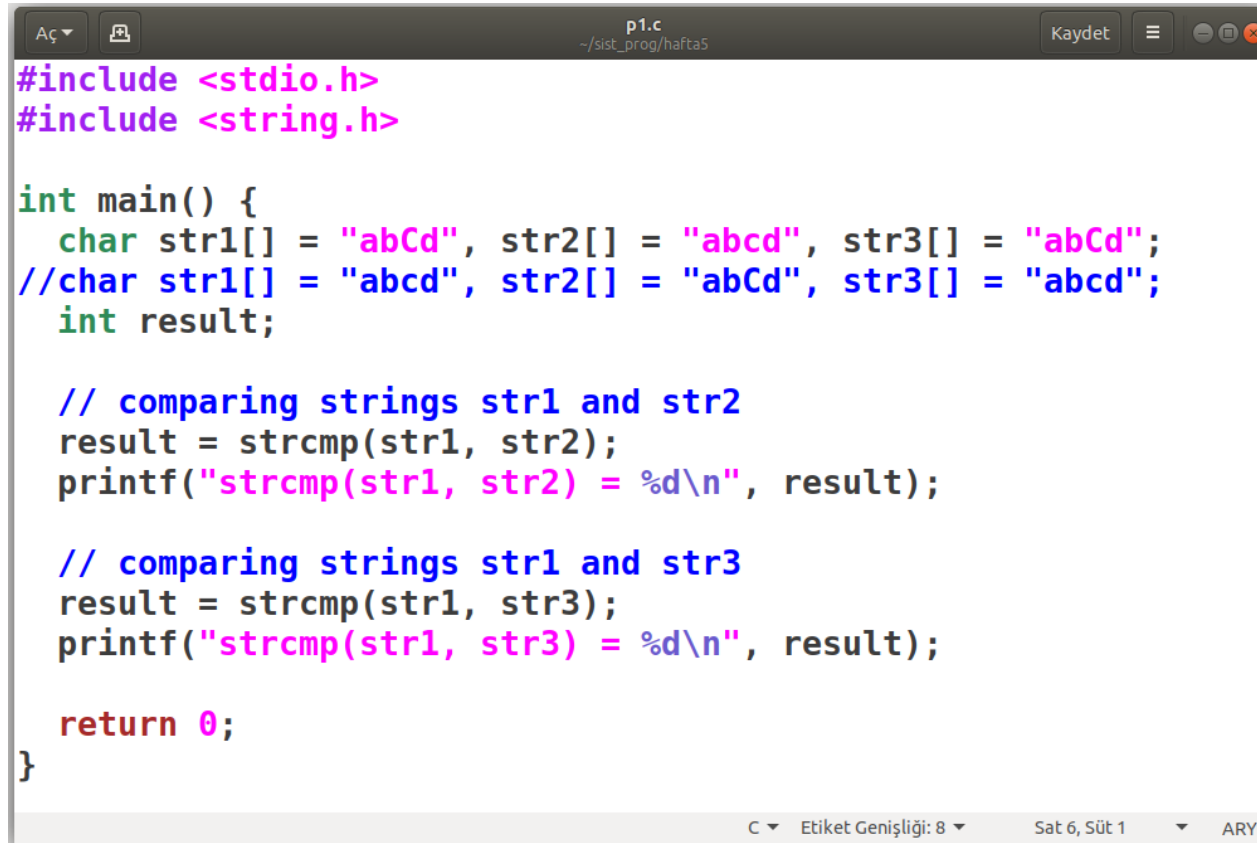
    printf("%s %s %s\n", give, him, six);
    printf("%ld %ld %ld\n", strlen(give), strlen(him), strlen(six));
    return 0;
}
```

UNIX> **bin/strlen**
Give Him Six!
4 3 4

String-strcmp() ve strncmp()

- ❑ **Strcmp()**, iki string ifadenin sözlüksel bir karşılaştırmasını gerçekleştir.
 - ❑ *Eşitlerse 0,*
 - ❑ *s1 s2'den küçükse negatif,*
 - ❑ *aksi takdirde pozitif bir sayı döndürür.*
- ❑ Oldukça fazla strcmp() kullanacağız, çünkü iki stringi karşılaştırmanın en kolay yolu bu.
- ❑ **Strncmp()**, boş karaktere henüz ulaşılmadıysa, n karakterden sonra karşılaştırmayı durdurur

String-strcmp() ve strncmp()



```
p1.c
~/sist_prog/hafta5
Kaydet

#include <stdio.h>
#include <string.h>

int main() {
    char str1[] = "abCd", str2[] = "abcd", str3[] = "abCd";
    //char str1[] = "abcd", str2[] = "abCd", str3[] = "abcd";
    int result;

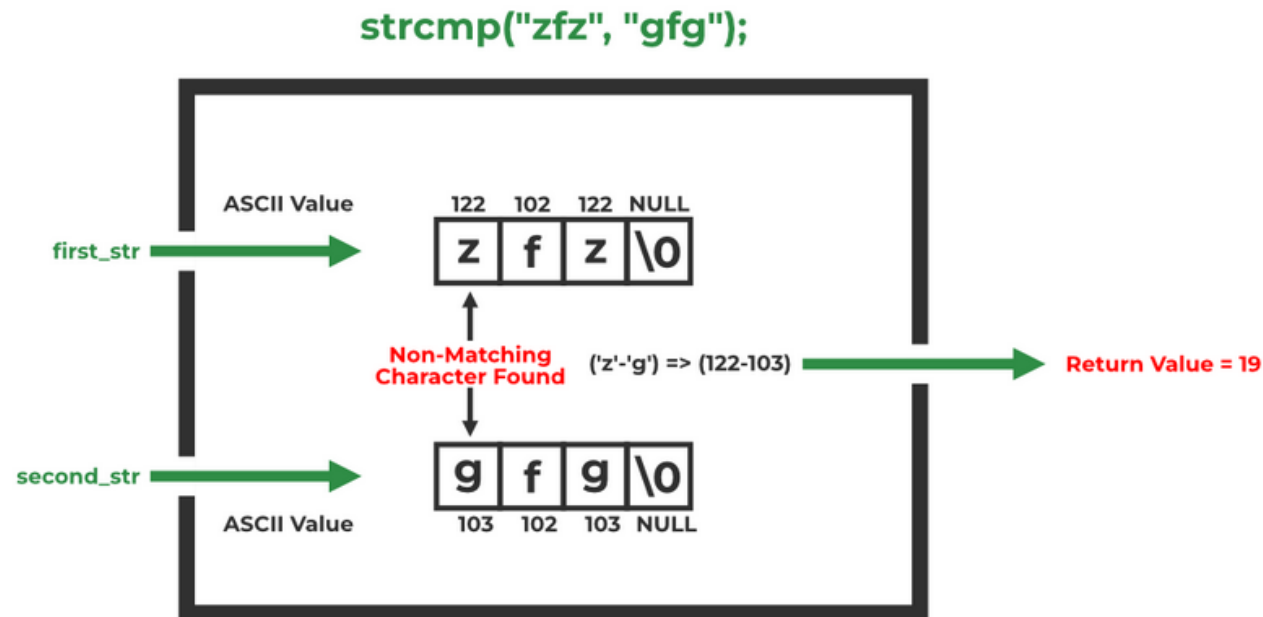
    // comparing strings str1 and str2
    result = strcmp(str1, str2);
    printf("strcmp(str1, str2) = %d\n", result);

    // comparing strings str1 and str3
    result = strcmp(str1, str3);
    printf("strcmp(str1, str3) = %d\n", result);

    return 0;
}
```

C Etiket Geniřlięi: 8 Sat 6, Süt 1 ARY

String-strcmp() ve strncmp()



<https://www.geeksforgeeks.org/strcmp-in-c/>

String-strncmp() ve strncmp()

```
#include <stdio.h>
#include <string.h>

int main () {
    char str1[20];
    char str2[20];
    int result;

    strcpy(str1, "hello");
    strcpy(str2, "hello WORLD");

    //This will compare the first 4 characters
    result = strncmp(str1, str2, 4);

    if(result > 0) {
        printf("ASCII value of first unmatched character of str1 is greater than str2");
    } else if(result < 0) {
        printf("ASCII value of first unmatched character of str1 is less than str2");
    } else {
        printf("Both the strings str1 and str2 are equal");
    }

    return 0;
}
```

String-strchr()

char *strchr(const char *s, int c);

❑ **Strchr()**, C string'lerinde tek karakterler için "**bulma**" işlemidir.

❑ c bir tamsayıdır, ancak bir karakter olarak ele alınır. Strchr(), s cinsinden c'ye eşit karakterin ilk geçtiği yere bir işaretçi döndürür. s, c'yi içermiyorsa, NULL döndürür.

```
/* Use strchr() to determine if each line of standard input has a space. */  
  
#include <stdio.h>  
#include <string.h>  
  
int main()  
{  
    char line[100];  
    char *ptr;  
  
    while (fgets(line, 100, stdin) != NULL) {  
        ptr = strchr(line, ' ');  
        if (ptr == NULL) {  
            printf("No spaces\n");  
        } else {  
            printf("Space at character %ld\n", ptr-line);  
        }  
    }  
    return 0;  
}
```

UNIX> **bin/strchr Jim**

No spaces

Jim Plank Space at character 3

James Plank Space at character 5

HI! Space at character 0

HI!! Space at character 0 <

CNTL-D>

UNIX>

String-strchr()

□ Biraz modifikasyon ile bütün karakterlerin yerlerini gösterebiliriz;

```
#include <stdio.h>
#include <string.h>

int main()
{
    char line[100];
    char *ptr;

    while (fgets(line, 100, stdin) != NULL) {
        ptr = strchr(line, ' ');
        if (ptr == NULL) {
            printf("No spaces\n");
        } else {
            while (ptr != NULL) {
                printf("Space at character %ld\n", ptr-line);
                ptr = strchr(ptr+1, ' ');
            }
        }
    }
    return 0;
}
```

```
UNIX> bin/strchr2
Jim
No spaces
Jim Plank
Space at character 3
Jim Plank
Space at character 3
Space at character 4
    Give Him Six!!!
Space at character 0
Space at character 1
Space at character 6
Space at character 7
Space at character 8
Space at character 12
Space at character 13
Space at character 14
<CNTL-D>
UNIX>
```

String-scanf()

- ❑ `scanf()`, `printf()` gibidir. Ancak parametreleri uçbirime yazmak yerine uçbirimden okur (veya standart girdi ne ise).
- ❑ `scanf()`'in insanların kafasını karıştırdığı nokta, C'de referans değişkenleri olmamasıdır, dolayısıyla işaretçiler kullanmanız gerekir.
- ❑ Biçim dizgisine `"%d"` yazarsanız, `scanf()` bir tamsayı okuyacaktır. Geçmeniz gereken parametre, okumak istediğiniz tamsayıya bir işaretçidir.
- ❑ Tamsayı için depolama alanı mevcut olmalıdır. `scanf()`, tamsayıyı standart girdiden okuyacak ve tamsayının dört baytını dolduracaktır.

String-scanf()

□ scanf()'i çağırdığımızda, standart girdiden bir tamsayı okumasını ve bu dört baytı o tamsayı ile doldurmasını söylüyorum. Scanf() yaptığı başarılı okumaların sayısını döndürür.

```
/* Read a single integer from standard input using scanf. */

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    if (scanf("%d", &i) == 1) {
        printf("Just read i: %d (0x%x)\n", i, i);
    } else {
        printf("Scanf() failed for some reason.\n");
    }
    exit(0);
}
```

```
UNIX> bin/scanf1
10
Just read i: 10 (0xa)
UNIX> bin/scanf1
Fred
Scanf() failed for some reason.
UNIX> bin/scanf1
15.999999999999999
Just read i: 15 (0xf)
UNIX> bin/scanf1
-15.999999999999999
Just read i: -15 (0xffffffff1)
UNIX> bin/scanf1
<CNTL-D>
Scanf() failed for some reason.
UNIX> echo "" | bin/scanf1
Scanf() failed for some reason.
UNIX> echo 15fred | bin/scanf1
Just read i: 15 (0xf)
UNIX>
```

String-scanf()

- ❑ Derlenir (her ne kadar bazı derleyiciler hatalı olduğunu anlayabilir ve size uyarabilir).
- ❑ Hatanın ortaya çıkıp çıkmaması bir şans meselesidir. ***i başlatılmayınca hata**

```
int main()
{
    int *i;

    printf("i = 0x%lx\n", (unsigned long) i);
    if (scanf("%d", i) == 1) {
        printf("Just read i: %d (0x%x)\n", *i, *i);
    } else {
        printf("Scanf() failed for some reason.\n");
    }
    exit(0);
}
```

```
UNIX> echo 10 | bin/scanf2
i = 0x7fff5fc01052
Bus error
UNIX>
```

String-scanf()

- ❑ Aşağıdaki program (src/scanf3.c), standart girdiden bir dize okumak ve ardından tek tek karakterleri yazdırmak için scanf()'i kullanır:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char s[10];
    int i;

    if (scanf("%s", s) != 1) exit(0);

    for (i = 0; s[i] != '\0'; i++) {
        printf("Character: %d: %3d %c\n", i, s[i], s[i]);
    }
    exit(0);
}
```

```
UNIX> echo "Jim-Plank" | bin/scanf3
Character: 0:  74 J
Character: 1: 105 i
Character: 2: 109 m
Character: 3:  45 -
Character: 4:  80 P
Character: 5: 108 l
Character: 6:  97 a
Character: 7: 110 n
Character: 8: 107 k
UNIX>
```


String-scanf()

- ❑ String'ler ile Scanf() sorunludur. Özellikle, 10'dan fazla karakter içeren bir string girdiğinizde ne olacağını düşünün.
- ❑ Bellek, tıpkı yukarıdaki "T. J. Houshmanzadeh" ile strcpy() ve strcat() örneklerinde olduğu gibi zorlanacaktır.
- ❑ Örneğin bin/scanf3'e 80.000 'j' karakterli bir dizi gönderelim:

```
UNIX> bin/makej 80000 | bin/scanf3  
Segmentation fault: 11  
UNIX>
```

String-Sscanf()

- ❑ Sscanf(), ilk parametresi olarak ek bir string alması ve standart girdi yerine bu string'den "okuması" dışında, tıpkı scanf() gibidir.
- ❑ Yaptığı doğru eşleşmelerin sayısını döndürür. Bu nedenle, stringleri tamsayılara ve double değişkene dönüştürmek için oldukça uygundur.

```
#include <stdio.h>

int main()
{
    char buf[1000];
    int i, h;
    double d;

    while (fgets(buf, 1000, stdin) != NULL) {
        if (sscanf(buf, "%d", &i) == 1) {
            printf("When treated as an integer, the value is %d\n", i);
        }
        if (sscanf(buf, "%x", &h) == 1) {
            printf("When treated as hex, the value is 0x%x (%d)\n", h, h);
        }
        if (sscanf(buf, "%lf", &d) == 1) {
            printf("When treated as a double, the value is %lf\n", d);
        }
        if (sscanf(buf, "0x%x", &h) == 1) {
            printf("When treated as a hex with 0x%x formatting, the value is 0x%x (%d)\n", h, h);
        }
        printf("\n");
    }
}
```

UNIX> bin/sscanf1

10

When treated as an integer, the value is 10

When treated as hex, the value is 0x10 (16)

When treated as a double, the value is 10.000000

55.9

When treated as an integer, the value is 55

When treated as hex, the value is 0x55 (85)

When treated as a double, the value is 55.900000

.5679

When treated as a double, the value is 0.567900

a

When treated as hex, the value is 0xa (10)

0x10

When treated as an integer, the value is 0

When treated as hex, the value is 0x10 (16)

When treated as a double, the value is 16.000000

When treated as a hex with 0x%x formatting, the value is 0x10 (16)

UNIX>

String-Strdup()

- ❑ Bellek ayırarak dizenin bir kopyasını oluşturur.
- ❑ Malloc()'u çağırdığı için, eğer kopyayla işiniz bittiyse, bellek sızıntılarını önlemek için sonrasında free()'i çağırmanız gerekir.

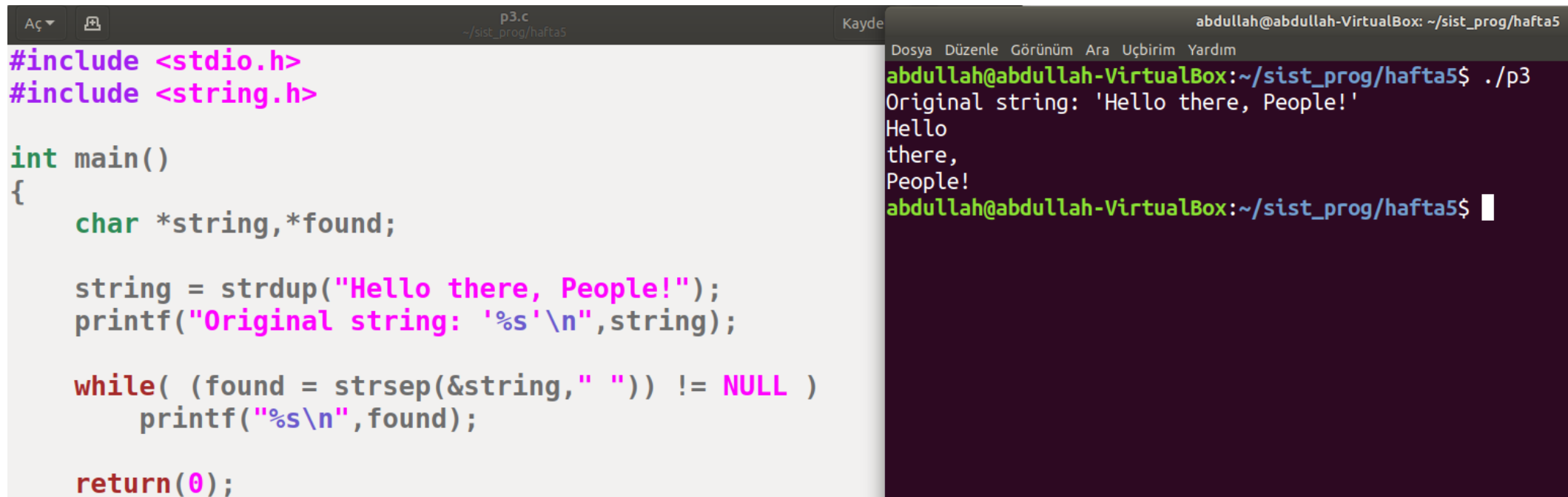
```
char *strdup(const char *s);
```

```
char *strdup(const char s)
{
    return strcpy(malloc(strlen(s)+1), s);
}
```

Diğerleri

- **strrchr()** bir karakterin son geçtiği yeri bulur
- **strstr()** bir alt-string (Substring) bulur.
- **strcasestr()** bir alt-string (Substring) bulur ancak büyük/küçük harf durumunu yok sayar.
- **strsep()**, sınırlayıcılarla dizeleri ayırmanıza yardımcı olur.
- **strncpy()** kısıtlı bir strcpy yapar.
- **memcpy()**, belleğin bir bölgesini diğerine kopyalar.
- **memcmp()**, belleğin iki bölgesinin bayt bayt karşılaştırmasını yapar.
- **bzero()**, her bayt sıfır olacak şekilde bir bellek bölgesi ayarlar.

Strsep()



The image shows a C program in a text editor and its execution in a terminal. The program, named p3.c, uses the strsep() function to split a string. The terminal output shows the original string and the words 'Hello', 'there,', and 'People!' printed on separate lines.

```
p3.c
~/sist_prog/hafta5
Aç Kayde

#include <stdio.h>
#include <string.h>

int main()
{
    char *string,*found;

    string = strdup("Hello there, People!");
    printf("Original string: '%s'\n",string);

    while( (found = strsep(&string," ")) != NULL )
        printf("%s\n",found);

    return(0);
}
```

```
abduallah@abduallah-VirtualBox: ~/sist_prog/hafta5
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~/sist_prog/hafta5$ ./p3
Original string: 'Hello there, People!'
Hello
there,
People!
abduallah@abduallah-VirtualBox:~/sist_prog/hafta5$
```

Strstr()

```
int main()
{
    // Take any two strings
    char s1[] = "Home Sweet Home";
    char s2[] = "Sweet";
    char* p;

    // Find first occurrence of s2 in s1
    p = strstr(s1, s2);

    // Prints the result
    if (p) {
        printf("String found\n");
        printf("First occurrence of string '%s' in '%s' is '%s'", s2, s1, p);
    } else
        printf("String not found\n");

    return 0;
}
```

```
abduallah@abduallah-VirtualBox: ~/sist_prog/hafta5
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~/sist_prog/hafta5$ ./p4
String found
First occurrence of string 'Sweet' in 'Home Sweet Home' is 'Sweet Home'
abduallah@abduallah-VirtualBox:~/sist_prog/hafta5$
```

Soru

1-) Terminalden 2 cümle giriyoruz (scanf). Karşılaştıracacağız (strcmp). Alfabetik olarak önce gelen cümlede boşlukların yerlerini yazdıralım (strchr).

2-) Scanf ile terminalden 3 tane kelime okuyalım . Alfabetik sıraya göre sıralayıp (pointer) strcat ile birleştir ekrana yaz.

Sistem Programlama

DR. ÖĞR. ÜYESİ ABDULLAH SEVİN

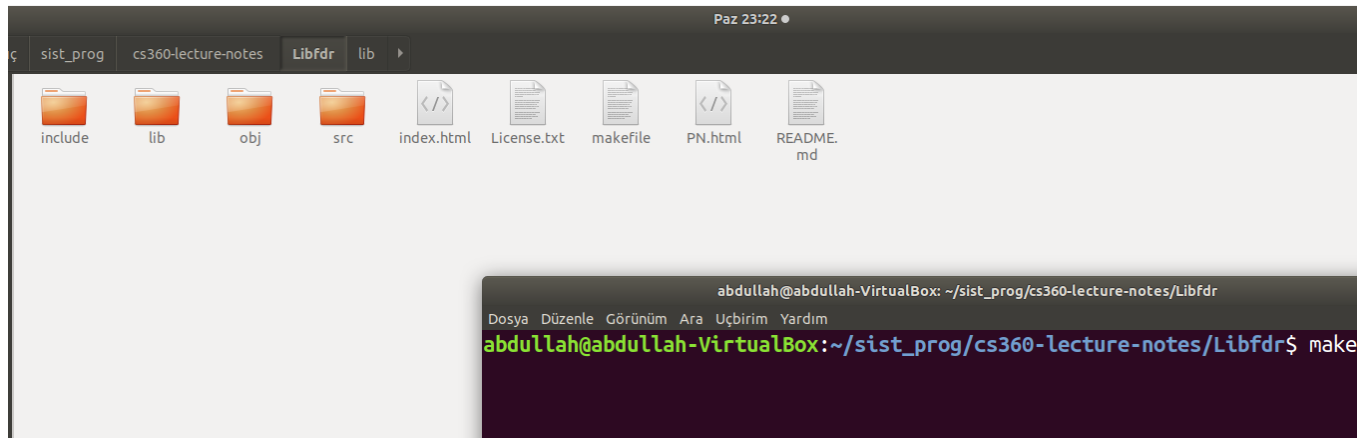
İçerik

a) **Libfdr: Jvals, Fields, Dllists, Red-Black Trees**

- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/Libfdr/index.html>
- <http://web.eecs.utk.edu/~jplank/plank/classes/cs360/360/notes/Fields/index.html>

Jvals, Fields, Dllists, Red-Black Trees

- ❑ **Fields** - girdi işlemeyi basitleştiren bir kitaplık
- ❑ **Jvals** - genel bir veri türü
- ❑ **Dllists** - çift yönlü bağlı listeler için bir kitaplık
- ❑ **Red-Black Trees** - kırmızı-siyah ağaçlar için bir kitaplık
- ❑ Kütüphanemizi derlemek için dizinde make yapıyoruz



makefile

```
makefile
~/sist_prog/cs360-lecture-notes/libfdr

# Libraries for fields, doubly-linked lists and red-black trees.
# Copyright (C) 2018 James S. Plank

CFLAGS = -O3 -Iinclude

all: lib/libfdr.a

OBJS = obj/dllist.o obj/fields.o obj/jval.o obj/jrb.o

lib/libfdr.a: $(OBJS)
    ar ru lib/libfdr.a $(OBJS)
    ranlib lib/libfdr.a

clean:
    rm -f obj/* lib/*

obj/fields.o: src/fields.c include/fields.h
    gcc $(CFLAGS) -c -o obj/fields.o src/fields.c

obj/jval.o: src/jval.c include/jval.h
    gcc $(CFLAGS) -c -o obj/jval.o src/jval.c

obj/dllist.o: src/dllist.c include/dllist.h include/jval.h
    gcc $(CFLAGS) -c -o obj/dllist.o src/dllist.c
```

<https://www.mkssoftware.com/docs/man1/ar.1.asp>

<https://www.geeksforgeeks.org/ranlib-command-in-linux-with-examples/>

Fields

- ❑ **Fields** kitaplığı, girdi okumayı `getchar()`, `scanf()` veya `fgets()` kullanmaktan daha kolay hale getiren bir rutinler paketidir.
- ❑ Kaynaktaki yazarlar tarafından yazılan bir kitaplık -- Unix'te standart değil ama herhangi bir C derleyicisi ile çalışması gerekir (buna DOS/Windows da dahildir).
- ❑ Kaynak kodu bu depoda, "Libfdr" dizinindedir.
- ❑ Bu sınıftaki **Fields** prosedürlerini kullanmak için, `fields.h` dosyasını eklemelisiniz.
- ❑ Tam yol adını C dosyanıza dahil etmek yerine şunları yapın:

```
#include "fields.h"
```

fields.h

```
/* The fields library -- making input processing easier */

#include <stdio.h>
#define MAXLEN 1001
#define MAXFIELDS 1000

typedef struct inputstruct {
    const char *name;          /* File name */
    FILE *f;                   /* File descriptor */
    int line;                   /* Line number */
    char text1[MAXLEN];        /* The line */
    char text2[MAXLEN];        /* Working -- contains fields */
    int NF;                     /* Number of fields */
    char *fields[MAXFIELDS];   /* Pointers to fields */
    int file;                   /* 1 for file, 0 for popen */
} *IS;

extern IS new_inputstruct(const char *filename); /* Use NULL for stdin. Returns NULL on failure. */
extern IS pipe_inputstruct(const char *shell_command); /* Returns NULL on failure. */
extern int get_line(IS inputstruct); /* returns NF, or -1 on EOF. */
extern void jettison_inputstruct(IS inputstruct); /* frees the IS and fclose/pclose the file */
#endif
```

Fields

- ❑ **Fields** kitaplığıyla bir dosyayı okumak için new_inputstruct() ögesini çağırırsınız.
- ❑ New_inputstruct(), giriş parametresi olarak dosya adını alır (standart girdi için NULL) ve sonuç olarak bir IS döndürür.
- ❑ IS'nin bir struct inputstruct için bir işaretçi olduğuna dikkat edin.
- ❑ Bu, new_inputstruct() çağırısında malloc()' tanımlaması mevcuttur.

```
#define talloc(ty, sz) (ty *) malloc (sz * sizeof(ty))
```
- ❑ new_inputstruct() dosyayı açamıyorsa, NULL döndürür ve hatanın nedenini yazdırmak için perror()'u arayabilirsiniz (bu konuda bilgi edinmek istiyorsanız, perror()'daki man sayfasını okuyun).

Fields

❑ IS yapısında, bir satırı okumak için `get_line()`'ı çağırırsınız. `Get_line()`, satırın okunmasını yansıtmak için IS'nin durumunu değiştirir. Özellikle:

1. Satırın içeriğini `text1`'e koyar.
2. Satırı kelimelere ayırır. `NF field`, `field`'deki kelime sayısını içerir. `Fields` dizisinin `NF` slotları, `NF` sözcüklerinin her birine işaret eder (ve bu sözcükler boş sonlandırılmıştır).
3. `line field`, satırın satır numarasını içerir.
4. `Get_line()`, dönüş değeri olarak `NF field` döndürür.
5. Dosyanın sonuna geldiğinde `-1` döndürür.

❑ `Jettison_inputstruct()`, IS ile ilişkili dosyayı kapatır ve IS'yi serbest bırakır (serbest bırakır).

src/printwords.c

```
/* Use the fields library to print each word on standard input, labeled with its line number. */
#include <stdio.h>
#include <stdlib.h>
#include "fields.h"

int main(int argc, char **argv)
{
    IS is;
    int i;

    if (argc != 2) { fprintf(stderr, "usage: printwords filename\n"); exit(1); }

    /* Open the file as an inputstruct. Error check. */

    is = new_inputstruct(argv[1]);
    if (is == NULL) {
        perror(argv[1]);
        exit(1);
    }

    /* Read each line with get_line(). Print out each word. */

    while(get_line(is) >= 0) {
        for (i = 0; i < is->NF; i++) {
            printf("%d: %s\n", is->line, is->fields[i]);
        }
    }

    /* Free up the memory allocated with new_inputstruct, and
       close the open file. This is not necessary in this program,
       since we are exiting anyway, but I just want to show how you free it up. */

    jettison_inputstruct(is);
    return 0;
}
```

- Bu prosedürler, girdi dosyalarını işlemek için çok uygundur. Örneğin, aşağıdaki program (src/printwords.c'de) bir girdi dosyasındaki her kelimeyi satır numarasının başına gelecek şekilde yazdırır.

```
UNIX> bin/printwords txt/rex-1.txt
1: June:
1: Hi
1: ...
1: I
1: missed
1: you!
2: Rex:
2: Same
2: here!
2: You're
2: all
2: I
2: could
2: think
2: about!
3: June:
3: I
3: was?
UNIX>
```


stderr

- ❑ C programlama dilinde, standart çıktı olarak da bilinen farklı dosya tanımlayıcıları vardır.
- ❑ Standart giriş için stdin,
- ❑ standart çıkış için stdout ve
- ❑ hata mesajı çıkışı için stderr olmak üzere 3 standart I/O yöntemi vardır.

```
#include <stdio.h> int main() {  
    fprintf(stdout, "This is message 1\n");  
    fprintf(stderr, "This is message 2\n");  
    fprintf(stdout, "This is message 3\n");  
    return(0); }
```

```
This is message 1  
This is message 2  
This is message 3
```

stdin

```
#include <stdio.h>
int main( ) {

    char a[100];

    fprintf( "Enter a string :");
    fscanf(stdin,"%s", a);

    fprintf( stdout,"\nYou entered the following string: %s ", a);

    fprintf(stdout,"\n");
    return 0;

}
```

fgets

```
char *fgets(char *str, int n, FILE *stream)
```

- ❑ Parametreler
- ❑ **str** - Bu, okunan string'in depolandığı bir karakter dizisinin işaretçisidir.
- ❑ **n** – Bu, okunacak maksimum karakter sayısıdır (son boş karakter dahil). Genellikle str olarak geçirilen dizinin uzunluğu kullanılır.
- ❑ **stream** – Bu, karakterlerin okunduğu akışı tanımlayan bir FILE nesnesinin işaretçisidir.

Fields

- ❑ Fields kitaplığıyla ilgili dikkat edilmesi gereken önemli bir nokta, malloc()'un çağrıldığı tek zamanın new_inputstruct() sırasında olmasıdır.
- ❑ Get_line() sadece IS yapısının alanlarını doldurur --- bellek tahsisi gerçekleştirmez.
- ❑ Bu, bir satırı veya alanı depolamak ve sonraki get_line() çağrısının üzerine yazılmasını istemiyorsanız, genellikle strdup() ile bunun bir kopyasını oluşturmanız gerektiği anlamına gelir.
- ❑ Bu çok önemli, genel olarak fgets() ile yapılan en yaygın hata, bir kopyaya ihtiyaç duyduklarında bir kopya çıkarmamaktır.
- ❑ İşaretçiler ve malloc() konusunda size yardımcı olacak bu hatayı burada göstereceğiz.

Fields

- ❑ Amacımız, standart girdinin son n satırını basan program kuyruğunu yazmak olacaktır.
- ❑ n'nin değeri varsayılan olarak 10'dur, ancak bunu komut satırında belirtebilmelisiniz.
- ❑ Fields kitaplığını kullanarak son 10 satırı yazdırmaya çalışacak olan src/tail10-bad.c yazarak başlayalım.
- ❑ Bu, yukarıda bahsedilen yaygın hatayı gösterecektir.
- ❑ İşte oldukça basit olan kod. 10 karakterden oluşan bir * dizimiz olacak, her satırı okuduğumuz zaman basitçe is->text1 olarak ayarlayacağız:

src/tail10-bad.c,

```
/* A buggy program to print the last 10 lines of standard input. */

#include <stdio.h>
#include <stdlib.h>
#include "fields.h"

int main(int argc, char **argv)
{
    IS is;
    int i, n;
    char *lines[10];    /* This array will hold the last 10 lines of standard input. */

    /* Read the lines of standard input, and only keep the last ten. */

    is = new_inputstruct(NULL);
    n = 0;
    while (get_line(is) >= 0) {
        lines[n%10] = is->text1;    /* This is the bad line -- it doesn't copy the string. */
        n++;
    }

    /* Print the last 10 lines, or fewer if there are fewer lines.
       Remember that is->text1 has a newline at the end. */

    i = (n >= 10) ? (n-10) : 0;    /* This is the line number of the 10th line from the end. */
    for ( ; i < n; i++) printf("%s", lines[i%10]);    /* Print this line to the last line. */

    return 0;
}
```

src/tail10-bad.c,

<pre>UNIX> cat txt/tail-input-15.txt 1 Elijah Christian Shatterproof 2 Cameron Ostracod 3 Ryan Sargent 4 Christopher Tempest 5 Aiden Circumferential 6 Carson Carcass 7 Caroline Jazz 8 Molly Jade 9 Jordan Equivalent MD 10 Aaron Nagging 11 Isaac Bandwidth 12 Leah Bulk 13 Victoria Glutamate 14 Lucas Workmen 15 Sofia Godlike UNIX></pre>	<pre>UNIX> bin/tail10-bad < txt/tail-input-15.txt 15 Sofia Godlike 15 Sofia Godlike 15 Sofia Godlike 15 Sofia Godlike 15 Sofia Godlike 15 Sofia Godlike 15 Sofia Godlike 15 Sofia Godlike 15 Sofia Godlike 15 Sofia Godlike UNIX></pre>
---	--

- ❑ Cat komutu, Herhangi bir seçenek belirtmeden komutun kendisi bir dosyanın içeriklerini okuyacak ve konsolda gösterecektir.
- ❑ Cat komutu, standart çıktı ekranında dosyalar oluşturmanıza, birleştirmenize veya bastırmanıza izin verir

src/tail10-bad-print.c,

UNIX> `bin/tail10-bad-print < txt/tail-input-15.txt`

I have set lines[0] to 0x7fc014002614, which is currently	1	Elijah Christian Shatterproof
I have set lines[1] to 0x7fc014002614, which is currently	2	Cameron Ostracod
I have set lines[2] to 0x7fc014002614, which is currently	3	Ryan Sargent
I have set lines[3] to 0x7fc014002614, which is currently	4	Christopher Tempest
I have set lines[4] to 0x7fc014002614, which is currently	5	Aiden Circumferential
I have set lines[5] to 0x7fc014002614, which is currently	6	Carson Carcass
I have set lines[6] to 0x7fc014002614, which is currently	7	Caroline Jazz
I have set lines[7] to 0x7fc014002614, which is currently	8	Molly Jade
I have set lines[8] to 0x7fc014002614, which is currently	9	Jordan Equivalent MD
I have set lines[9] to 0x7fc014002614, which is currently	10	Aaron Nagging
I have set lines[0] to 0x7fc014002614, which is currently	11	Isaac Bandwidth
I have set lines[1] to 0x7fc014002614, which is currently	12	Leah Bulk
I have set lines[2] to 0x7fc014002614, which is currently	13	Victoria Glutamate
I have set lines[3] to 0x7fc014002614, which is currently	14	Lucas Workmen
I have set lines[4] to 0x7fc014002614, which is currently	15	Sofia Godlike
15 Sofia Godlike		
15 Sofia Godlike		
15 Sofia Godlike		
15 Sofia Godlike		
15 Sofia Godlike		
15 Sofia Godlike		
15 Sofia Godlike		
15 Sofia Godlike		
15 Sofia Godlike		
15 Sofia Godlike		

UNIX>

```
printf("I have set lines[%d] to 0x%lx, which is currently %s",
      n%10, (unsigned long) (lines[n%10]), lines[n%10]);
```


src/tail10-memory-leak.c,

```
is = new_inputstruct(NULL);
n = 0;
while (get_line(is) >= 0) {
    lines[n%10] = strdup(is->text1);    /* This is the only change - we call strdup(). */
    n++;
}
}
```

UNIX> bin/tail10-memory-leak < txt/tail-input-15.txt

```
6 Carson Carcass
7 Caroline Jazz
8 Molly Jade
9 Jordan Equivalent MD
10 Aaron Nagging
11 Isaac Bandwidth
12 Leah Bulk
13 Victoria Glutamate
14 Lucas Workmen
15 Sofia Godlike
```

UNIX>

- ❑ Basit düzeltme strdup() kullanmaktır.
- ❑ Bu, satırın bir kopyası için bellek ayıracak ve ardından satırı kopyalayacaktır.
- ❑ Kod src/tail10-memory-leak.c'de, adından da anlayabileceğiniz gibi, kendine ait bazı sorunları olacak. Tek değişiklik, artık satırları[n%10] is->text1'e atamamız, bunun yerine strdup() ile bir kopya oluşturmamızdır.

src/tail10-memory-leak.c,

- ❑ Aslında, çoğu girişte gayet iyi çalışır.
- ❑ Ancak adından da anlaşılacağı gibi bir bellek sızıntısı var.
- ❑ n , 10'dan büyük veya eşit olduğunda, `strdup()` satırı şu anda $[n\%10]$ satırlarında bulunan işaretçinin üzerine yazar ve işaretçi sonsuza dek kaybolur.
- ❑ Bununla birlikte, işaret ettiği bellek yine de tahsis edilmiştir ve programdan çıkılana kadar yeniden tahsis edilmeyecektir.
- ❑ Bellek sızıntısının tam tanımı budur.
- ❑ Bunu çok sayıda satır içeren bir girdi üzerinde çalıştırırsak, programın bellek kullanımı patlayacak ve sonunda makineniz durma noktasına gelecek ve/veya `strdup()` başarısız olduğunda sonlanacaktır.

src/tail10-memory-leak.c

- `UNIX> echo "" | awk '{ while (1) print "XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX" }' | bin/tail10-memory-leak &`
- awk script'i X'lerle sonsuz sayıda satır yazdırır

[illegible]

Then, I take a look at how the program is running with **top**:

```
UNIX> top
```

```
PID      COMMAND      %CPU  TIME      #TH   #WQ  #PORT MEM      PURG    CMPRS    PGRP    PPID    STATE
.....  All my running processes ....
```

Here is how the program is running at 6 seconds, 30 seconds and 60 seconds:

```

84909  tail10-memor 99.8  00:06.13 1/1  0    12    745M+ 0B    0B    84907 79428 running
...
84909  tail10-memor 99.8  00:30.87 1/1  0    12    3807M+ 0B    0B    8490  79428 running
...
84909  tail10-memor 99.8  01:00.83 1/1  0    12    7469M+ 0B    0B    84907 79428 running

```

src/tail10-good.c

- ❑ Öyleyse düzeltelim. strdup() malloc()'u çağırırız, böylece artık dizeye ihtiyacımız kalmadığında ve işaretçinin üzerine yazmak üzereyken dizeyi serbest bırakırız.

```
is = new_inputstruct(NULL);
n = 0;
while (get_line(is) >= 0) {
    if (n >= 10) free(lines[n%10]);    /* This line prevents the memory leak. */
    lines[n%10] = strdup(is->text1);
    n++;
}
```

PID	COMMAND	%CPU	TIME	#TH	#WQ	#PORT	MEM	PURG	CMPRS	PGRP	PPID	STATE
85101	tail10-good	99.7	00:07.44	1/1	0	12	492K	0B	0B	85099	79428	running
...												
85101	tail10-good	99.9	00:30.15	1/1	0	12	492K	0B	0B	85099	79428	running
...												
85101	tail10-good	99.9	01:00.11	1/1	0	12	492K	0B	0B	85099	79428	running

tail10-good print

```
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/Fields/bin$ ./tail10-good < ../txt/tail-input-15.txt
I have set lines[0] to 0x5556ea6719b0, which is currently 1 Elijah Christian Shatterproof
I have set lines[1] to 0x5556ea671df0, which is currently 2 Cameron Ostracod
I have set lines[2] to 0x5556ea671e20, which is currently 3 Ryan Sargent
I have set lines[3] to 0x5556ea671e40, which is currently 4 Christopher Tempest
I have set lines[4] to 0x5556ea671e70, which is currently 5 Aiden Circumferential
I have set lines[5] to 0x5556ea671ea0, which is currently 6 Carson Carcass
I have set lines[6] to 0x5556ea671ec0, which is currently 7 Caroline Jazz
I have set lines[7] to 0x5556ea671ee0, which is currently 8 Molly Jade
I have set lines[8] to 0x5556ea671f00, which is currently 9 Jordan Equivalent MD
I have set lines[9] to 0x5556ea671f30, which is currently 10 Aaron Nagging
I have set lines[0] to 0x5556ea671f50, which is currently 11 Isaac Bandwidth
I have set lines[1] to 0x5556ea671f70, which is currently 12 Leah Bulk
I have set lines[2] to 0x5556ea671df0, which is currently 13 Victoria Glutamate
I have set lines[3] to 0x5556ea671e20, which is currently 14 Lucas Workmen
I have set lines[4] to 0x5556ea671f90, which is currently 15 Sofia Godlike
6 Carson Carcass
7 Caroline Jazz
8 Molly Jade
9 Jordan Equivalent MD
10 Aaron Nagging
11 Isaac Bandwidth
12 Leah Bulk
13 Victoria Glutamate
14 Lucas Workmen
15 Sofia Godlike
```

tailanyf

Komut satırında satır sayısını belirttiğiniz tail'in genel versiyonudur.

Bu program, alışmanız gereken birkaç şeyi göstermektedir:

1. Bir string'i tamsayıya dönüştürmek ve çalışıp çalışmadığını test etmek için sscanf() işlevini kullanma.
2. Çalışma zamanına kadar boyutu bilinmeyen bir diziye ayırmak için malloc() kullanma.
3. Fields kitaplığını kullanma ve bir dizenin bir kopyasını depolamak için strdup() kullanma.

tailanyf

```
/* This program is more like tail -- it takes the number of lines, n,
   as a command line argument, and prints the last n lines of standard input. */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "fields.h"

int main(int argc, char **argv)
{
    char **lastn;
    int nlines, i, n;
    IS is;

    /* Error check the command line. */

    if (argc != 2) { fprintf(stderr, "usage: tailany1 n\n"); exit(1); }
    if (sscanf(argv[1], "%d", &n) == 0 || n <= 0) {
        fprintf(stderr, "usage: tailany1 n\n");
        fprintf(stderr, "      bad n: %s\n", argv[1]);
        exit(1);
    }

    /* Allocate the array */

    lastn = (char **) malloc(sizeof(char *)*n);
    if (lastn == NULL) { perror("malloc"); exit(1); }
```

```
    /* Allocate the array */

    lastn = (char **) malloc(sizeof(char *)*n);
    if (lastn == NULL) { perror("malloc"); exit(1); }

    /* Allocate the IS */

    is = new_inputstruct(NULL);
    if (is == NULL) { perror("stdin"); exit(1); }

    /* Read the input */

    nlines = 0;
    while (get_line(is) >= 0) {
        if (nlines >= n) free(lastn[nlines%n]); /* Prevent the memory leak. */
        lastn[nlines%n] = strdup(is->text1);
        nlines++;
    }

    /* Print the last n lines */

    i = (nlines < n) ? 0 : nlines-n;
    for (; i < nlines; i++) {
        printf("%s", lastn[i%n]);
    }

    /* Don't bother freeing stuff when you're just exiting anyway. */

    return 0;
}
```

tailanyf print

```
abdullah@abdullah-VirtualBox: ~/sist_prog/cs360-lecture-notes/Fields/bin
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/Fields/bin$ ./taila
nyf < ../txt/tail-input-15.txt 5
11 Isaac Bandwidth
12 Leah Bulk
13 Victoria Glutamate
14 Lucas Workmen
15 Sofia Godlike
```


pipe_inputstruct()

- ❑ Bu, popen() ile açılan bir kanaldan(pipe) okumanızı sağlar.
- ❑ src/pipetest.c programı, src dizinindeki tüm .c dosyalarındaki satır sayısını saymak için pipe_inputstruct() yöntemini kullanır.
- ❑ Bunu, "cat src/*.c" standart çıktısını bir inputstruct'a almak için pipe_inputstruct() kullanarak yapar:

pipe_inputstruct()

```
/* pipetest.c counts the number of lines in all the .c files in the
   src directory.  It does this by using pipe_inputstruct to get
   the standard output of the cat command into an inputstruct */

#include <stdio.h>
#include <stdlib.h>
#include "fields.h"

int main()
{
    IS is;
    int nlines;

    is = pipe_inputstruct("cat src/*.c");
    if (is == NULL) { perror("cat src/*.c"); exit(1); }

    nlines = 0;
    while (get_line(is) >= 0) nlines++;

    printf("# lines in src/*.c: %d\n", nlines);

    return 0;
}
```

pipetest print

```
abduallah@abduallah-VirtualBox: ~/sist_prog/cs360-lecture-notes/Fields/bin
Dosya Düzenle Görünüm Ara Uçbirim Yardım
abduallah@abduallah-VirtualBox:~/sist_prog/cs360-lecture-notes/Fields/bin$ ./pipetest
# lines in src/*.c: 226
abduallah@abduallah-VirtualBox:~/sist_prog/cs360-lecture-notes/Fields/bin$

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    IS is;
    int nlines;

    //is = pipe_inputstruct("cat src/*.c");
    is = pipe_inputstruct("cat ../src/*.c");
```

Soru

1-) istenilen dosyadan okunan son n satırdaki m adet kelimeyi yazdıran kod.